

signus

Blind PSK/QAM/FSK/Chirp Demodulator — 필사용 코드북

실행에 필요한 코드 전량 · 20개 파일 · 3,478줄 · 총 69쪽 | 2026-08-29 · 9744592

한 줄이 100칸을 넘으면 왼쪽 줄번호 자리에 ↵ 표시가 붙고 아랫줄로 이어집니다 — 원래는 한 줄이니 붙여서 입력하세요. 세로 점선은 들여쓰기 4칸 눈금입니다.

0 · 프로젝트 설정

□ <code>pyproject.toml</code>	패키지 정의 · 의존성 · ruff/pytest 설정	40줄	2쪽
□ <code>signus/__init__.py</code>	패키지 진입점	3줄	3쪽

1 · 신호 입출력과 기준표

□ <code>signus/sigio.py</code>	샘플 파일 I/O — 파일명이 fs·iq·real·샘플타임을 실어 나른다	180줄	4쪽
□ <code>signus/constellations.py</code>	성상도 · 그레이 비트 매핑 · FSK 레벨 (공용 기준표)	99줄	7쪽

2 · 스펙트럼 뷰

□ <code>signus/spectrum.py</code>	스펙트럼 · 위터폴 뷰 (표시 전용, 탐지엔 미사용)	38줄	9쪽
-----------------------------------	-------------------------------	-----	----

3 · 수신 DSP 코어

□ <code>signus/dsp.py</code>	수신 DSP 단계 (벡터화) — 이 프로젝트의 심장	390줄	10쪽
□ <code>signus/eq.py</code>	블라인드 심볼간격 FIR 등화기 — CMA 획득 후 판정지향 LMS	44줄	17쪽
□ <code>signus/sync.py</code>	블라인드 반복 프리앰블 동기 (정의 불요)	111줄	18쪽
□ <code>signus/classify.py</code>	결정층 — 변조 분류 · 회전 정렬 · 락 품질 · SNR	123줄	20쪽
□ <code>signus/_accel.py</code>	선택적 numba 가속 (없으면 동일한 순수 numpy로 풀백)	100줄	23쪽

4 · 변조별 수신 경로

□ <code>signus/fsk.py</code>	FSK/CPM 경로 — 정포락선 게이트 + 주파수 판별기 복조	181줄	25쪽
□ <code>signus/chirp.py</code>	선형 처프 / CSS(LoRa) 블라인드 탐지 · 특성화	97줄	29쪽

5 · 종단 파이프라인

□ <code>signus/pipeline.py</code>	블라인드 복조 종단 조립 — 두 계열의 진입점	419줄	31쪽
-----------------------------------	---------------------------	------	-----

6 · 사용자 인터페이스

□ <code>signus/cli.py</code>	CLI — analyze / serve (합성·채점 명령은 개발기 전용)	163줄	38쪽
□ <code>signus/server.py</code>	표준 라이브러리만 쓰는 웹 서버 + POST /api/analyze	88줄	41쪽

7 · 웹 프론트엔드

□ <code>signus/web/index.html</code>	UI 구조	153줄	43쪽
□ <code>signus/web/style.css</code>	UI 스타일	236줄	46쪽
□ <code>signus/web/app.js</code>	업로드 · 분석 요청 · 성상도/위터폴 캔버스 렌더링	612줄	51쪽

8 · 장비 프로브 (signus/ 옆에 같은 이름으로 저장)

□ <code>probes/sigc.py</code>	특징 4줄 + 단계 관찰 — 회신은 맥의 tools/sigc.py check	206줄	62쪽
□ <code>probes/sa.py</code>	종합 관찰: 스펙트로그램+검출 띠+버스트별 $x^2 \cdot x^4 \cdot x^8$ -AM 판독	195줄	66쪽

```
1  [build-system]
2  requires = ["setuptools>=68"]
3  build-backend = "setuptools.build_meta"
4
5  [project]
6  name = "signus"
7  version = "2.0.0"
8  description = "Blind PSK/QAM demodulator: auto parameter detection + constellation UI"
9  requires-python = ">=3.11"
10 dependencies = ["numpy>=1.26", "scipy>=1.11"]
11
12 [project.optional-dependencies]
13 dev = ["pytest>=8.0", "ruff>=0.4"]
14 # 필사 인쇄물 발급기(tools/codebook.py) 전용. 분석기 본체는 여전히 numpy+scipy 둘뿐이다.
15 tools = ["pygments>=2.17"]
16 # Optional JIT for the sequential equalizer/carrier loops (16-34x on those loops).
17 # Falls back to identical pure-numpy when absent. NOTE: numba pins numpy<2.5.
18 fast = ["numba>=0.60"]
19
20 [project.scripts]
21 signus = "signus.cli:main"
22
23 [tool.setuptools.packages.find]
24 include = ["signus*"]
25
26 [tool.setuptools.package-data]
27 signus = ["web/*"] # the served frontend: without this a pip install 404s the whole UI
28
29 [tool.pytest.ini_options]
30 pythonpath = ["."]
31 testpaths = ["tests"]
32 addopts = "-q"
33 markers = ["stretch: slow/marginal cases, opt-in"]
34
35 [tool.ruff]
36 target-version = "py311"
37 line-length = 100
38
39 [tool.ruff.lint]
40 select = ["E", "F", "I", "UP", "B", "SIM", "NPY"]
```

```
1 """signus – blind signal demodulator with automatic parameter detection."""
2
3 __version__ = "2.0.0"
```

```

1  """Sample file I/O. Filename carries read-necessities ONLY (fs, cplx|real, sample
2  type, endian, bitrev); ground truth lives in a `<filename>.json` sidecar the
3  analyzer never reads. SigMF sidecars (`<stem>.sigmf-meta`) are also understood.
4  Endianness falls back to SIGNUS_ENDIAN when the filename carries no .be/.le token."""
5
6  import json
7  import os
8  import re
9  from dataclasses import dataclass
10
11 import numpy as np
12
13 # fs/rf 토막은 옛 밑줄 형식 전용이다. 점으로도 끊길 수 있어 경계에 둘 다 넣는다 -- 새 형식은
14 # 샘플레이트를 접두사 없이 cplx|real 바로 뒤에 놓으므로 이 정규식에 걸리지 않는다.
15 _FS_TOK = re.compile(r"(?:^[_.])fs(?:\d+(?:\.\d+)?(?:e[+-]?\d+)?(?:[_.]|$)", re.I)
16 _RF_TOK = re.compile(r"(?:^[_.])rf(?:\d+(?:\.\d+)?(?:e[+-]?\d+)?(?:[_.]|$)", re.I)
17
18 # token -> (numpy code, full-scale divisor, offset). u-types are offset binary
19 # (e.g. 80 / RTL-SDR u8: 0..255 with 128 = zero).
20 _DTYPES = {
21     "i8": ("i1", 128.0, 0.0), "u8": ("u1", 128.0, 128.0),
22     "i16": ("i2", 32768.0, 0.0), "u16": ("u2", 32768.0, 32768.0),
23     "f32": ("f4", 1.0, 0.0), "f64": ("f8", 1.0, 0.0),
24 }
25 # baudline-style aliases: <bits>t = two's complement, <bits>o = offset binary
26 _ALIAS = {"8t": "i8", "8o": "u8", "16t": "i16", "16o": "u16", "32f": "f32", "64f": "f64"}
27 _TOK = {v: k for k, v in _ALIAS.items()} # 저장할 땐 baudline 표기로 되돌린다
28 _FMT = {"cplx": "iq", "real": "real", "iq": "iq"} # iq = 옛 밑줄 형식의 이름
29 _BITREV = np.array([int(f"{i:08b}"[::-1]), 2) for i in range(256)], dtype=np.uint8)
30
31 # SigMF core:datatype -> (dtype token, fmt, endian)
32 _SIGMF = {"cf32": ("f32", "iq"), "ci16": ("i16", "iq"), "ci8": ("i8", "iq"),
33           "cu8": ("u8", "iq"), "rf32": ("f32", "real"), "ri16": ("i16", "real")}
34
35
36 @dataclass
37 class Meta:
38     fs: float | None = None
39     fmt: str | None = None # 'iq' | 'real'
40     dtype: str = "i16" # key of _DTYPES
41     endian: str = "le" # 'le' | 'be' (floats included)
42     bitrev: bool = False # bit order reversed within each byte
43     rf_center: float | None = None # RF centre freq (Hz); None = unknown (report baseband)
44
45     def ok(self) -> bool:
46         return self.fs is not None and self.fmt in ("iq", "real") and self.dtype in _DTYPES
47
48
49 def parse_name(name: str) -> Meta:
50     """파일명이 읽기 필수 정보를 실어 나른다 (정답은 사이드카에만 있고 절대 읽지 않는다).
51
52     새 형식 <이름>.cplx|real.<샘플레이트 정수>.<16t>[.be][.bitrev].pcm
53     옛 형식 <이름>_fs<샘플레이트>_iq|real_<i16>[.be][.bitrev].iq
54
55     샘플레이트는 새 형식에서 접두사 없이 온다 -- 그래서 '숫자처럼 생긴 토막'이 아니라
56     cplx|real **바로 다음 자리**로 찾는다. 이름 자체가 숫자여도(20260801.cplx...) 안 헛갈린다.
57     확장자를 떼지 않고 통째로 쪼갬다: 새 형식은 마지막 토막(.pcm)이 포맷을 뜻하지 않고,
58     옛 형식은 확장자(.iq)가 이미 다른 토막과 같은 말을 하므로 어느 쪽도 손해가 없다."""
59     base = os.path.basename(name).lower()
60     toks = re.split(r"[_.]", base)

```

```

61 # 진짜 포맷 토막 = "숫자 + 아는 제원 토막"을 데리고 다니는 **마지막** 후보. 세 오독을 막는다:
62 # · 라벨의 real/cplx/iq(rec_iq_20260801.cplx...)가 fs/fmt 를 강탈 -- 마지막 후보 규칙이 막고,
63 # · 점 긴 샘플레이트(cap.cplx.2.4e6... -> fs=2 Hz)와 라벨 숫자를 fs 로 발명(...iq_20260728.raw)
64 # -- 숫자 뒤가 제원 토막(16t/be/.../pcm)이어야 한다는 관문이 막는다. 관문에 걸리면 fs 없음으로
65 # 크게 죽는다: 조용한 오답 대신 시끄러운 실패가 이 저장소의 원칙이다.
66 cands = [k for k, t in enumerate(toks) if t in _FMT]
67 hits = [k for k in cands if k + 2 < len(toks) and toks[k + 1].isdigit()
68         and (toks[k + 2] in _DTYPES or toks[k + 2] in _ALIAS
69             or toks[k + 2] in ("be", "bitrev", "pcm") or toks[k + 2].startswith("rf"))]
70 i = hits[-1] if hits else (cands[0] if cands else None)
71 m = Meta()
72 if i is not None: # 'is not None' 이어야 한다 -- 0 번 토막(cplx...)도 유효하다
73     m.fmt = _FMT[toks[i]]
74     if i in hits:
75         m.fs = float(toks[i + 1])
76     if m.fs is None and (mt := _FS_TOK.search(base)):
77         m.fs = float(mt.group(1))
78     if rt := _RF_TOK.search(base):
79         m.rf_center = float(rt.group(1))
80     tail = toks[i + 1:] if i is not None else toks # 제원은 포맷 토막 **뒤**에만 산다 --
81     m.dtype = next((_ALIAS.get(t, t) for t in tail # 라벨의 u8/f32/be 가 제원을 오염 못 하게
82                   if t in _DTYPES or t in _ALIAS), "i16")
83     # 엔디안: 파일명 토큰(.be/.le)이 최우선, 없으면 SIGNUS_ENDIAN, 그것도 없으면 le.
84     # 실장비 녹음기는 BE 라 장비에서는 export SIGNUS_ENDIAN=be 한 줄로 전 파일이 읽힌다
85     # (2026-08-21: BE 캡처를 LE 로 읽어 균등분포 잡음으로 보이던 사고 뒤에 넣었다).
86     env = os.environ.get("SIGNUS_ENDIAN", "le").lower()
87     if env not in ("le", "be"):
88         raise ValueError(f"SIGNUS_ENDIAN 은 le 또는 be 여야 합니다: {env!r}")
89     m.endian = "be" if "be" in tail else ("le" if "le" in tail else env)
90     m.bitrev = "bitrev" in tail
91     return m
92
93
94 def parse_sigmf(path: str) -> Meta | None:
95     """Read `<stem>.sigmf-meta` (core:sample_rate + core:datatype) if present."""
96     try:
97         with open(os.path.splitext(path)[0] + ".sigmf-meta") as fh:
98             doc = json.load(fh)
99             g = doc["global"]
100             code = g["core:datatype"].replace("_le", "").replace("_be", "")
101             dtype, fmt = _SIGMF[code]
102             endian = "be" if g["core:datatype"].endswith("_be") else "le"
103             fs = float(g["core:sample_rate"])
104     except (OSError, KeyError, ValueError, TypeError, AttributeError):
105         return None # MANDATORY fields malformed: fall back to filename tokens
106     try:
107         # rf centre is OPTIONAL: a malformed value must cost only this field, never the whole
108         # sidecar -- discarding valid fs/fmt/dtype here silently re-read the file with filename
109         # tokens, which can lie about fs (a quiet garbage decode).
110         caps = doc.get("captures") or []
111         rf = caps[0].get("core:frequency") if caps else None
112         rf = None if rf is None else float(rf)
113     except (KeyError, ValueError, TypeError, AttributeError, IndexError):
114         rf = None
115     return Meta(fs, fmt, dtype, endian, rf_center=rf)
116
117
118 def make_name(label: str, m: Meta) -> str:
119     """저장은 새 형식만: <이름>.cplx|real.<샘플레이트>.<16t>[.be][.bitrev].pcm"""
120     extra = ("be" if m.endian == "be" else "") + (".bitrev" if m.bitrev else "")

```

```

121     kind = "real" if m.fmt == "real" else "cplx"
122     return f"{label}.{kind}.{m.fs:.0f}._TOK.get(m.dtype, m.dtype)}.{extra}.pcm"
123
124
125 def decode(raw: bytes, meta: Meta) -> np.ndarray:
126     """Raw bytes -> samples: complex128 for 'iq', float64 for 'real'.
127     Tolerates truncated files (drops trailing partial samples)."""
128     code, scale, off = _DTYPES[meta.dtype]
129     if meta.bitrev:
130         raw = _BITREV[np.frombuffer(raw, dtype=np.uint8)].tobytes()
131     step = np.dtype(code).itemsize
132     dt = ("<" if meta.endian == "le" else ">") + code
133     x = np.frombuffer(raw[: len(raw) - len(raw) % step], dtype=dt).astype(np.float64)
134     x = (x - off) / scale
135     if meta.fmt == "iq":
136         x = x[: x.size & ~1] # drop dangling half-sample
137         return x[0::2] + 1j * x[1::2]
138     return x
139
140
141 def read(path: str, meta: Meta | None = None) -> tuple[np.ndarray, Meta]:
142     meta = meta or parse_sigmf(path) or parse_name(path)
143     if not meta.ok():
144         raise ValueError(f"need fs + fmt (filename tokens, SigMF, or --fs/--fmt): {path}")
145     with open(path, "rb") as fh:
146         x = decode(fh.read(), meta)
147     if x.size < 256:
148         raise ValueError(f"신호가 너무 짧습니다 ({x.size} samples): {path}")
149     return x, meta
150
151
152 def write(path: str, x: np.ndarray, meta: Meta) -> None:
153     """Quantize (ints: scaled to 99.9th-percentile full scale) and write interleaved."""
154     code, scale, off = _DTYPES[meta.dtype]
155     flat = np.column_stack([x.real, x.imag]).ravel() if np.iscomplexobj(x) else x.real
156     if scale != 1.0: # integer types
157         peak = np.percentile(np.abs(flat), 99.9) or 1.0
158         flat = np.round(flat * (scale - 1) / peak) + off
159         info = np.iinfo(code)
160         flat = np.clip(flat, info.min, info.max)
161     out = flat.astype(("<" if meta.endian == "le" else ">") + code)
162     if meta.bitrev:
163         out = np.frombuffer(_BITREV[np.frombuffer(out.tobytes(), np.uint8)].tobytes(),
164                             dtype=out.dtype)
165     out.tofile(path)
166
167
168 def sidecar_write(path: str, meta: Meta, truth: dict, gen: dict) -> None:
169     doc = {"fs": meta.fs, "fmt": meta.fmt, "dtype": meta.dtype,
170           "endian": meta.endian, "bitrev": meta.bitrev, "truth": truth, "gen": gen}
171     with open(path + ".json", "w") as fh:
172         json.dump(doc, fh, indent=1)
173
174
175 def sidecar_read(path: str) -> dict | None:
176     try:
177         with open(path + ".json") as fh:
178             return json.load(fh)
179     except (OSError, ValueError):
180         return None

```

```

1  """Constellations, Gray labels, differential tables, FSK levels (shared reference)."""
2
3  import numpy as np
4
5  MODS = ("bpsk", "qpsk", "8psk", "16qam", "64qam")          # blind-classify targets
6  FSK_MODS = ("fsk2", "fsk4", "msk")
7  DIFF_MODS = ("dbpsk", "dqpsk", "pi4dqpsk")
8  GEN_MODS = MODS + ("32qam", "oqpsk") + DIFF_MODS + FSK_MODS # generator vocabulary
9
10 # (data-alphabet order, constellation rotational symmetry). pi4dqpsk: 4 transition
11 # values per symbol but an 8-point composite constellation.
12 _INFO = {"bpsk": (2, 2), "qpsk": (4, 4), "8psk": (8, 8), "16qam": (16, 4),
13          "64qam": (64, 4), "32qam": (32, 4), "oqpsk": (4, 4), "pi4dqpsk": (4, 8),
14          "dbpsk": (2, 2), "dqpsk": (4, 4), "fsk2": (2, 0), "fsk4": (4, 0), "msk": (2, 0)}
15
16 # differential phase increment per Gray symbol value
17 DIFF_PHASES = {
18     "dbpsk": np.array([0.0, np.pi]),
19     "dqpsk": np.array([0.0, np.pi / 2, np.pi, -np.pi / 2]),
20     "pi4dqpsk": np.array([np.pi / 4, 3 * np.pi / 4, -3 * np.pi / 4, -np.pi / 4]),
21 }
22
23
24 def family(mod: str) -> str:
25     return "fsk" if mod in FSK_MODS else "linear"
26
27
28 def mod_order(mod: str) -> int:
29     return _INFO[mod][0]
30
31
32 def mod_symmetry(mod: str) -> int:
33     """Lowest power p for which x**p strips the modulation (square QAM: 4)."""
34     return _INFO[mod][1]
35
36
37 def ideal_points(mod: str) -> np.ndarray:
38     if mod in ("bpsk", "dbpsk"):
39         return np.array([1.0 + 0j, -1.0 + 0j])
40     if mod in ("qpsk", "oqpsk", "dqpsk"):
41         return np.exp(1j * (np.pi / 4 + np.arange(4) * np.pi / 2))
42     if mod in ("8psk", "pi4dqpsk"):
43         return np.exp(2j * np.pi * np.arange(8) / 8)
44     if mod == "32qam": # 6x6 cross: grid minus the four corners
45         lv = np.arange(6) * 2.0 - 5
46         i, q = np.meshgrid(lv, lv)
47         pts = (i + 1j * q).ravel()
48         pts = pts[np.abs(pts.real * pts.imag) != 25]
49         return pts / np.sqrt(np.mean(np.abs(pts) ** 2))
50     n = 4 if mod == "16qam" else 8
51     lv = np.arange(n) * 2.0 - (n - 1)
52     i, q = np.meshgrid(lv, lv)
53     pts = (i + 1j * q).ravel()
54     return pts / np.sqrt(np.mean(np.abs(pts) ** 2))
55
56
57 def _gray(k: np.ndarray) -> np.ndarray:
58     return k ^ (k >> 1)
59
60

```

```

61 def bit_labels(mod: str) -> np.ndarray:
62     """Gray label per ideal_points index (PSK: phase Gray; square QAM: per-axis).
63     32qam uses plain index labels (cross grid has no clean per-axis Gray)."""
64     m = mod_order(mod)
65     k = np.arange(m)
66     if mod in ("16qam", "64qam"):
67         n = 4 if mod == "16qam" else 8
68         return _gray(k // n) * n + _gray(k % n)
69     if mod == "32qam":
70         return k
71     return _gray(k)
72
73
74 def fsk_levels(mod: str) -> tuple[np.ndarray, np.ndarray]:
75     """(frequency levels, Gray label per level) - e.g. fsk4: [-3,-1,1,3], [0,1,3,2]."""
76     m = mod_order(mod)
77     return np.arange(m) * 2.0 - (m - 1), _gray(np.arange(m))
78
79
80 def to_bits(labels: np.ndarray, mod: str) -> np.ndarray:
81     """Integer labels -> flat 0/1 array, MSB first, log2(M) bits per label."""
82     k = mod_order(mod).bit_length() - 1
83     return (labels[:, None] >> np.arange(k - 1, -1, -1) & 1).ravel().astype(np.uint8)
84
85
86 def demap_bits(symbols: np.ndarray, mod: str) -> np.ndarray:
87     """Hard-decide to nearest ideal point, return Gray bits."""
88     pts = ideal_points(mod)
89     idx = np.argmin(np.abs(symbols[:, None] - pts[None, :]), axis=1)
90     return to_bits(bit_labels(mod)[idx], mod)
91
92
93 def demap_diff_bits(symbols: np.ndarray, mod: str) -> np.ndarray:
94     """Differential demap: quantize successive phase differences to the mod's
95     transition set. Rotation-ambiguity-free (no absolute phase reference)."""
96     ph = DIFF_PHASES[mod]
97     d = np.angle(symbols[1:] * np.conj(symbols[:-1]))
98     err = np.angle(np.exp(1j * (d[:, None] - ph[None, :]))) # wrapped distance
99     return to_bits(np.argmin(np.abs(err), axis=1), mod)

```

```

1  """Spectrum + spectrogram-strip views (display only, never used for detection)."""
2
3  import numpy as np
4  from scipy.signal import stft, welch
5
6
7  def _db(p: np.ndarray) -> np.ndarray:
8      return 10 * np.log10(p + 1e-20)
9
10
11 def spectrum(x: np.ndarray, fs: float, bins: int = 512) -> dict:
12     """sa 프로브의 PSD 판과 같은 조판: 실수부 welch(nperseg<=2048), 단축 0..fs/2 (kHz).
13     표시 칸으로 줄일 때는 최대 풀링 -- 좁은 바늘이 살아남게."""
14     xd = x.real if np.iscomplexobj(x) else x
15     f, p = welch(xd, fs=fs, nperseg=int(min(2048, max(64, xd.size))))
16     p = _db(p)
17     if f.size > bins: # max-pool so narrow tones survive decimation
18         k = f.size // bins
19         f, p = f[:k * bins].reshape(bins, k).mean(1), p[:k * bins].reshape(bins, k).max(1)
20     return {"f": np.round(f / 1e3, 3).tolist(), "db": np.round(p, 2).tolist()}
21
22
23 def strip(x: np.ndarray, fs: float, max_cols: int = 1600) -> dict:
24     """sa 프로브의 PNG 스펙트로그램 띠와 같은 조판의 화색조 이미지 (0..235, row-major,
25     위 행 = +fs/2 쪽). hamming 256/128 STFT, 바닥 p25 + 대비폭 10..35 dB 클램프, 열린
26     최대 풀링으로만 줄인다 -- 보간이 짧은 버스트를 뭉개지 않게."""
27     xd = x.real if np.iscomplexobj(x) else x
28     nper = int(min(256, max(64, 1 << int(np.log2(max(xd.size // 2, 64)))))
29     _, _, z = stft(xd, fs=fs, window="hamming", nperseg=nper, noverlap=nper // 2,
30                   return_onesided=True, boundary=None, padded=False)
31     db = 10 * np.log10(np.abs(z) ** 2 + 1e-12)
32     lo = float(np.percentile(db, 25))
33     rg = float(max(10.0, min(35.0, np.percentile(db, 99.5) - lo)))
34     g = np.clip((db - lo) / rg, 0, 1)[::-1]
35     kp = max(1, g.shape[1] // max_cols)
36     g = g[:, :g.shape[1] // kp * kp].reshape(g.shape[0], -1, kp).max(2)
37     return {"rows": int(g.shape[0]), "cols": int(g.shape[1]),
38           "g": np.round(g * 235).astype(int).ravel().tolist()}

```

```

1  """Receiver DSP stages for blind PSK/QAM demodulation. Vectorized; per the house
2  anti-shared-bug rule this imports only constellation reference data, never gen.py."""
3
4  from fractions import Fraction
5
6  import numpy as np
7  from scipy.ndimage import uniform_filter1d
8  from scipy.signal import get_window, hilbert, resample_poly, stft, welch
9
10 from ._accel import dd_carrier
11 from .constellations import ideal_points
12
13
14 def analytic(x: np.ndarray) -> np.ndarray:
15     """Complex passthrough as complex128; real input -> Hilbert analytic signal. Non-finite
16     samples are zeroed first: a single NaN/Inf otherwise spreads through every FFT (and Inf
17     overflows on the  $|x|^2$  the estimators take)."""
18     x = np.nan_to_num(x, posinf=0.0, neginf=0.0)
19     x = np.where(np.abs(x) > 1e150, 0.0, x) # a finite spike that overflows  $|x|^2$  is corrupt too
20     if np.iscomplexobj(x):
21         return x.astype(np.complex128)
22     return hilbert(np.asarray(x, dtype=np.float64)).astype(np.complex128)
23
24
25 def _parab(ydb: np.ndarray, k: int) -> float:
26     """Sub-bin peak offset from a 3-point parabola fit (inputs already in dB)."""
27     if k <= 0 or k >= ydb.size - 1:
28         return 0.0
29     a, b, c = ydb[k - 1], ydb[k], ydb[k + 1]
30     d = a - 2 * b + c
31     # clamp to +-half a bin: a true peak's sub-bin offset cannot exceed that, but a near-flat
32     # ( $d \sim 0$ ) or non-max triple can explode the ratio and drive baud negative -> a resample crash.
33     return float(np.clip(0.5 * (a - c) / d, -0.5, 0.5)) if d != 0 else 0.0
34
35
36 def _kmeans1d(a: np.ndarray, k: int, iters: int = 50) -> tuple[np.ndarray, np.ndarray, float]:
37     """1-D k-means on quantile-seeded centers; return (centers, labels, rms spread).
38     Assignment is a running min over the k centers (no (N, k) broadcast temporary);
39     strict '<' keeps the lowest-index center on ties, so labels are identical to a
40     broadcast argmin. Shared by the classify amplitude-ring test and the FSK level demod."""
41     c = np.quantile(a, (np.arange(k) + 0.5) / k)
42     lab = np.zeros(a.size, dtype=int)
43     for _ in range(iters):
44         d = np.abs(a - c[0])
45         lab = np.zeros(a.size, dtype=int)
46         for j in range(1, k):
47             dj = np.abs(a - c[j])
48             closer = dj < d
49             lab[closer] = j
50             d[closer] = dj[closer]
51         nc = np.array([a[lab == j].mean() if np.any(lab == j) else c[j] for j in range(k)])
52         if np.allclose(nc, c):
53             break
54         c = nc
55     return c, lab, float(np.sqrt(np.mean((a - c[lab]) ** 2)))
56
57
58 # --- burst detection -----
59
60 _SPIKE_PAR = 60.0 # raw peak-to-mean power above which a candidate run is an impulse smear, not a

```

```

61 #          burst. Modulated bursts sit low: constant-envelope FSK ~1, RRC-shaped PSK/QAM
62 #          peaks ~6-12, high-order QAM tails to ~20; a single-sample impulse smeared over
63 #          a ~win-wide run scores ~win (hundreds+). 60 clears every real mod with margin.
64
65
66 def _otsu(v: np.ndarray, bins: int = 128) -> float:
67     """Otsu split threshold of a 1-D value distribution (log powers, column scores)."""
68     hist, edges = np.histogram(v, bins=bins)
69     centers = (edges[:-1] + edges[1:]) / 2
70     w = np.cumsum(hist).astype(float)
71     m = np.cumsum(hist * centers)
72     mb = m / np.where(w > 0, w, 1)
73     mf = (m[-1] - m) / np.where(w[-1] - w > 0, w[-1] - w, 1)
74     return float(centers[int(np.argmax(w * (w[-1] - w) * (mb - mf) ** 2))])
75
76
77 def _cell_power(x: np.ndarray, fs: float, nperseg: int = 256) -> tuple[np.ndarray, int, int]:
78     """Waterfall cell power for detection: (P[fbin, col], hop, nperseg). nperseg
79     auto-shrinks on short records; shared core for find_bursts."""
80     nperseg = int(min(nperseg, max(64, 1 << int(np.log2(max(x.size // 2, 64)))))
81     hop = nperseg // 2
82     win = get_window("blackmanharris", nperseg)
83     _, _, z = stft(x, fs=fs, window=win, nperseg=nperseg, noverlap=nperseg - hop,
84     return_onesided=False, boundary=None, padded=False)
85     return np.abs(z) ** 2, hop, nperseg
86
87
88 def find_bursts(x: np.ndarray, fs: float) -> list[tuple[int, int]]:
89     """Waterfall (cell-level) burst detector. Per-bin noise floors give the same
90     narrowband processing gain the operator's eye gets on a spectrogram (a burst at
91     wideband 0 dB is +12 dB per cell when it occupies 1/16 of the band); column
92     scores then drive the run/merge/guard machinery on the time axis.
93     Returns bursts in time order, or [(0, size)] when nothing stands out."""
94     n = x.size
95     pw = np.abs(x) ** 2
96     # A burst is a TIME-ENERGY event. A continuous signal that merely shuffles its spectrum
97     # (multi-tone FSK) lights different cells per column and once shattered into 78 phantom
98     # bursts -- but its wideband envelope is FLAT. Envelope Otsu separation measured 0.014-
99     # 0.052 decades on continuous FSK vs >=0.30 on every real burst regime: nothing turns
100    # on or off -> the whole record is the burst.
101    lp = np.log10(uniform_filter1d(pw, max(64, n // 1000)) + 1e-20)
102    e_thr = _otsu(lp)
103    e_lo, e_hi = lp[lp < e_thr], lp[lp >= e_thr]
104    if not e_lo.size or not e_hi.size or float(e_hi.mean() - e_lo.mean()) < 0.12:
105        return [(0, n)]
106    # nperseg 128: bin 7.8 kHz keeps the narrowband gain for real signal widths while the
107    # 64-sample hop resolves inter-burst gaps down to ~200 samples (256 could not split a
108    # 300-sample gap -- no column fits inside it).
109    P, hop, nperseg = _cell_power(x, fs, nperseg=128)
110    if P.shape[1] < 4:
111        return [(0, n)]
112    # Per-bin floor at a LOW percentile: tracks coloured noise bin by bin and stays on the
113    # noise even when a bin is signal-occupied up to ~90% of the time (high-duty trains).
114    # A record-filling signal owns its bins' floors entirely -> flat score -> [(0, n)].
115    floor_f = np.percentile(P, 10, axis=1)[:, None]
116    r = P / (floor_f + 1e-30)
117    k = max(3, P.shape[0] // 16)
118    sc = np.log10(np.sort(r, axis=0)[-k:].mean(axis=0) + 1e-30) # column score (decades)
119    # A single noise column can spike +0.53 decades over base -- OVERLAPPING the weakest
120    # real bursts (+0.57). Like the eye, the discriminator is PERSISTENCE, not height: a

```

```

121 # 3-column smooth drops the noise worst-case to 0.31-0.33 while a real burst (>=3
122 # columns) keeps its level (0.56+) -- the gap the fixed bars below live in.
123 sc = uniform_filter1d(sc, 3)
124 # The noise base is the LOWEST MODE of the score histogram (Otsu, as the wideband
125 # version proved out): fixed quantile anchors cannot serve every duty regime, and
126 # spread ESTIMATES broke three ways (median blind >50% duty, two-sided MAD inflated by
127 # skirts, one-sided by the score's left tail). The margins are FIXED instead: a noise
128 # column's score is distribution-pinned at  $C = \log_{10}((\ln(\text{nbins}/k)+1)/0.105)$  whatever
129 # the noise level, so its fluctuation is a constant of (nbins, k) -- measured worst
130 # 0.33 over 24 noise records vs 0.56 for the weakest keeper burst.
131 thr = _otsu(sc, bins=64)
132 quiet = sc[sc < thr]
133 if not quiet.size:
134     return [(0, n)]
135 base = float(np.median(quiet))
136 # No noise-quiet column at all (base far above the pinned C): a continuous signal
137 # shuffling its spectrum (a 4-tone FSK read as 78 phantom bursts before this guard).
138 c_noise = float(np.log10((np.log(P.shape[0] / k) + 1) / 0.105))
139 if base > c_noise + 0.25:
140     return [(0, n)]
141 hi = base + 0.45
142 lo = base + 0.25
143 above_lo = sc >= lo
144 dif = np.diff(above_lo.astype(np.int8))
145 starts = list(np.where(dif == 1)[0] + 1)
146 ends = list(np.where(dif == -1)[0] + 1)
147 if above_lo[0]:
148     starts.insert(0, 0)
149 if above_lo[-1]:
150     ends.append(sc.size)
151 runs = [(s, e) for s, e in zip(starts, ends, strict=True) if (sc[s:e] >= hi).any()]
152 if not runs:
153     return [(0, n)]
154 merged = [list(runs[0])]
155 for s, e in runs[1:]:
156     # A real inter-burst gap returns to the score base; a marginal-level burst dips just
157     # below lo without reaching it and would otherwise shatter into fragments. Merge
158     # across any valley whose median stays off the base.
159     valley = sc[merged[-1][1]:s]
160     if s - merged[-1][1] < 2 or float(np.median(valley)) > base + 0.12:
161         merged[-1][1] = e
162     else:
163         merged.append([s, e])
164 # Shape gate: a modulated burst has a near-flat raw envelope (peak/mean ~ a few); an
165 # impulse smear is one spike over noise (ratio ~span). Reject spike-dominated candidates.
166 out, spiked = [], 0
167 for c0, c1 in merged:
168     if c1 - c0 < 2: # single-column flicker (noise / impulse smear)
169         continue
170     s, e = max(0, c0 * hop), min(n, (c1 - 1) * hop + nperseg)
171     if c1 - c0 >= 6:
172         # long bursts: trim one hop of smear/quantization slack per edge. The noise
173         # tail otherwise fed to the demod sits on a knife edge for dense QAM (58 extra
174         # leading samples flipped a 16qam@24dB slice to 64qam lock 18). Short runs keep
175         # their full span -- trimming there would eat into the burst itself.
176         s, e = s + hop, e - hop
177     seg = pw[s:e]
178     if float(seg.max() / (seg.mean() + 1e-30)) > _SPIKE_PAR:
179         spiked += e - s
180     continue

```

```

181         out.append((int(s), int(e)))
182         # A detection must EXPLAIN the candidate mass the SPIKE gate threw away. If an impulse
183         # inside the strongest burst killed its candidate, returning the survivors would hand
184         # analyze() the wrong emitter -- fall back instead. Only spike kills count: the
185         # single-column drops above are routine flicker filtering.
186         covered = sum(e - s for s, e in out)
187         if out and covered < 0.6 * (covered + spiked):
188             return [(0, n)]
189         # A continuous signal OWNS its bins' floors and is invisible to the cell path -- but its
190         # edge transients are not, and once returned [(0, 192)] for a 64k channel whose 16qam
191         # ran the whole record. If the detections cover almost none of the envelope's high-power
192         # mass, the cell path missed the main event: return the ENVELOPE's high span instead
193         # (not the raw record -- a dead tail fed to the demod flipped that 16qam to 64qam).
194         # 0.1 keeps partial detections (a strong burst next to a lo-only sibling covers ~30%).
195         if out and covered < 0.1 * int((lp >= e_thr).sum()):
196             idx = np.where(lp >= e_thr)[0]
197             return [(int(idx[0]), int(idx[-1]) + 1)]
198         return out or [(0, n)]
199
200
201 def find_burst(x: np.ndarray, fs: float) -> tuple[int, int]:
202     """The most energetic burst of find_bursts."""
203     return max(find_bursts(x, fs), key=lambda b: float(np.sum(np.abs(x[b[0]:b[1]]) ** 2)))
204
205
206 # --- carrier estimation -----
207
208 def est_carrier(
209     x: np.ndarray, fs: float, powers: tuple[int, ...] = (2, 4, 8),
210     nfft: int = 65536, blocks: int = 4,
211 ) -> tuple[float, int, bool]:
212     """Blind M-th-power carrier estimate; returns (fc, symmetry, ambiguous). No DC
213     nulling: the front-end is DC-blocked, so a peak at DC is a genuine fc=0."""
214     freqs = np.fft.fftshift(np.fft.fftfreq(nfft, 1 / fs))
215     best_par, best_s, sym = -1.0, None, powers[0]
216     for p in powers:
217         xp = x ** p
218         blk = xp.size // blocks
219         segs = [xp[i * blk:(i + 1) * blk] for i in range(blocks)] if blk >= 8 else [xp]
220         acc = np.zeros(nfft)
221         for s in segs:
222             acc += np.abs(np.fft.fftshift(np.fft.fft(s * np.hanning(s.size), nfft)))
223         spec = acc / len(segs)
224         par = spec.max() / spec.mean()
225         if par > best_par:
226             best_par, best_s, sym = par, spec, p
227
228     if best_s is None: # degenerate input (all-zero / no energy): no M-th-power tone exists
229         return 0.0, int(powers[0]), True
230     ydb = 20 * np.log10(best_s + 1e-20)
231     k = int(np.argmax(best_s))
232     f_peak = freqs[k] + _parab(ydb, k) * (freqs[1] - freqs[0])
233     fc = f_peak / sym
234     ambiguous = abs(sym * fc) > 0.4 * fs
235     return float(fc), int(sym), bool(ambiguous)
236
237
238 def resolve_alias(x: np.ndarray, fs: float, fc: float, sym: int) -> float:
239     """The M-th-power tone wraps mod fs, so fc is only known mod fs/sym. Candidates tile the
240     whole band at spacing fs/sym; pick the one whose alias CELL holds the most signal power

```

```

241     (wrap-aware). Cell-energy beats a spectral centroid at the band edge, where a signal
242     straddling  $\pm fs/2$  corrupts the two-sided centroid (bpsk  $fc=0.495*fs$  picked  $fc=-5000$ )."""
243     f, pxx = welch(x, fs=fs, nperseg=min(4096, x.size), return_onesided=False)
244     p = np.maximum(pxx - np.median(pxx), 0)
245     # range must reach  $\pm fs/2$  for EVERY sym: |k| up to sym//2 (an old fixed range(-2,3) left
246     # 8psk beyond  $0.3125*fs$  with no candidate).
247     cands = [fc + k * fs / sym for k in range(-(sym // 2) - 1, sym // 2 + 2)]
248     if abs(fc + k * fs / sym) < 0.5 * fs]
249     half = fs / (2 * sym) # each candidate owns a fs/sym-wide cell
250     return max(cands, key=lambda c: float(p[np.abs(((f - c + fs / 2) % fs) - fs / 2) < half].sum()))
251
252
253 def mix(x: np.ndarray, fs: float, fc: float) -> np.ndarray:
254     """Downconvert by fc (complex heterodyne)."""
255     return x * np.exp(-2j * np.pi * fc * np.arange(x.size) / fs)
256
257
258 # --- symbol rate + rolloff -----
259
260 def est_baud(
261     x: np.ndarray, fs: float, lo: float | None = None, hi: float | None = None,
262     blocks: int = 4,
263 ) -> tuple[float, float]:
264     """Cyclostationary line at the symbol rate in  $|x|^2$ ; returns (baud, confidence).
265     Low confidence flags a weak line (near-zero rolloff); caller decides. Fewer blocks =
266     a longer, stronger line (worth trying on a short burst where the default 4 is too weak)."""
267     u = np.abs(x) ** 2
268     u = u - u.mean()
269     blk = u.size // blocks
270     segs = [u[i * blk:(i + 1) * blk] for i in range(blocks)] if blk >= 8 else [u]
271     nfft = 1 << max(16, int(np.ceil(np.log2(max(s.size for s in segs))))))
272     acc = np.zeros(nfft // 2 + 1)
273     for s in segs:
274         acc += np.abs(np.fft.rfft(s * np.hanning(s.size), nfft))
275     spec = acc / len(segs)
276     freqs = np.fft.rfftfreq(nfft, 1 / fs)
277
278     lo = lo if lo is not None else 0.005 * fs
279     hi = hi if hi is not None else 0.45 * fs
280     band = (freqs >= lo) & (freqs <= hi)
281     if not band.any(): # degenerate window (e.g. noise-derived prior): default band
282         band = (freqs >= 0.005 * fs) & (freqs <= 0.45 * fs)
283     idx = np.where(band)[0]
284     k = idx[int(np.argmax(spec[idx]))]
285     ydb = 20 * np.log10(spec + 1e-20)
286     baud = freqs[k] + _parab(ydb, k) * (freqs[1] - freqs[0])
287     conf = spec[k] / (np.median(spec[band]) + 1e-20)
288     return float(baud), float(conf)
289
290
291 def occupied_bw(x: np.ndarray, fs: float) -> float:
292     """Two-sided occupied bandwidth without a baud prior: threshold 10% of the way
293     from the 25th-percentile floor to the 90th-percentile in-band level, walk
294     outward from DC, stop at the first >20-bin below-threshold gap (notch tolerant).
295     Returns 0.0 when no band hugs DC (caller must not trust the prior then)."""
296     f, pxx = welch(x, fs=fs, nperseg=min(8192, x.size), return_onesided=False)
297     floor = np.percentile(pxx, 25)
298     order = np.argsort(np.abs(f))
299     idx = np.flatnonzero(pxx[order] > floor + 0.10 * (np.percentile(pxx, 90) - floor))
300     if idx.size == 0:

```

```

301         return 0.0
302     gaps = np.flatnonzero(np.diff(idx) > 20)    # 틈 정지가 없으면 대역 밖 스퍼(믹싱돼
303     last = idx[gaps[0]] if gaps.size else idx[-1]    # -fc 에 떨어진 LO 누설)가 bw 를 부풀린다
304     return float(2 * np.abs(f[order][last]))
305
306
307 def est_rolloff(x: np.ndarray, fs: float, baud: float) -> float:
308     """Occupied-bandwidth band-edge estimate mapped through the v1 rolloff calibration."""
309     n = x.size
310     nper = min(8192, max(256, n // 8))
311     f, pxx = welch(x, fs=fs, nperseg=nper, return_onesided=False)
312     f, pxx = np.fft.fftshift(f), np.fft.fftshift(pxx)
313     af = np.abs(f)
314     inband = np.median(pxx[af < 0.25 * baud])
315     noise = np.median(pxx[af > 0.9 * baud])
316     thr = noise + 0.10 * (inband - noise)
317     band = (af > 0.3 * baud) & (af < 1.0 * baud) & (pxx > thr)
318     if not band.any():
319         return 0.35
320     raw = 2 * af[band].max() / baud - 1
321     return float(np.clip(1.5 * raw + 0.026, 0.05, 1.0))
322
323
324 # --- resampling + matched filter -----
325
326 def to_sps(x: np.ndarray, fs: float, baud: float, sps: int = 4) -> np.ndarray:
327     """Rational resample to exactly sps samples/symbol."""
328     r = Fraction(sps * baud / fs).limit_denominator(2000)
329     return resample_poly(x, r.numerator, r.denominator)
330
331
332 def _rx_rrc(alpha: float, span: int, sps: int) -> np.ndarray:
333     """RX matched RRC taps, unit energy, odd length; singularities at t=0 and
334     |t|=1/4a patched. Written independently of gen.py's TX shaper (anti-shared-bug)."""
335     m = (span * sps) // 2
336     t = np.arange(-m, m + 1) / sps
337     a = alpha
338     num = np.sin(np.pi * t * (1 - a)) + 4 * a * t * np.cos(np.pi * t * (1 + a))
339     den = np.pi * t * (1 - (4 * a * t) ** 2)
340     h = np.divide(num, den, out=np.zeros_like(t), where=den != 0)
341     h[t == 0] = 1 - a + 4 * a / np.pi
342     if a > 0:
343         s = np.isclose(np.abs(t), 1 / (4 * a))
344         h[s] = a / np.sqrt(2) * ((1 + 2 / np.pi) * np.sin(np.pi / (4 * a))
345                                + (1 - 2 / np.pi) * np.cos(np.pi / (4 * a)))
346     return h / np.sqrt(np.sum(h ** 2))
347
348
349 def matched(x: np.ndarray, sps: int, alpha: float, span: int = 10) -> np.ndarray:
350     """Apply the RX matched RRC filter."""
351     return np.convolve(x, _rx_rrc(alpha, span, sps), "same")
352
353
354 # --- timing recovery -----
355
356 def timing(x: np.ndarray, sps: int, block: int = 256, out: int = 1) -> np.ndarray:
357     """Block-wise Oerder-Meyr timing recovery; returns `out` samples/symbol
358     (1 = decisions, 2 = clock-tracked T/2 stream for the fractionally-spaced eq).
359     Per block the spectral-line phase gives the offset; unwrapped across blocks
360     to track clock drift, then interpolated per symbol."""

```

```

361     n = sps
362     nsym = x.size // n
363     if nsym < 1:
364         return x[:0].astype(np.complex128)
365     xt = x[:nsym * n]
366     p = np.abs(xt) ** 2
367     ph = np.exp(-2j * np.pi * np.arange(xt.size) / n)
368     bs = block * n
369     if xt.size <= bs: # single block: one global fractional offset
370         eps = np.full(nsym, -np.angle(np.sum(p * ph)) / (2 * np.pi))
371     else:
372         nblk = xt.size // bs
373         c = (p[:nblk * bs].reshape(nblk, bs) * ph[:nblk * bs].reshape(nblk, bs)).sum(1)
374         eps_b = np.unwrap(-np.angle(c)) / (2 * np.pi)
375         eps = np.interp(np.arange(nsym), (np.arange(nblk) + 0.5) * block, eps_b)
376     pos = (np.arange(nsym * out) // out + np.repeat(eps, out)) * n
377     idx = np.arange(xt.size)
378     return np.interp(pos, idx, xt.real) + 1j * np.interp(pos, idx, xt.imag)
379
380
381 # --- decision-directed carrier sync -----
382
383 def ddsync(symbols: np.ndarray, mod: str, alpha: float = 0.05, beta: float = 0.002) -> np.ndarray:
384     """Decision-directed PI carrier loop, general across PSK/QAM (never a PSK-only
385     Costas loop, which jitters QAM at the correct points). Sequential feedback (each
386     phase update depends on the prior decision) -> numba kernel when available."""
387     pts = ideal_points(mod)
388     z = symbols / np.sqrt(np.mean(np.abs(symbols) ** 2)) # AGC to unit power
389     return dd_carrier(np.ascontiguousarray(z, dtype=np.complex128),
390                      np.ascontiguousarray(pts, dtype=np.complex128), alpha, beta)

```

```

1  """Blind symbol-spaced FIR equalizer: CMA acquisition then decision-directed LMS,
2  to undo the multipath ISI (the generator's `taps=` channel) that a phase-only
3  carrier loop cannot remove."""
4
5  import numpy as np
6
7  from ._accel import cma_dd_equalize, cma_dd_equalize_fse
8  from .constellations import ideal_points
9
10
11 def equalize(symbols: np.ndarray, mod: str, n_taps: int = 11, mu_cma: float = 5e-3,
12             mu_dd: float = 2e-3, warmup: int = 1500) -> np.ndarray:
13     """Blind FIR equalizer for symbol-spaced samples; returns the cleaned symbols.
14
15     Phase 1 is constant-modulus (Godard) acquisition, phase 2 refines by decisions,
16     then the converged taps refilter the whole record so early symbols are clean too.
17     """
18     pts = ideal_points(mod)
19     r2 = np.mean(np.abs(pts) ** 4) / np.mean(np.abs(pts) ** 2) # Godard dispersion const
20     x = symbols / np.sqrt(np.mean(np.abs(symbols) ** 2)) # AGC to unit power
21     # The adaptive passes (every tap update depends on the previous output/decision) are
22     # the one part that cannot be vectorized; they run as a numba kernel when available.
23     return cma_dd_equalize(np.ascontiguousarray(x, dtype=np.complex128),
24                           np.ascontiguousarray(pts, dtype=np.complex128),
25                           float(r2), n_taps, mu_cma, mu_dd, warmup)
26
27
28 def equalize_fse(x2: np.ndarray, mod: str, n_taps: int = 75, mu_cma: float = 1e-3,
29                 mu_dd: float = 8e-4, warmup: int = 4000) -> np.ndarray:
30     """T/2 fractionally-spaced CMA->DD equalizer: 2 samples/symbol in, symbols out.
31
32     Sees the unaliased spectrum, so it inverts channels whose symbol-rate folding
33     puts zeros near the unit circle (e.g. a strong 1.5-symbol echo) where the
34     symbol-spaced `equalize` cannot. A strong echo's inverse is LONG (0.8 gain
35     needs ~70 T/2 taps for -20 dB); the clock must already be tracked, so feed it
36     `timing(ym, sps, out=2)` -- fixed taps cannot follow a drifting clock.
37     Calibrated on 2ray(0.8, 1.5 sym) x seeds 0-2: lock 77-80 (61 taps: 57-66)."""
38     pts = ideal_points(mod)
39     r2 = np.mean(np.abs(pts) ** 4) / np.mean(np.abs(pts) ** 2) # Godard dispersion const
40     x = x2 / np.sqrt(np.mean(np.abs(x2) ** 2)) # AGC to unit power
41     # Seven sequential passes (2 CMA + 4 DD + 1 tracked output); numba kernel when available.
42     return cma_dd_equalize_fse(np.ascontiguousarray(x, dtype=np.complex128),
43                               np.ascontiguousarray(pts, dtype=np.complex128),
44                               float(r2), n_taps, mu_cma, mu_dd, warmup)

```

```

1  """Blind repeated-preamble synchronization (definition-free).
2
3  A real packetized emission begins with a preamble: a short block repeated a few times so a
4  receiver can lock. The repetition is detectable and exploitable WITHOUT knowing its content --
5  the delay-and-correlate (Schmidl-Cox) metric  $M(d) = |\text{Sum conj}(x[d+k]) x[d+k+P]|^2 / (\text{Sum } |x|^2)^2$ 
6  rises to  $\sim 1$  over the span where two period- $P$  windows coincide (inside the preamble), and the PHASE
7  of that correlation is a low-variance carrier-frequency-offset (CFO) estimate that works from only a
8  few symbols -- where the blind  $M$ -th-power carrier estimator, needing many symbols to average, fails.
9
10 Used as a gated, keep-best rescue in pipeline.analyze(): a signal with no preamble yields no
11 plateau -> no rescue -> the existing blind chain is untouched (sweep stays byte-identical)."""
12
13 from dataclasses import dataclass
14
15 import numpy as np
16
17 _CONF_MIN = 0.80      # plateau strength floor (median M over the run); calibrated vs no-preamble
18 _RUN_MIN = 4.0        # plateau length >= this * period. RRC pulse memory spans a FIXED  $\sim 4.5$  symbols,
19 #                      so in periods it exceeds this only at a 1-2 symbol lag and dies by  $P \sim 3$  syms;
20 #                      a real preamble's plateau is  $(R-1)$  periods long -> this rejects the short
21 #                      memory (needs  $R \sim 5$  repeats; residual random false-alarms are caught by the
22 #                      pipeline keep-best-lock gate -- a spurious CFO lowers lock and is discarded).
23 _HARM_MIN = 0.50      # the SAME plateau must also correlate at lag  $2P$  (a real preamble,  $\geq 3$  repeats)
24 _VALLEY_MAX = 0.80    # ...AND DROP at lag  $1.5P$  (between harmonics): a real preamble is peaked at
25 #                      multiples of  $P$  (a valley in between), so  $\text{median}(M@1.5P) < 0.85 * \text{conf}$ . RRC pulse
26 #                      memory is a smooth decay ( $\sim$ equal at  $P$  and  $1.5P$ ) -> rejected. Peak/valley +
27 #                      run length is what lets the period be scanned directly, WITHOUT a reliable
28 #                      baud (est_baud is exactly what fails on the short bursts we target).
29 _PMIN = 12            # smallest period to scan (samples); below this is intra-symbol pulse memory
30 _PMAX = 512           # largest period to scan; a preamble block is short ( $L \leq \sim 12$  syms,  $\text{sps} \leq \sim 40$ )
31 _WMAX = 8192          # only the leading window is scanned -- a preamble sits at the burst START
32
33
34 @dataclass
35 class Preamble:
36     start: int         # sample index where the repeated region begins
37     end: int           # sample index where the preamble ends (data starts ~here)
38     period: int        # repetition period in samples (=  $L * \text{sps}$ )
39     cfo_hz: float      # carrier-frequency offset estimated from the preamble phase slope
40     conf: float        # 0..1 plateau confidence (median metric over the detected run)
41
42
43 def _metric(x: np.ndarray, P: int) -> tuple[np.ndarray, np.ndarray]:
44     """Normalized delay-correlation  $M(d)$  in  $[0,1]$  and the raw correlation  $P_{\text{met}}(d)$ , lag  $P$ , window
45      $W=P$ . Normalized by BOTH windows' energy (Cauchy-Schwarz) so  $M \leq 1$  -- a true correlation
46     coefficient,  $\sim 1$  only where the two period- $P$  windows are genuinely identical (a preamble)."""
47     prod = np.conj(x[:-P]) * x[P:]          # conj( $x[n]$ )  $x[n+P]$ 
48     ea = np.abs(x[:-P]) ** 2                # first-window energy
49     eb = np.abs(x[P:]) ** 2                 # second-window energy
50     cp = np.concatenate([[0.0 + 0j], np.cumsum(prod)])
51     ca = np.concatenate([[0.0], np.cumsum(ea)])
52     cb = np.concatenate([[0.0], np.cumsum(eb)])
53     W = P
54     pm = cp[W:] - cp[:-W]                  # Sum prod[d:d+W]
55     da = ca[W:] - ca[:-W]
56     db = cb[W:] - cb[:-W]
57     return np.abs(pm) ** 2 / (da * db + 1e-30), pm
58
59
60 def _seg_median(x: np.ndarray, lag: int, s: int, ln: int) -> float:

```

```

61     """Median of the delay-correlation metric at `lag` over the plateau [s, s+ln)."""
62     m = _metric(x, lag)[0]
63     e = min(s + ln, m.size)
64     return float(np.median(m[s:e])) if e > s else 0.0
65
66
67 def _longest_run(mask: np.ndarray) -> tuple[int, int]:
68     """(start, length) of the longest True run in a boolean array; (0, 0) if none."""
69     if not mask.any():
70         return 0, 0
71     d = np.diff(np.concatenate([[0], mask.view(np.int8), [0]]))
72     starts = np.where(d == 1)[0]
73     ends = np.where(d == -1)[0]
74     i = int(np.argmax(ends - starts))
75     return int(starts[i]), int(ends[i] - starts[i])
76
77
78 def find_preamble(x: np.ndarray, fs: float, baud_hint: float | None = None) -> Preamble | None:
79     """Detect a repeated preamble and estimate its (start, end, period, CF0, confidence).
80     Returns None when no repeated block stands out. The period is scanned DIRECTLY (not derived
81     from baud) so it works even when est_baud fails on the short burst -- exactly the target case.
82     baud_hint, if given, only tightens the scan's upper bound for speed."""
83     x = np.asarray(x, dtype=np.complex128)
84     x = x - x.mean()
85     if x.size < 256 or not np.all(np.isfinite(x)):
86         return None
87     xw = x[:_WMAX] # a preamble sits at the burst start
88     n = xw.size
89     pmax = _PMAX if not (baud_hint and baud_hint > 0) else min(_PMAX, int(12 * fs / baud_hint))
90     pmax = min(pmax, (n - 8) // 3) # need >=3 windows for the 2P harmonic check
91     best = None
92     for P in range(_PMIN, max(_PMIN + 1, pmax + 1)):
93         M, pm = _metric(xw, P)
94         run_start, run_len = _longest_run(M >= _CONF_MIN)
95         if run_len < _RUN_MIN * P:
96             continue
97         conf = float(np.median(M[run_start:run_start + run_len]))
98         # peak/valley gate: a genuinely periodic preamble correlates at 2P (harmonic) but DROPS at
99         # 1.5P (between harmonics); RRC pulse memory (which fakes a plateau at a 1-2 symbol lag) is
100         # a smooth decay -- high at both -> rejected, letting the period be scanned directly.
101         harm = _seg_median(xw, 2 * P, run_start, run_len)
102         valley = _seg_median(xw, int(round(1.5 * P)), run_start, run_len)
103         if harm < _HARM_MIN or valley > _VALLEY_MAX * conf:
104             continue
105         score = run_len * conf # long AND strong wins
106         if best is None or score > best[0]:
107             ang = float(np.angle(pm[run_start:run_start + run_len].sum())) # coherent, wrap-safe
108             cfo = ang * fs / (2 * np.pi * P)
109             end = run_start + run_len + 2 * P # past the last correlated repeat -> data
110             best = (score, Preamble(run_start, min(end, x.size), P, cfo, conf))
111     return best[1] if best else None

```

```

1  """Decision layer: modulation classification, rotation alignment, lock quality, SNR."""
2
3  from dataclasses import dataclass
4
5  import numpy as np
6
7  from .constellations import ideal_points, mod_symmetry
8  from .dsp import _kmeans1d
9
10 # Amplitude-ring separators, calibrated through the real front-end (seeds 0-3, snr 14-25):
11 # 16qam <=0.80 and 64qam <=0.99 vs 32qam >=1.16 on s3*s9/s5**2.
12 _D32 = 1.07
13 _C42_PSK = 0.80 # constant-modulus 4th cumulant: qpsk ~1.0, QAM <0.7
14 _CM_TOL = 0.2 # 1-ring spread guard for constant-modulus signals
15 _R39 = 2.7 # s3/s9: 16qam ~2.1, 64qam >=3.0
16
17
18 def _agc(z: np.ndarray) -> np.ndarray:
19     """Scale to unit average power."""
20     return z / np.sqrt(np.mean(np.abs(z) ** 2))
21
22
23 def _c42(z: np.ndarray) -> float:
24     """Magnitude of the normalized 4th-order cumulant C42 (rotation invariant)."""
25     z = _agc(z)
26     p = np.mean(np.abs(z) ** 2)
27     return float(abs(np.mean(np.abs(z) ** 4) - abs(np.mean(z**2)) ** 2 - 2 * p**2))
28
29
30 def _spread(a: np.ndarray, k: int) -> float:
31     """RMS within-cluster spread from the shared 1-D k-means (sorted-value ring test)."""
32     return _kmeans1d(a, k)[2]
33
34
35 def classify(z: np.ndarray, symmetry: int) -> str:
36     """Symmetry 2->bpsk, 8->8psk; 4-> qpsk/16qam/32qam/64qam via C42 then amplitude rings.
37
38     Not blind classes (documented): oqpsk collides with square-QAM on every ring/cumulant
39     feature; pi4dqpsk's 4th power carries a pi-per-symbol rotation, so the carrier estimate
40     absorbs baud/8 and it arrives here looking exactly like qpsk (its bits still decode
41     correctly via differential demapping).
42     """
43     if symmetry == 2:
44         return "bpsk"
45     if symmetry == 8:
46         return "8psk"
47     z = _agc(z)
48     if _c42(z) >= _C42_PSK:
49         return "qpsk"
50     a = np.sort(np.abs(z))
51     s1 = _spread(a, 1)
52     if s1 <= _CM_TOL: # constant-modulus guard (a PSK cloud that slipped the C42 gate)
53         return "qpsk"
54     s3, s5, s9 = _spread(a, 3), _spread(a, 5), _spread(a, 9)
55     if s3 * s9 / (s5 * s5 + 1e-18) > _D32: # only a 5-ring cross collapses at k=5
56         return "32qam"
57     # amplitude-ring ratio, NEVER best-fit EVM (a denser grid always fits closer)
58     return "16qam" if s3 / (s9 + 1e-12) < _R39 else "64qam"
59
60

```

```

61 _SNR_CEIL = 40.0 # above this the moments cannot resolve the noise term
62
63
64 def snr_m2m4(z: np.ndarray, mod: str) -> float:
65     """Blind symbol-SNR (dB) from the 2nd/4th moments, corrected for the
66     constellation kurtosis. Saturates at _SNR_CEIL (noise below the moments'
67     resolution); NaN when the moments leave the valid region entirely."""
68     ka = np.mean(np.abs(ideal_points(mod)) ** 4) # unit-power points => already /M2**2
69     m2, m4 = np.mean(np.abs(z) ** 2), np.mean(np.abs(z) ** 4)
70     disc = 2 * m2**2 - m4
71     if disc <= 0:
72         return float("nan")
73     s = np.sqrt(disc / (2 - ka))
74     n = m2 - s
75     return min(float(10 * np.log10(s / n)), _SNR_CEIL) if n > 0 else _SNR_CEIL
76
77
78 def _nearest_dist(w: np.ndarray, pts: np.ndarray) -> np.ndarray:
79     """Distance to nearest ideal point. Running min over the M points instead of a
80     (... , M) broadcast temporary: identical values, far less memory, ~1.5-5x faster."""
81     d = np.abs(w - pts[0])
82     for p in pts[1:]:
83         d = np.minimum(d, np.abs(w - p))
84     return d
85
86
87 def align(z: np.ndarray, mod: str) -> np.ndarray:
88     """AGC + resolve the M-fold rotation ambiguity by minimizing nearest-point distance."""
89     z = _agc(z)
90     pts = ideal_points(mod)
91     sub = z[:1000]
92     ang = np.linspace(0, 2 * np.pi / mod_symmetry(mod), 64, endpoint=False)
93     rot = sub[None, :] * np.exp(1j * ang)[: , None]
94     cost = _nearest_dist(rot, pts).mean(axis=1)
95     i = int(np.argmin(cost))
96     c0, c1, c2 = cost[(i - 1) % 64], cost[i], cost[(i + 1) % 64] # parabolic sub-grid refine
97     denom = c0 - 2 * c1 + c2
98     # clip: on a flat cost surface (e.g. pure noise) the fit can explode
99     delta = float(np.clip(0.5 * (c0 - c2) / denom, -1, 1)) if denom else 0.0
100     return z * np.exp(1j * (ang[i] + delta * (ang[1] - ang[0])))
101
102
103 @dataclass
104 class Quality:
105     evm: float
106     mer_db: float
107     lock: float
108     occupied: int
109
110
111 def quality(z: np.ndarray, mod: str) -> Quality:
112     """Alignment + EVM/MER, plus an occupancy-gated lock score in [0, 100]."""
113     z = align(z, mod)
114     pts = ideal_points(mod)
115     m = pts.size
116     idx = np.argmin(np.abs(z[:, None] - pts[None, :]), axis=1)
117     d = pts[idx]
118     evm = float(np.sqrt(np.mean(np.abs(z - d) ** 2) / np.mean(np.abs(d) ** 2)))
119     mer_db = float(-20 * np.log10(evm + 1e-12))
120     thr = max(3, 0.6 * len(z) / m)

```

```
121 |     occupied = int(np.sum(np.bincount(idx, minlength=m) >= thr))
122 |     lock = float(np.clip((mer_db - 6) / 19 * 100, 0, 100) * (occupied / m))
123 |     return Quality(evm, mer_db, lock, occupied)
```

```

1  """Optional numba acceleration for the sequential feedback loops (adaptive equalizer
2  taps, decision-directed carrier) that CANNOT be vectorized -- each update depends on
3  the previous output, so they are the only python-loop hot spots in an otherwise
4  vectorized pipeline. Numba is an OPTIONAL extra ('pip install signus[fast]'): when it
5  is absent 'njit' degrades to a no-op and the identical bodies run as plain numpy, so
6  results are unchanged and the numpy+scipy-only baseline still works. The kernels use
7  the same expressions as the pure-numpy originals, so the numba path stays numerically
8  equivalent (verified: byte-identical 'w @ win', argmin/abs/angle exact)."""
9
10 import os
11
12 import numpy as np
13
14 HAVE_NUMBA = False
15 if not os.environ.get("SIGNUS_NO_NUMBA"): # escape hatch: force the pure-numpy path
16     try:
17         import numba
18         njit = numba.njit(cache=True, fastmath=False) # fastmath off: keep IEEE results
19         HAVE_NUMBA = True
20     except ImportError: # numba absent: same bodies run as plain numpy (slower, identical)
21         pass
22 if not HAVE_NUMBA:
23     def njit(fn):
24         return fn
25
26
27 @njit
28 def cma_dd_equalize(x, pts, r2, n_taps, mu_cma, mu_dd, warmup):
29     """Symbol-spaced CMA acquisition -> decision-directed LMS -> converged refilter."""
30     n = x.size
31     mid = n_taps // 2
32     xp = np.concatenate((np.zeros(mid, dtype=np.complex128), x,
33                          np.zeros(mid, dtype=np.complex128)))
34     w = np.zeros(n_taps, dtype=np.complex128)
35     w[mid] = 1.0
36     out = np.empty(n, dtype=np.complex128)
37     warm = min(warmup, n)
38     for i in range(warm): # Phase 1: CMA (blind, modulus-only)
39         win = xp[i:i + n_taps]
40         y = w @ win
41         w = w - mu_cma * (y * (abs(y) ** 2 - r2)) * np.conj(win)
42     for i in range(warm, n): # Phase 2: decision-directed LMS
43         win = xp[i:i + n_taps]
44         y = w @ win
45         d = pts[np.argmax(np.abs(y - pts))]
46         w = w - mu_dd * (y - d) * np.conj(win)
47     for i in range(n): # refilter whole record with final taps
48         win = xp[i:i + n_taps]
49         y = w @ win
50         d = pts[np.argmax(np.abs(y - pts))]
51         w = w - mu_dd * (y - d) * np.conj(win)
52         out[i] = y
53     return out
54
55
56 @njit
57 def cma_dd_equalize_fse(x, pts, r2, n_taps, mu_cma, mu_dd, warmup):
58     """T/2 fractionally-spaced: 2 CMA passes -> 4 DD passes -> tracked output pass."""
59     nsym = x.size // 2
60     mid = n_taps // 2

```

```

61     xp = np.concatenate((np.zeros(mid, dtype=np.complex128), x,
62                             np.zeros(n_taps, dtype=np.complex128)))
63     w = np.zeros(n_taps, dtype=np.complex128)
64     w[mid] = 1.0
65     out = np.empty(nsym, dtype=np.complex128)
66     warm = min(warmup, nsym)
67     for _ in range(2):                                     # CMA acquisition, 2 data-reuse passes
68         for i in range(warm):
69             win = xp[2 * i:2 * i + n_taps]
70             y = w @ win
71             w = w - mu_cma * (y * (abs(y) ** 2 - r2)) * np.conj(win)
72     for _ in range(4):                                     # decision-directed refinement
73         for i in range(nsym):
74             win = xp[2 * i:2 * i + n_taps]
75             y = w @ win
76             d = pts[np.argmin(np.abs(y - pts))]
77             w = w - mu_dd * (y - d) * np.conj(win)
78     for i in range(nsym):                                   # final tracked output pass
79         win = xp[2 * i:2 * i + n_taps]
80         y = w @ win
81         d = pts[np.argmin(np.abs(y - pts))]
82         w = w - mu_dd * (y - d) * np.conj(win)
83         out[i] = y
84     return out
85
86
87 @njit
88 def dd_carrier(z, pts, alpha, beta):
89     """Per-symbol decision-directed PI carrier loop (general PSK/QAM, not PSK Costas)."""
90     out = np.empty(z.size, dtype=np.complex128)
91     phase = 0.0
92     freq = 0.0
93     for i in range(z.size):
94         y = z[i] * np.exp(-1j * phase)
95         d = pts[np.argmin(np.abs(y - pts))]
96         e = np.angle(y * np.conj(d))
97         out[i] = y
98         phase += freq + alpha * e
99         freq += beta * e
100    return out

```

```

1  """FSK/CPM receive path: constant-envelope gate + frequency-discriminator demod.
2  Runs on the DC-blocked analytic burst, before any carrier work (linear front-end
3  does not apply). Imports only constellation reference data (anti-shared-bug rule)."""
4
5  import numpy as np
6  from scipy.ndimage import uniform_filter1d
7  from scipy.signal import firwin, resample_poly, welch
8
9  from .constellations import fsk_levels, to_bits
10 from .dsp import _kmeans1d, _parab, analytic, mix
11
12 _CV_MAX = 0.24    # envelope coeff-of-variation ceiling. Recalibrated on a broad grid: real
13 # FSK (all h x SNR>=10 x high baud) tops out at cv 0.224, but constant-modulus PSK at very
14 # high baud (fs/baud~3) or near +-fs/2 dips to cv>=0.250 and used to slip the old 0.30 gate
15 # (false FSK). 0.24 sits in the 0.026 gap -> keeps every real FSK, rejects the PSK misfires.
16 _SEP_MIN = 3.1    # instantaneous-freq 2-means separation floor (FSK>=3.39, rest<=2.85)
17 _K4_RATIO = 2.6    # sp(k=2)/sp(k=4) collapse above which 4 levels beat 2
18 _MIN_SYMS = 8      # fewer symbols than this cannot cluster 2/4 levels (empty-quantile crash)
19 _PROM_MIN = 10.0    # a symbol-rate line peak/median below this is unreliable -- BUT only confident-
20 #                    wrong when paired with too few symbols (a long low-index burst has a weak line
21 #                    yet a correct baud). The sweep sits at prom 23-73.
22 _NSYM_MIN = 200    # ...so reject a weak line only below this symbol count. A 100 floor was tried
23 #                    (to recover more short bursts) but an independent grid found weak-prominence
24 #                    half-rate folds in the 100-199 band that still cluster tight enough to lock >=60
25 #                    (msk h=0.5 baud 24-32% off, lock 61-65) -- confident-wrong. 200 keeps them out;
26 #                    precision over recall (the short-burst confident-wrong is the cardinal sin).
27 _DECIM_FRAC = 0.30 # decimate a heavily-oversampled burst so the tones fill ~this of Nyquist
28 _DECIM_MIN = 3      # ...but only when that needs a factor >= this, so the sps~10 demod grid
29 #                    (fsk2/fsk4/msk baud=fs/10) is untouched -> the sweep stays byte-identical
30
31
32 def _bw99(x: np.ndarray, fs: float) -> float:
33     """Two-sided 99%-power occupied bandwidth (Hz), noise-pedestal subtracted."""
34     f, p = welch(x, fs=fs, nperseg=min(4096, x.size), return_onesided=False, detrend=False)
35     f, p = np.fft.fftshift(f), np.fft.fftshift(p)
36     p = np.maximum(p - np.median(p), 0.0)
37     cs = np.cumsum(p) / (p.sum() + 1e-30)
38     lo = f[min(np.searchsorted(cs, 0.005), f.size - 1)] # clamp BOTH edges: a degenerate PSD
39     hi = f[min(np.searchsorted(cs, 0.995), f.size - 1)] # (all-zero after floor subtraction)
40     return float(abs(hi - lo)) # otherwise indexes one past the end
41
42
43 def _dcblock(x: np.ndarray) -> np.ndarray:
44     """Complex128 analytic burst, DC-blocked (front-end convention)."""
45     x = analytic(x)
46     return x - x.mean()
47
48
49 def _ifreq(x: np.ndarray, fs: float) -> np.ndarray:
50     """Instantaneous frequency (Hz) from sample-to-sample phase differences."""
51     return np.angle(x[1:] * np.conj(x[:-1])) * fs / (2 * np.pi)
52
53
54 def _sep(a: np.ndarray) -> float:
55     """2-means between/within separation ratio of a 1-D sample set."""
56     c, _, sp = _kmeans1d(a, 2)
57     return abs(c[1] - c[0]) / (sp + 1e-9)
58
59
60 def _est_baud(f: np.ndarray, fs: float) -> tuple[float, float]:

```

```

61     """Symbol rate from the spectral line of |diff(smoothed f)| (transition impulses),
62     harmonic-folded to the fundamental (2-level lines alias to 2*baud). Returns
63     (baud, prominence) where prominence = peak/median of the in-band line: a weak line
64     (few symbols, no clean symbol clock) means the baud is UNRELIABLE -> caller rejects."""
65     d = np.abs(np.diff(uniform_filter1d(f, 2)))
66     d = d - d.mean()
67     n = 1 << int(np.ceil(np.log2(d.size)))
68     spec = np.abs(np.fft.rfft(d * np.hanning(d.size), n))
69     freqs = np.fft.rfftfreq(n, 1 / fs)
70     df = freqs[1] - freqs[0]
71     lo, hi = 0.002 * fs, 0.45 * fs
72     idx = np.where((freqs >= lo) & (freqs <= hi))[0]
73     prom = float(spec[idx].max() / (np.median(spec[idx]) + 1e-20)) if idx.size else 0.0
74     k = idx[int(np.argmax(spec[idx]))]
75
76     def _hp(kk: int) -> float:                                     # harmonic-comb energy: the TRUE fundamental
77         return float(sum(spec[m * kk] for m in range(1, 6) if m * kk < spec.size)) # captures most
78     # in-range harmonics ONLY: clamping out-of-range ones to the last bin multi-counted the
79     # Nyquist bin, inflating _hp at a 2*baud peak enough to veto the legitimate fold -> 2x baud.
80
81     for div in (3, 2):
82         ks = int(round(freqs[k] / div / df))
83         if freqs[ks] >= lo:
84             w = spec[max(0, ks - 2):ks + 3]
85             kk = max(0, ks - 2) + int(np.argmax(w)) if w.size else k
86             # fold to the sub-harmonic ONLY if it is genuinely the fundamental -- its harmonic comb
87             # must beat the current peak's. A real baud/2 fundamental captures MORE harmonic lines
88             # than the 2*baud peak; spurious low-h/low-SNR energy at baud/2 does not (it once
89             # slipped the 0.34 gate and folded baud -> baud/2: a confident WRONG decode). The comb
90             # guard vetoes that. On the sps~10 grid the fold never fired (0.34 unmet) -> sweep same.
91             if w.size and w.max() > 0.34 * spec[k] and _hp(kk) >= _hp(k):
92                 k = kk
93     return float(freqs[k] + _parab(20 * np.log10(spec + 1e-20), k) * df), prom
94
95
96 def fsk_gate(x: np.ndarray, fs: float) -> bool:
97     """True when the burst looks like FSK/CPM: near-constant envelope AND a bimodal
98     instantaneous frequency. Calibrated on generate() over all GEN_MODS x seeds 0..3
99     x snr {25,15}: every linear mod fails one test with margin -- bpsk/dbpsk have
100     cv~0.40 (>_CV_MAX), the rest have freq-sep<=2.85 (<_SEP_MIN); fsk2/fsk4/msk keep
101     cv<=0.22 and freq-sep>=3.39 down to snr 10 (margins ~0.08 and ~0.29).
102     Recenter by the mean instantaneous frequency first: at a large carrier offset a
103     linear mod's phase-transition spikes wrap past +/-fs/2 and fake a second mode."""
104     x = _dcblock(x)
105     a = np.abs(x)
106     if a.std() / (a.mean() + 1e-12) >= _CV_MAX:
107         return False
108     f = _ifreq(x, fs)
109     x = x * np.exp(-2j * np.pi * np.mean(f) / fs * np.arange(x.size))
110     return bool(_sep(uniform_filter1d(_ifreq(x, fs), 4)) > _SEP_MIN)
111
112
113 def analyze_fsk(x: np.ndarray, fs: float) -> dict:
114     """Full FSK/MSK demod by frequency discrimination. Returns mod, fc, baud, h,
115     lock (0-100), level_freqs (Hz, sorted, centered), symbols (normalized f per
116     symbol) and Gray-demapped bits."""
117     x = _dcblock(x)
118     f = _ifreq(x, fs)
119     # A carrier OFFSET (tones sit at fc +/- dev, not around DC) corrupts the symbol-rate estimate two
120     # ways -- off-centre it trips _est_baud's harmonic fold, and the tones fall outside the decim

```

```

121 # LPF passband -- either way a confident WRONG baud. Recentre the discriminator on the carrier
122 # (mean instantaneous freq) first; only for a significant offset, so a centred burst
123 # (fc~0, the whole demod/sweep grid) is byte-identical. Levels are already centred downstream.
124 fc0 = float(np.mean(f))
125 if abs(fc0) > 0.005 * fs:
126     x = mix(x, fs, fc0)
127     f = _ifreq(x, fs)
128 else:
129     fc0 = 0.0
130 baud, prom = _est_baud(f, fs)
131 # heavy oversampling (fs/baud >> 10) buries the symbol-rate line under broadband transition
132 # artifacts -> _est_baud locks a spurious peak (baud ~25% off) while the clean tones still
133 # cluster (lock ~94): a confident WRONG decode. Re-estimate baud on a decimated copy (tones ->
134 # ~_DECIM_FRAC of Nyquist). Gated at _DECIM_MIN so the sps~10
135 # grid is untouched (fsk2/fsk4/msk baud=fs/10 -> d<3 -> no re-estimate -> sweep byte-identical).
136 bw = _bw99(x, fs)
137 d = int(_DECIM_FRAC * fs / bw) if bw > 0 else 1
138 if d >= _DECIM_MIN:
139     xd = np.convolve(x, firwin(129, min(0.9 * (fs / d) / 2, bw) / (fs / 2)), "same")
140     baud, prom = _est_baud(_ifreq(resample_poly(xd, 1, d), fs / d), fs / d)
141 if baud <= 0:
142     raise ValueError("FSK 버스트에서 심볼율을 추정할 수 없습니다 (너무 짧음)")
143 sps = fs / baud
144 f = uniform_filter1d(f, max(1, int(round(sps / 2))))
145 resid = float(f.mean())
146 fc = fc0 + resid # absolute carrier = recentre offset + in-band residual
147 fcen = f - resid
148 nsym = int(fcen.size / sps) - 1
149 # a WEAK symbol-rate line (prom < _PROM_MIN) is unreliable ONLY when symbols are also too few to
150 # average (nsym < _NSYM_MIN): a short burst's spurious peak still clusters tight (few points ->
151 # high lock) = a confident WRONG baud. A long low-index (h~0.35) burst also has a weak line but
152 # MANY symbols -> its baud is correct -> NOT rejected. Sweep: nsym~3000/prom 23-73 -> untouched.
153 if nsym < _MIN_SYMS or (prom < _PROM_MIN and nsym < _NSYM_MIN):
154     raise ValueError(f"FSK 버스트가 너무 짧거나 심볼율이 불확실합니다 ({nsym} 심볼)")
155 ctr = (np.arange(nsym) + 0.5) * sps
156
157 # symbol sampling phase: the offset whose sampled levels separate best
158 v, best = fcen[:nsym], -1.0
159 for o in np.linspace(0, sps, 8, endpoint=False):
160     s = fcen[np.clip((ctr + o).astype(int), 0, fcen.size - 1)]
161     sep = _sep(s)
162     if sep > best:
163         best, v = sep, s
164
165 # level count via spread-collapse (k=2 vs k=4), then cluster
166 k = 4 if _kmeans1d(v, 2)[2] / (_kmeans1d(v, 4)[2] + 1e-9) > _K4_RATIO else 2
167 c, lab, spread = _kmeans1d(v, k)
168 order = np.argsort(c)
169 ranks = np.argsort(order)[lab] # per-symbol level index, 0 = lowest freq
170 csort = c[order]
171 mid = float(csort.mean()) # debias center: symmetric levels sum ~0
172 fc, csort, v = fc + mid, csort - mid, v - mid
173 spacing = float(np.mean(np.diff(csort)))
174 h = spacing / baud
175
176 mod = "msk" if (k == 2 and abs(h - 0.5) < 0.15) else f"fsk{k}"
177 _, glabels = fsk_levels(mod)
178 bits = to_bits(glabels[ranks], mod).astype(np.uint8)
179 lock = float(np.clip((spacing / (spread + 1e-9) - 1.5) / 6.5 * 100, 0, 100))
180 return dict(mod=mod, fc=fc, baud=baud, h=h, lock=lock, level_freqs=csort,

```

100

```

1  """Blind linear-chirp / CSS (LoRa) detection + characterization. A constant-envelope
2  signal whose instantaneous frequency ramps linearly de-chirps to a tone: the delay-
3  conjugate  $x[t] \cdot \text{conj}(x[t-\tau])$  is a beat tone at  $\mu \cdot \tau$  (NONZERO), unlike FSK/CW (beat at
4  DC), PSK/QAM (not constant-envelope), or FM-voice/noise (broadband). Characterize-only --
5  reported like analog/tone, NEVER demodulated: blind CSS symbol decode needs preamble
6  CF0/ST0 sync plus LoRa's reverse-engineered Gray/interleave/Hamming/whitening framing.
7  Calibrated on gen chirps vs every other family: chirp beat-PAR $\geq$ 936, non-chirp $\leq$ 264."""
8
9  import numpy as np
10 from scipy.ndimage import uniform_filter1d
11 from scipy.signal import welch
12
13 _PAR_MIN = 400.0  # delay-conjugate beat-tone peak-to-mean floor (chirp $\geq$ 936, non-chirp $\leq$ 264)
14
15
16 def _bw99(x: np.ndarray, fs: float) -> float:
17     """99%-power occupied bandwidth (a chirp fills its band ~flat). Floor-subtracted so the
18     noise pedestal in the channel's LPF passband does not inflate the width."""
19     f, p = welch(x, fs=fs, nperseg=min(4096, x.size), return_onesided=False, detrend=False)
20     f, p = np.fft.fftshift(f), np.fft.fftshift(p)
21     p = np.maximum(p - np.median(p), 0.0)  # remove the noise pedestal
22     cs = np.cumsum(p) / (p.sum() + 1e-30)
23     lo = f[min(np.searchsorted(cs, 0.005), f.size - 1)]  # clamp BOTH edges: a degenerate PSD
24     hi = f[min(np.searchsorted(cs, 0.995), f.size - 1)]  # (all-zero after floor subtraction)
25     return float(abs(hi - lo))  # otherwise indexes one past the end
26
27
28 def _beat(x: np.ndarray, fs: float) -> tuple[float, float]:
29     """(peak-to-mean,  $\mu$ [Hz/s]) of the DC-nulled delay-conjugate spectrum."""
30     x = x - x.mean()
31     tau = max(1, x.size // 200)
32     y = x[tau:] * np.conj(x[:-tau])
33     nf = 1 << int(np.ceil(np.log2(y.size)))
34     spec = np.abs(np.fft.fftshift(np.fft.fft((y - y.mean()) * np.hanning(y.size), nf))) ** 2
35     fr = np.fft.fftshift(np.fft.fftfreq(nf, 1 / fs))
36     spec[np.abs(fr) < 0.002 * fs] = 0.0  # null DC band: FSK/CW/tone beat here, chirps do not
37     k = int(np.argmax(spec))
38     return float(spec[k] / (spec.mean() + 1e-30)), float(fr[k] * fs / tau)
39
40
41 def is_chirp(x: np.ndarray, fs: float) -> bool:
42     """True when a channel is a linear chirp / CSS (LoRa/FMCW). The beat-tone PAR is the
43     discriminator: a huge margin (chirp $\geq$ 936 vs PSK/QAM $\leq$ 94, FSK $\leq$ 91, FM $\leq$ 264, noise $\leq$ 13)
44     makes an envelope-CV pre-gate redundant AND harmful (it rejects noisy-but-clear chirps)."""
45     if x.size < 256:
46         return False
47     return _beat(x, fs)[0] >= _PAR_MIN
48
49
50 _SWEEP_PM_MAX = 1.45  # instantaneous-freq histogram peak-to-mean: a band-sweep stays ~1
51 _SWEEP_OCC_MIN = 0.6  # ...and fills its band; FSK sits on 2-4 discrete tones (peaky, sparse)
52
53
54 def sweeps_band(x: np.ndarray, fs: float, bins: int = 40) -> bool:
55     """True when the instantaneous frequency SWEEPS the channel (linear chirp / CSS) rather
56     than hopping between a few discrete FSK tones. A linear FMCW ramp reads bimodal to
57     fsk_gate AND trips is_chirp's beat test, so analyze's chirp gate needs this to tell a sweep
58     from FSK: a sweep's IF histogram is flat and fully occupied; FSK's spikes at its tones.
59     Calibrated on extracted channels: 0/222 FSK/MSK (snr 8-40) pass, 214/216 FMCW + 60/60 LoRa
60     pass (misses only the narrowest, gentlest ramps -- which stay 'fsk', never a regression).

```

```

61 BOTH the raw and a lightly-smoothed IF must look like a sweep: at moderate SNR (~14) noise
62 flattens integer-h FSK's raw histogram to sweep-like pm ~1.44 (knife-edge), but smoothing
63 restores its tone spikes (pm 2.5+) while a genuine ramp stays flat (pm ~1.1) -- and at LOW
64 SNR smoothing can flatten FSK instead, which the raw view still catches. The conjunction
65 only ever narrows 'sweep', so every calibrated chirp above keeps its label (margin >=0.3)."""
66 if x.size < 256:
67     return False
68 x = x - x.mean()
69 f0 = np.diff(np.unwrap(np.angle(x))) * fs / (2 * np.pi)
70 for f in (f0, uniform_filter1d(f0, 4)):
71     lo, hi = np.percentile(f, 1), np.percentile(f, 99)
72     if hi - lo < 1:
73         return False
74     fv = f[(f >= lo) & (f <= hi)]
75     h = np.histogram(fv, bins=bins)[0] / fv.size
76     pm = h.max() / (h.mean() + 1e-30) # discrete tones spike this; a sweep ~1
77     occ = float((h > 0.005).mean()) # a sweep fills the band; tones occupy few bins
78     if not (pm < _SWEEP_PM_MAX and occ >= _SWEEP_OCC_MIN):
79         return False
80 return True
81
82
83 def analyze_chirp(x: np.ndarray, fs: float) -> dict:
84     """Characterize a chirp: {mu[Hz/s], up, bw, par, sf, rs, tsym}. A LoRa hypothesis
85     (sf, symbol rate, symbol time) is filled when bw^2/|mu| snaps to 2^(7..12); otherwise
86     it stays a generic linear chirp / FMCW. bw is measured on the channel, not asserted."""
87     par, mu = _beat(x, fs)
88     bw = _bw99(x, fs)
89     sf = rs = tsym = None
90     if mu != 0 and bw > 0:
91         r = float(np.log2(bw * bw / abs(mu)))
92         if 7 <= round(r) <= 12 and abs(r - round(r)) < 0.3: # power-of-two chip count -> LoRa
93             sf = int(round(r))
94             rs = abs(mu) / bw
95             tsym = 2 ** sf / bw
96     return {"mu": mu, "up": bool(mu > 0), "bw": float(bw),
97             "par": par, "sf": sf, "rs": rs, "tsym": tsym}

```

```

1  """End-to-end blind demodulation. Two families:
2      linear (PSK/QAM): front-end -> carrier -> baud -> matched filter -> timing ->
3                          classify -> fine sync -> [equalizer rescue] -> quality
4      fsk (CPFSK/MSK): frequency discriminator (gated by constant envelope)
5  Classification runs BEFORE fine sync so the decision-directed loop knows the
6  constellation. Differential demapping is an option, not a detection: dqpsk and
7  qpsk share a constellation."""
8
9  import json
10 from dataclasses import dataclass, field
11
12 import numpy as np
13 from scipy.signal import firwin, resample_poly
14
15 from . import classify as cl
16 from . import dsp
17 from .chirp import _bw99, analyze_chirp, is_chirp, sweeps_band
18 from .constellations import demap_bits, demap_diff_bits, mod_order
19 from .eq import equalize, equalize_fse
20 from .fsk import analyze_fsk, fsk_gate
21 from .sigio import Meta, parse_name, read
22 from .spectrum import spectrum, strip
23 from .sync import find_preamble
24
25 _BITS_CAP = 65536
26 _EQ_LOCK = 60.0    # below this a linear signal is worth an equalizer attempt
27 _EQ_GAIN = 3.0     # ...keep it only if lock improves by this much
28 _EQ_FLOOR = 50.0   # ...and clears this floor, so a wrong-mod fit is never locked in
29 _EQ_QAM_FLOOR = 60.0 # ...but a DENSE-QAM (32/64) eq fit must clear a higher bar: the DD loop can
30 #                   drag a lower-order cloud onto a 32/64 lattice at a marginal lock (~51), a
31 #                   confident WRONG order. Genuine multipath recoveries clear ~72; the sweep's
32 #                   eq cases are qpsk/16qam (never 32/64 via eq) so this bar is byte-identical.
33 _BAUD_GAIN = 5.0   # an in-band baud hypothesis must beat the global one by this lock
34 _SHORT_LOCK = 60.0 # below this, retry the baud line with fewer blocks (short-burst rescue)
35 _SYNC_LOCK = 60.0  # below this, try a repeated-preamble carrier/timing sync (packetized bursts)
36 _ALIAS_MARGIN = 10.0 # a preamble carrier alias must beat the next by this lock, else ambiguous.
37 #                   This is the LOAD-BEARING rotation guard: even when the coarse anchor itself
38 #                   aliases and validates a rotation at ~80 lock, that rotation wins by a thin
39 #                   margin (~8) -- so a 10-lock margin refuses it. Because the margin catches it,
40 #                   the lock floor can stay modest (78, not 82) -> ~5% more genuine recoveries.
41 _SYNC_ACCEPT = 78.0 # ...AND clear this absolute lock: below it correct/rotated aliases overlap. A
42 #                   65 floor was tried (to recover more moderate bursts) but an independent
43 #                   snr-12 grid found 8psk rotated aliases locking 71-72 in the opened 65-78 band
44 #                   (carrier off by one baud/L step -> confident garbage bits). Even 78 leaks a
45 #                   a few hard-corner rotations (a separate pre-existing gate limit): hold at 78.
46 _SYNC_ADIST = 1.5  # ...AND sit within this many alias steps of the coarse est_carrier fc -- a
47 #                   soft backstop to the margin: it catches the one rotation the margin misses,
48 #                   but tuned LOOSE (1.5, not 0.75) so it does not needlessly reject genuine
49 #                   large-offset recoveries. The three together give 0 confident-wrong across
50 #                   1920 hard-corner bursts at the max recall this gate reaches.
51 _DIFF_OF = {"bpsk": "dbpsk", "qpsk": "dqpsk"} # pi4dqpsk arrives as qpsk -> dqpsk
52
53
54 @dataclass
55 class Result:
56     meta: Meta
57     family: str # 'linear' | 'fsk' | 'chirp'
58     burst: tuple[int, int]
59     fc: float
60     baud: float

```

```

61     mod: str
62     lock: float
63     symbols: np.ndarray          # aligned symbols (linear) / normalized levels (fsk)
64     bits: np.ndarray
65     iq_corr: np.ndarray          # exportable corrected baseband
66     burst_x: np.ndarray = field(repr=False, default=None) # for spectrum views
67     symmetry: int = 0
68     carrier_ambiguous: bool = False
69     baud_conf: float = 0.0
70     rolloff: float | None = None
71     h: float | None = None       # FSK modulation index
72     evm: float = float("nan")
73     mer_db: float = float("nan")
74     occupied: int = 0
75     snr_est: float = float("nan")
76     eq_applied: bool = False
77     eq_mode: str | None = None   # 'sym' | 'fse'
78     alias_resolved: bool = False
79     baud_fallback: bool = False
80     bursts: list = field(default_factory=list)
81     burst_idx: int = 0
82     preamble: object = None     # sync.Preamble when a repeated preamble drove the decode
83     chirp: dict | None = None   # chirp/CSS characterization (family 'chirp': no symbols)
84
85     def to_json(self, max_points: int = 6000, views: bool = True) -> dict:
86         z = np.nan_to_num(self.symbols) # a dead/DC capture -> NaN symbols -> invalid JSON
87         if z.size > max_points: # keep TIME ORDER: the UI animates this sequence
88             z = z[np.linspace(0, z.size - 1, max_points).astype(int)]
89         det = {"fc": round(self.fc, 3), "baud": round(self.baud, 3), "mod": self.mod,
90              "symmetry": self.symmetry, "carrier_ambiguous": self.carrier_ambiguous,
91              "baud_conf": round(self.baud_conf, 1),
92              "alias_resolved": self.alias_resolved,
93              "baud_fallback": self.baud_fallback}
94         det["rolloff"] = None if self.rolloff is None else round(self.rolloff, 3)
95         rf = self.meta.rf_center # real RF = capture centre + baseband fc
96         det["rf_hz"] = None if rf is None else round(rf + self.fc, 3)
97         if self.h is not None:
98             det["h"] = round(self.h, 3)
99         if self.preamble is not None: # a repeated preamble drove the sync
100             p = self.preamble
101             det["preamble"] = {"period": p.period, "cfo_hz": round(p.cfo_hz, 1),
102                               "conf": round(p.conf, 2), "start": p.start, "end": p.end}
103         if self.chirp is not None: # chirp/CSS characterization -- no constellation fields
104             c = self.chirp
105             det["chirp"] = {"mu": _r(c["mu"], 1), "up": bool(c["up"]), "bw": _r(c["bw"], 1),
106                             "sf": c["sf"], "rs": _r(c["rs"], 1), "tsym": _r(c["tsym"], 6)}
107         doc = {
108             "fs": self.meta.fs, "fmt": self.meta.fmt, "dtype": self.meta.dtype, # dtype 은 한 줄
109             "rf_center": self.meta.rf_center, # 보고에 필수: lock 0 의 1순위 용의자다
110
111             "family": self.family, "n_samples": int(self.iq_corr.size),
112             "burst": {"start": self.burst[0], "end": self.burst[1]},
113             "bursts": [{"start": s, "end": e} for s, e in self.bursts],
114             "burst_idx": self.burst_idx,
115             "detected": det,
116             "quality": {"lock": _r(self.lock, 1) or 0.0, "mer_db": _r(self.mer_db),
117                        "evm": _r(self.evm, 4), "occupied": int(self.occupied)},
118             "snr_est_db": _r(self.snr_est),
119             "eq": {"applied": self.eq_applied, "mode": self.eq_mode},
120             "constellation": {"i": np.round(z.real, 4).tolist(),

```

```

121         "q": np.round(z.imag, 4).tolist()},
122         "bits": "".join(map(str, self.bits[:_BITS_CAP])),
123     }
124     if views and self.burst_x is not None:
125         doc["spectrum"] = spectrum(self.burst_x, self.meta.fs)
126     return doc
127
128     def save_iq(self, path: str) -> None:
129         np.column_stack([self.iq_corr.real, self.iq_corr.imag]).astype("<f4").tofile(path)
130
131     def save_symbols(self, path: str) -> None:
132         np.save(path, self.symbols.astype(np.complex64))
133
134     def save_bits(self, path: str, packed: bool = False) -> None:
135         if packed:
136             np.packbits(self.bits).tofile(path)
137         else:
138             with open(path, "w") as fh:
139                 fh.write("".join(map(str, self.bits)))
140
141     def save_report(self, path: str) -> None:
142         with open(path, "w") as fh:
143             json.dump(self.to_json(), fh, indent=1)
144
145
146     def _r(v: float, nd: int = 2) -> float | None:
147         return None if v is None or not np.isfinite(v) else round(float(v), nd)
148
149
150     def _fine(z: np.ndarray, mod: str) -> np.ndarray:
151         """Bulk-phase removal, decision-directed carrier loop, final rotation lock."""
152         return cl.align(dsp.ddsync(cl.align(z, mod), mod), mod)
153
154
155     def _blocks(n: int) -> int:
156         """M-th-power spectra are averaged INCOHERENTLY, so a weak tone (high symmetry,
157         low SNR) wants fewer, longer segments; only long records need more for
158         stationarity. Worth ~2 dB at the 8PSK edge over a fixed blocks=4."""
159         return int(np.clip(n // 30000, 1, 8))
160
161
162     def _demod(xd: np.ndarray, fs: float, baud: float, symmetry: int) -> tuple:
163         """Matched filter + timing + classify + fine sync at one baud hypothesis."""
164         alpha = dsp.est_rolloff(xd, fs, baud)
165         ym = dsp.matched(dsp.to_sps(xd, fs, baud), 4, alpha)
166         raw = dsp.timing(ym, 4)
167         mod = cl.classify(raw, symmetry)
168         # align BEFORE ddsync: the coarse carrier leaves only micro-Hz residual, so removing
169         # the bulk phase first kills the loop transient that would corrupt the first symbols
170         syms = _fine(raw, mod)
171         return alpha, ym, raw, mod, syms, cl.quality(syms, mod)
172
173
174     def _rescue(eq_fn, z: np.ndarray, seed_mod: str, symmetry: int) -> tuple:
175         """Equalize with a seed constellation, then let the OPENED eye pick the mod:
176         heavy ISI corrupts the ring test and even the 2/8 symmetry vote, so re-classify
177         with the estimated symmetry AND the neutral order-4 gate, keep the best lock."""
178         z1 = eq_fn(z, seed_mod)
179         cand_s = []
180         for m in {cl.classify(z1, symmetry), cl.classify(z1, 4)}: # symmetry too, per the docstring

```

```

181         s = _fine(z1 if m == seed_mod else eq_fn(z, m), m)
182         cand_s.append((cl.quality(s, m), s, m))
183     best = max(cands, key=lambda c: c[0].lock)
184     # Occam de-fold for the PSK point-doubling: CMA is modulus-only, so a bpsk signal under a
185     # multipath echo ALSO fits a qpsk ring at a marginally higher lock (phantom 2->4 points).
186     # A bpsk candidate that clears the accept floor cannot be an over-fit of qpsk, so prefer it.
187     # Only fires when symmetry==2 put bpsk in the set; a genuine qpsk reads symmetry 4 -> the
188     # candidate set is the qpsk singleton -> this branch is unreachable and it stays untouched.
189     lo = [c for c in cands if c[2] == "bpsk" and c[0].lock >= _EQ_FLOOR]
190     if lo and best[2] == "qpsk":
191         return lo[0]
192     return best
193
194
195 def analyze(x: np.ndarray, meta: Meta, diff: bool = False,
196            burst: int | None = None) -> Result:
197     """Run the blind chain. `diff` demaps phase transitions (D-BPSK/D-QPSK);
198     `burst` selects one detected burst (default: the most energetic)."""
199     if x.size:
200         # pathological UNIFORM scale must be fixed BEFORE analytic(): its magnitude sanitizer
201         # zeroes samples above 1e150 sample-WISE, so a whole capture near/over that ceiling was
202         # wiped wholesale (garbage mods, lock=nan at 1e154) before the rms-normalize below could
203         # ever help. The 99th percentile is robust to spike outliers (a single corrupt 1e300
204         # sample leaves it at signal scale -> untouched here, still zeroed sample-wise later);
205         # normal captures sit far inside (1e-100, 1e100) -> untouched -> sweep byte-identical.
206         scale = float(np.percentile(np.abs(x[:16]), 99)) # strided: scale detection needs the
207         # BULK magnitude, not every sample -- 16x cheaper on a large direct capture
208         if np.isfinite(scale) and scale > 0 and not (1e-100 < scale < 1e100):
209             x = x / scale
210     x = dsp.analytic(x)
211     x = x - x.mean() # DC block: LO leakage otherwise corrupts every estimator
212     if x.size < 64: # empty / trivially short: nothing to estimate, and it crashes downstream
213         raise ValueError(f"신호가 너무 짧습니다 ({x.size} 샘플)")
214     rms = float(np.sqrt(np.mean(np.abs(x) ** 2)))
215     if rms and (rms < 1e-6 or rms > 1e6): # only PATHOLOGICAL scale: normalize so the scale-variant
216         x = x / rms # spots (fsk_gate's CV epsilon, est_carrier's x**p over/
217         # underflow) stay invariant. Normal captures rms~0(1) ->
218         # untouched -> the acceptance sweep is byte-identical.
219     bursts = dsp.find_bursts(x, meta.fs)
220     if burst is None or not 0 <= burst < len(bursts):
221         burst = int(np.argmax([np.sum(np.abs(x[s:e]) ** 2) for s, e in bursts]))
222     s, e = bursts[burst]
223     xb = x[s:e] if e - s >= 64 else x
224
225     # chirp gate: an FMCW/CSS sweep trips fsk_gate (bimodal IF) and would force-demodulate
226     # into confident garbage FSK. sweeps_band keeps real FSK (0/222 pass) out of this branch,
227     # so only a genuine band-sweep lands here -- characterized, never constellation-demodulated.
228     # The gates read a band-ISOLATED copy: mix to the spectral centroid, low-pass to the
229     # 99%-power band, and decimate so the signal fills ~28% of Nyquist. An off-centre
230     # narrowband sweep in a wide record otherwise dilutes the IF statistics with
231     # out-of-band noise and leaks through as garbage FSK.
232     pw = np.abs(np.fft.fft(xb)) ** 2 # sweep centre = spectral centroid
233     fr = np.fft.fftfreq(xb.size, 1 / meta.fs) # (est_carrier means nothing on a mover)
234     fcg = float(np.sum(fr * pw) / (np.sum(pw) + 1e-30))
235     xg = dsp.mix(xb, meta.fs, fcg)
236     bwg = _bw99(xg, meta.fs)
237     want = max(1.25 * bwg, 1e-4 * meta.fs)
238     d = int(np.clip(0.28 * meta.fs / want, 1, max(1, xg.size // 256)))
239     fsg = meta.fs / d
240     xg = np.convolve(xg, firwin(129, max(min(0.9 * fsg / 2, bwg), 1e-4 * meta.fs))

```

```

241         / (meta.fs / 2)), "same")
242     if d > 1:
243         xg = resample_poly(xg, 1, d)
244     if is_chirp(xg, fsg) and sweeps_band(xg, fsg):
245         info = analyze_chirp(xg, fsg)
246         return Result(meta, "chirp", (s, e), fcg, info["rs"] or 0.0,
247                                "lora" if info["sf"] else "chirp", 0.0, np.zeros(0, complex),
248                                np.zeros(0, np.uint8), x, burst_x=xb,
249                                bursts=bursts, burst_idx=burst, chirp=info)
250
251     if fsk_gate(xb, meta.fs):
252         r = analyze_fsk(xb, meta.fs)
253         sym = np.asarray(r["symbols"], dtype=np.complex128)
254         return Result(meta, "fsk", (s, e), r["fc"], r["baud"], r["mod"], r["lock"],
255                                sym, r["bits"], xb, burst_x=xb, h=r["h"],
256                                bursts=bursts, burst_idx=burst)
257
258     fc0, symmetry, _ = dsp.est_carrier(xb, meta.fs, blocks=_blocks(xb.size))
259     fc = dsp.resolve_alias(xb, meta.fs, fc0, symmetry)
260     amb = abs(symmetry * fc) > 0.4 * meta.fs
261     xd = dsp.mix(xb, meta.fs, fc)
262
263     baud, conf = dsp.est_baud(xd, meta.fs)
264     fell = False
265     alpha, ym, raw, mod, syms, q = _demod(xd, meta.fs, baud, symmetry)
266     bw = dsp.occupied_bw(xd, meta.fs)
267     if bw and not (0.72 * bw <= baud <= 1.05 * bw):
268         # the global |x|^2 peak disagrees with the occupied band: below alpha~0.1
269         # (and under harsh channels) data junk outruns the true line. Demod the
270         # in-band hypotheses too and let lock decide, with a clear margin.
271         for lo in (0.80, 0.67):
272             b2, c2 = dsp.est_baud(xd, meta.fs, lo=lo * bw, hi=1.02 * bw)
273             if abs(b2 - baud) <= 0.01 * b2:
274                 continue
275             t = _demod(xd, meta.fs, b2, symmetry)
276             if t[5].lock > q.lock + _BAUD_GAIN:
277                 alpha, ym, raw, mod, syms, q = t
278                 baud, conf, fell = b2, c2, True
279
280     if q.lock < _SHORT_LOCK:
281         # short / low-SNR burst rescue: the default 4-block |x|^2 line splits the record
282         # too finely and locks a junk peak. Fewer, longer blocks give a stronger line (the
283         # carrier_blocks logic, applied to baud). Run both and keep only a clearly-better
284         # lock -- so a long, already-locked signal (CORE) never enters here and is untouched.
285         for nb in (2, 1):
286             b2, c2 = dsp.est_baud(xd, meta.fs, blocks=nb)
287             if abs(b2 - baud) <= 0.01 * b2:
288                 continue
289             t = _demod(xd, meta.fs, b2, symmetry)
290             if t[5].lock > q.lock + _BAUD_GAIN:
291                 alpha, ym, raw, mod, syms, q = t
292                 baud, conf, fell = b2, c2, True
293
294     eq_applied, eq_mode, pre = False, None, None
295     if q.lock < _EQ_LOCK: # ISI (multipath) is what a phase-only loop cannot fix
296         # CMA is modulus-only, so it opens the eye without knowing the constellation.
297         qe, se, me = _rescue(lambda z, m: equalize(z, m), raw, mod, symmetry)
298         mode = "sym"
299         if qe.lock < max(_EQ_FLOOR, q.lock + _EQ_GAIN):
300             # symbol-spaced FIR could not invert the channel; retry T/2-spaced on

```

```

301         # the clock-tracked 2 sps stream (unaligned spectrum, longer inverse)
302         q2, s2, m2 = _rescue(lambda z, m: equalize_fse(z, m),
303                               dsp.timing(ym, 4, out=2), mod, symmetry)
304         if q2.lock > qe.lock:
305             qe, se, me, mode = q2, s2, m2, "fse"
306         floor = _EQ_QAM_FLOOR if me in ("32qam", "64qam") else _EQ_FLOOR
307         if qe.lock > q.lock + _EQ_GAIN and qe.lock >= floor:
308             syms, mod, q, eq_applied, eq_mode = se, me, qe, True, mode
309     elif mod in ("16qam", "32qam", "64qam"):
310         # confident square-QAM at high lock: a benign post-echo can fold a LOWER-order signal
311         # onto a QAM lattice with a clean-looking eye, so the low-lock rescue above never fires.
312         # Equalize once and accept ONLY if a strictly lower order emerges -- verified never to
313         # demote a genuine QAM (0/36 across 16/32/64qam x snr x seeds), so real QAM is untouched.
314         qe, se, me = _rescue(lambda z, m: equalize(z, m), raw, mod, symmetry)
315         mode = "sym"
316         if mod_order(me) < mod_order(mod) and qe.lock < _EQ_FLOOR:
317             # a lower order emerged but the symbol-spaced eye stayed shut (echo > ~2 symbols);
318             # retry T/2 fractionally-spaced. Only runs once a de-fold is INDICATED, so a genuine
319             # QAM (no lower order) never pays for the FSE.
320             qe, se, me = _rescue(lambda z, m: equalize_fse(z, m),
321                                   dsp.timing(ym, 4, out=2), mod, symmetry)
322             mode = "fse"
323         if mod_order(me) < mod_order(mod) and qe.lock >= _EQ_FLOOR:
324             syms, mod, q, eq_applied, eq_mode = se, me, qe, True, mode
325
326     if q.lock < _SYNC_LOCK:
327         # packetized-burst rescue -- LAST, after the equalizer: a repeated preamble pins the
328         # carrier/ baud/ timing from only a few symbols, where the blind M-th-power estimator (needing
329         # many symbols to average) fails. Runs only if the eq rescue above did NOT lift the lock, so
330         # a multipath signal (which the eq handles, and whose ISI can fake a short period) never
331         # reaches here. Detect the preamble, mix by its CFO, demod the DATA past it, keep-best -- a
332         # random / no-preamble signal yields no plateau or a CFO that does not improve lock and is
333         # dropped (the sweep stays byte-identical). The preamble period P = L*sps supplies the baud
334         # (fs*L/P per block length L), the phase slope gives the carrier (+-fs/P resolves the L-fold
335         # alias); every PSK/QAM symmetry is tried -- the right (carrier, baud, sym) locks clean.
336         ps = find_preamble(xb, meta.fs, baud_hint=baud)
337         if ps is not None:
338             # The Schmidl-Cox CFO is ambiguous modulo fs/P = baud/L: an M-PSK carrier off by baud/L
339             # rotates a whole constellation position per symbol, so the eye still looks locked but
340             # the bits shift -> a confident WRONG decode if the wrong alias is trusted. The coarse
341             # M-th-power fc lands within ~1 alias step of the truth even on these short bursts, so
342             # CENTRE the alias scan on it (k0) and sweep +-3 steps -- this reaches the correct alias
343             # for any offset (not just |fc|<baud/2); keep-best over them picks the cleanest eye.
344             step = meta.fs / ps.period
345             k0 = round((fc - ps.cfo_hz) / step)
346             cands = []
347             for b2 in sorted({round(meta.fs * L / ps.period) for L in (3, 4, 6, 8)}):
348                 for k in range(k0 - 3, k0 + 4):
349                     cfo = ps.cfo_hz + k * step
350                     data = dsp.mix(xb, meta.fs, cfo)[ps.end:]
351                     if data.size < 256:
352                         continue
353                     for sm in (2, 4, 8):
354                         t = _demod(data, meta.fs, float(b2), sm)
355                         cands.append((t[5].lock, cfo, float(b2), sm, t))
356             cands.sort(key=lambda c: -c[0])
357             # Adjacent aliases both look like clean M-PSK, so lock alone can pick the WRONG one by a
358             # hair. Require the winner to beat the best DIFFERENT-carrier candidate by a clear gap;
359             # otherwise the alias is genuinely ambiguous -> leave the honest low-lock result rather
360             # than emit a confident wrong decode. (Same-carrier baud/sym variants do not compete.)

```

```

361 # PRECISION GATE: the alias ambiguity (carrier off by baud/L) is blind-ambiguous -- a
362 # rotated alias still locks AS HIGH AS the truth (both are clean M-PSK), so lock alone
363 # cannot separate them (a rotated 8psk alias was measured locking 78 with ber 0.38). Two
364 # conjunctive conditions favour precision over recall: (1) a strong absolute lock, and
365 # (2) the winner sits within _SYNC_ADIST alias steps of the coarse est_carrier fc -- a
366 # baud/L rotation is >=1 step off the true carrier, so a far-from-anchor winner is a
367 # rotation. Genuine recoveries have a near-anchor carrier (adist<=0.5, lock 88+); on the
368 # hardest bursts the anchor itself is unreliable -> both conditions fail -> refuse.
369 if cands and cands[0][0] >= _SYNC_ACCEPT and cands[0][0] > q.lock + _EQ_GAIN:
370     top = cands[0]
371     other = next((c for c in cands if abs(c[1] - top[1]) > step / 2), None)
372     anchored = abs(top[1] - fc) <= _SYNC_ADIST * step
373     if anchored and (other is None or top[0] - other[0] >= _ALIAS_MARGIN):
374         alpha, ym, raw, mod, syms, q = top[4]
375         fc, baud, symmetry = top[1], top[2], top[3]
376         eq_applied, eq_mode, pre = False, None, ps
377         # diagnostics must describe the ACCEPTED decode, not the abandoned blind
378         # attempt: the carrier came from the preamble (not resolve_alias -> fc0=fc
379         # keeps alias_resolved False), the baud from the preamble period (no
380         # spectral line -> conf 0, no fallback), and ambiguity follows the new fc.
381         fc0, conf, fell = fc, 0.0, False
382         amb = abs(symmetry * fc) > 0.4 * meta.fs
383
384 bits = demap_diff_bits(syms, _DIFF_OF[mod]) if diff and mod in _DIFF_OF \
385     else demap_bits(syms, mod)
386 return Result(meta, "linear", (s, e), fc, baud, mod, q.lock, syms, bits, ym,
387     burst_x=xb, symmetry=symmetry, carrier_ambiguous=amb, baud_conf=conf,
388     rolloff=alpha, evm=q.evm, mer_db=q.mer_db, occupied=q.occupied,
389     snr_est=cl.snr_m2m4(syms, mod), eq_applied=eq_applied, eq_mode=eq_mode,
390     alias_resolved=abs(fc - fc0) > 1.0, baud_fallback=fell,
391     bursts=bursts, burst_idx=burst, preamble=pre)
392
393
394 def analyze_file(path: str, fs: float | None = None, fmt: str | None = None,
395     dtype: str | None = None, endian: str | None = None,
396     bitrev: bool | None = None, diff: bool = False,
397     burst: int | None = None, rf: float | None = None) -> Result:
398     """Analyze a file; explicit args override SigMF/filename tokens."""
399     from .sigio import parse_sigmf
400     m = parse_sigmf(path) or parse_name(path)
401     meta = Meta(fs or m.fs, fmt or m.fmt, dtype or m.dtype,
402         endian or m.endian, m.bitrev if bitrev is None else bitrev,
403         rf_center=rf if rf is not None else m.rf_center)
404     x, meta = read(path, meta)
405     return analyze(x, meta, diff, burst)
406
407
408 # --- web payload -----
409
410 def analyze_web(x: np.ndarray, meta: Meta, *, burst: int | None = None) -> dict:
411     """Payload for POST /api/analyze: the analyze() result as JSON, plus -- on the first
412     load of a multi-burst capture -- a whole-record spectrogram strip so the UI can draw
413     the burst map. Burst re-selection posts again with ?burst=N and skips the overview
414     (the UI keeps the one it already has)."""
415     r = analyze(x, meta, burst=burst)
416     out = r.to_json()
417     if burst is None and len(r.bursts) > 1:
418         out["overview"] = {"n": int(x.size), "fs": meta.fs, "strip": strip(x, meta.fs)}
419     return out

```

```

1  """CLI: analyze FILE / serve. 합성·채점(gen/dataset/sweep)은 개발기 전용 lab.py 가
2  단다 -- 격리망 장비에는 그 파일이 없어 명령 자체가 없다."""
3
4  import argparse
5  import importlib.util
6  import re
7  import sys
8  from zlib import crc32
9
10 from .pipeline import analyze_file
11 from .sigio import sidecar_read
12
13 _DTYPE_CHOICES = ("i8", "u8", "i16", "u16", "f32", "f64")
14
15
16 # --- 손으로 옮기는 한 줄 -----
17 # 실신호 장비는 인터넷도 클립보드도 없다. 결과는 사람이 화면을 보고 받아쳐서 나온다. 그래서
18 # 이 블록은 반드시 여기(필사 대상)에 있어야 한다 -- tools/ 에 두면 그 도구부터 필사해야 하는
19 # 본말전도가 된다. 줄 끝의 검출 코드가 "옮기다 한 글자 틀림"을 잡는다.
20
21 _ALPHA = "0123456789abcdefghijklmnopqrstuvwxyz" # 0/o, 1/l/i 처럼 화면에서 헛갈리는 글자를 뺀 32자
22 _BRIEF_FLAGS = [("eq", lambda d: d["eq"]["applied"]),
23                 ("al", lambda d: d["detected"]["alias_resolved"]),
24                 ("fb", lambda d: d["detected"]["baud_fallback"]),
25                 ("amb", lambda d: d["detected"]["carrier_ambiguous"]),
26                 ("pre", lambda d: "preamble" in d["detected"])]
27
28
29 def check_code(text: str) -> str:
30     """받아친 줄의 오타 검출 코드 (4글자 = 20비트). 공백은 '한 칸으로 줄이되 없애지는'
31     않는다 -- 전부 지우면 'fs1000000 16qam' 과 'fs10000001 6qam' (샘플레이트 10배!) 이 같은
32     코드가 되어, 값이 바뀌는 오타 32종이 조용히 통과했다. 대소문자와 줄 간격은 계속 무시한다."""
33     body = re.sub(r"\s+", " ", re.sub(r"#[0-9a-z]{4}\b", "", text.lower())).strip()
34     n = crc32(body.encode())
35     return "".join(_ALPHA[(n >> (5 * i)) & 31] for i in (3, 2, 1, 0))
36
37
38 def brief(doc: dict, mode: str) -> str:
39     """손으로 옮기는 한 줄 + 검출 코드. Result.to_json() 디렉터리에서 만든다 -- 객체
40     속성을 직접 포맷하면 to_json 이 이미 한 반올림과 두 번 겹쳐 장비와 맥이 갈린다."""
41     head = f"sig2 {mode} fs{doc['fs']:.0f} {doc['fmt']}--{doc['dtype']}"
42     d, q = doc["detected"], doc["quality"]
43     p = [head, d["mod"], f"fc{d['fc']:.0f}", f"bd{d['baud']:.0f}", f"lk{q['lock']:.0f}"]
44     if q["mer_db"] is not None:
45         p.append(f"mer{q['mer_db']:.1f}")
46     if d.get("h") is not None: # FSK 는 롤오프 대신 변조지수
47         p.append(f"h{d['h']:.2f}")
48     elif d["rolloff"] is not None:
49         p.append(f"rl{d['rolloff']:.2f}")
50     if len(doc["bursts"]) > 1:
51         p.append(f"b{doc['burst_idx'] + 1}/{len(doc['bursts'])}")
52     p += [f for f, get in _BRIEF_FLAGS if get(doc)]
53     body = " ".join(p)
54     return f"{body} #{check_code(body)}"
55
56
57 # --- analyze / serve -----
58
59 def _analyze(args: argparse.Namespace) -> int:
60     r = analyze_file(args.file, args.fs, args.fmt, args.dtype,

```

```

61         "be" if args.be else None, args.bitrev or None, args.diff,
62         None if args.burst is None else args.burst - 1, rf=args.rf)
63     j = r.to_json(views=False) # 한 줄 요약도 여기서 만든다 (반올림이 한 번만 걸리게)
64     d = j["detected"]
65     truth = (sidecar_read(args.file) or {}).get("truth") # display only, never detection
66     if len(r.bursts) > 1:
67         print(f"시간상 버스트 {len(r.bursts)}개 - 그중 {r.burst_idx + 1}번을 분석"
68               " (--burst N 으로 선택)")
69     c = d.get("chirp")
70     if c: # 처프/CSS: 성상도 제원이 없다 - 특성으로 보고
71         print("변조 " + (f"LoRa 추정 SF{c['sf']}" if c["sf"] else "처프(FMCW/CSS)"))
72         print(f"중심주파수 {d['fc']:.1f} Hz")
73         if d["rf_hz"] is not None:
74             print(f"실제 RF {d['rf_hz'] / 1e6:.6f} MHz")
75         print(f"점유대역폭 {c['bw']:.0f} Hz 방향 {'상승' if c['up'] else '하강'}"
76               f" 처프율 {c['mu']:.3g} Hz/s")
77         if c["sf"]:
78             print(f"심볼레이트 {c['rs']:.1f} Hz 심볼시간 {c['tsym'] * 1e3:.2f} ms")
79     else:
80         print(f"변조 {d['mod']}" + (f" (정답 {truth['mod']})" if truth else ""))
81         print(f"중심주파수 {d['fc']:.1f} Hz" + (f" (정답 {truth['fc']:.1f})" if truth else ""))
82         if d["rf_hz"] is not None:
83             print(f"실제 RF {d['rf_hz'] / 1e6:.6f} MHz")
84         print(f"baud {d['baud']:.1f} Hz"
85               + (f" (정답 {truth['baud']:.1f})" if truth else ""))
86         fsk = r.family == "fsk"
87         tail = f"변조지수 h {d['h']:.2f}" if fsk else f"롤오프 {d['rolloff']:.2f}"
88         mer = "" if fsk else f" MER {r.mer_db:.1f} dB"
89         print(f"{tail} lock {r.lock:.1f}{mer}")
90     if r.eq_applied:
91         print("다중경로 보정(등화기) 적용"
92               + (" (T/2 분수간격)" if r.eq_mode == "fse" else " (심볼간격)"))
93     if r.alias_resolved:
94         print("중심주파수 접힘(앨리어스) 보정: 후보 중 스펙트럼 무게중심에 맞는 것을 선택")
95     if r.baud_fallback:
96         print("baud 폴백: 스펙트럼에서 baud 선을 못 찾아 점유대역폭으로 추정 - baud 신뢰 낮음")
97     if r.carrier_ambiguous:
98         print("경고: 중심주파수가 접혀 있을 수 있음 - fs 에 비해 너무 높다"
99               " (fc 를 그대로 믿지 말 것)")
100    if args.save_iq:
101        r.save_iq(args.save_iq)
102    if args.save_symbols:
103        r.save_symbols(args.save_symbols)
104    if args.save_bits:
105        r.save_bits(args.save_bits, args.packed)
106    if args.report:
107        r.save_report(args.report)
108    if args.brief: # 사람용 출력을 대체하지 않고 맨 끝에 한 줄 더 --
109        print(brief(j, "an")) # 조기 return 이면 위 저장들이 조용히 무시된다
110    return 0
111
112
113 def _read_args(p: argparse.ArgumentParser) -> None:
114     """Read-side sample-format overrides for analyze (sidecar/filename win)."""
115     p.add_argument("--fs", type=float)
116     p.add_argument("--fmt", choices=("iq", "real"))
117     p.add_argument("--dtype", choices=DTTYPE_CHOICES)
118     p.add_argument("--be", action="store_true", help="big-endian 샘플")
119     p.add_argument("--bitrev", action="store_true", help="바이트 내 비트 역순")
120     p.add_argument("--rf", type=float, help="RF 중심주파수 (Hz) - 실제 주파수로 보고")

```

```

121     p.add_argument("--brief", action="store_true",
122                   help="손으로 옮길 한 줄 + 오타 검출 코드를 끝에 덧붙임")
123
124
125 def main(argv: list[str] | None = None) -> int:
126     ap = argparse.ArgumentParser(prog="signus")
127     sub = ap.add_subparsers(dest="cmd", required=True)
128
129     a = sub.add_parser("analyze", help="블라인드 복조 + 제원 탐지")
130     a.add_argument("file")
131     _read_args(a)
132     a.add_argument("--save-iq")
133     a.add_argument("--save-symbols")
134     a.add_argument("--save-bits")
135     a.add_argument("--packed", action="store_true")
136     a.add_argument("--report")
137     a.add_argument("--diff", action="store_true",
138                   help="차동 디맵(D-BPSK/D-QPSK): 회전 모호성 없이 비트 복원")
139     a.add_argument("--burst", type=int, help="분석할 버스트 번호 (1부터; 기본 최강 버스트)")
140
141     s = sub.add_parser("serve", help="웹 UI 서버")
142     s.add_argument("--host", default="127.0.0.1")
143     s.add_argument("--port", type=int, default=8000)
144
145     # 합성 생성·채점(gen/dataset/sweep)은 개발기 전용이다. 격리망 장비에는 lab.py 를
146     # 옮기지 않으므로 그 장비에서는 이 명령들이 아예 존재하지 않는다. try/except 로 삼키면
147     # 개발기에서 lab 내부의 진짜 임포트 오류까지 "명령 없음"으로 둔갑하니 파일 유무만 본다.
148     if importlib.util.find_spec("signus.lab"):
149         from . import lab
150         lab.add_commands(sub)
151
152     args = ap.parse_args(argv)
153     if args.cmd == "analyze":
154         return _analyze(args)
155     if args.cmd == "serve":
156         from .server import run
157         run(args.host, args.port)
158         return 0
159     return args.run(args)          # lab 이 단 명령 (gen / dataset / sweep)
160
161
162 if __name__ == "__main__":
163     sys.exit(main())

```

```

1  """Stdlib-only web server: static frontend + POST /api/analyze (raw body,
2  filename + optional fs/fmt/dtype overrides in the query string – no multipart)."""
3
4  import json
5  from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
6  from pathlib import Path
7  from urllib.parse import parse_qs, urlparse
8
9  from .pipeline import analyze_web
10 from .sigio import Meta, decode, parse_name
11
12 _WEB = Path(__file__).resolve().parent / "web" # inside the package so wheels/sdists ship it
13 _MIME = {"html": "text/html", "css": "text/css", "js": "text/javascript",
14         "json": "application/json", "png": "image/png", "svg": "image/svg+xml"}
15 _MAX_BODY = 256 * 1024 * 1024
16
17
18 class Handler(BaseHTTPRequestHandler):
19     def log_message(self, *_: object) -> None:
20         pass
21
22     def _json(self, code: int, obj: dict) -> None:
23         body = json.dumps(obj).encode()
24         self.send_response(code)
25         self.send_header("Content-Type", "application/json")
26         self.send_header("Content-Length", str(len(body)))
27         self.end_headers()
28         self.wfile.write(body)
29
30     def _meta(self, q: dict) -> Meta:
31         """Build Meta from query params with filename fallback; raise on unknown."""
32         name = q.get("name", [""])[0]
33         m = parse_name(name)
34         meta = Meta(float(q["fs"][0]) if "fs" in q else m.fs,
35                    q.get("fmt", [m.fmt])[0], q.get("dtype", [m.dtype])[0],
36                    q.get("endian", [m.endian])[0],
37                    q.get("bitrev", ["1" if m.bitrev else "0"])[0] == "1",
38                    rf_center=float(q["rf"][0]) if "rf" in q else m.rf_center)
39         if not meta.ok():
40             raise ValueError(f"샘플레이트/포맷을 알 수 없습니다: {name}")
41         return meta
42
43     def do_POST(self) -> None: # noqa: N802 (BaseHTTPRequestHandler API)
44         url = urlparse(self.path)
45         if url.path != "/api/analyze":
46             self._json(404, {"error": "not found"})
47             return
48         q = parse_qs(url.query)
49         n = int(self.headers.get("Content-Length") or 0)
50         if n <= 0:
51             self._json(411, {"error": "Content-Length 필요"})
52             return
53         if n > _MAX_BODY:
54             self._json(413, {"error": "파일이 너무 큼니다 (최대 256MB)"})
55             return
56         try:
57             meta = self._meta(q)
58             burst = int(q["burst"][0]) if "burst" in q else None
59             x = decode(self.rfile.read(n), meta)
60             if x.size < 256:

```

```

61         raise ValueError("신호가 너무 짧습니다")
62     except ValueError as exc:
63         self._json(400, {"error": str(exc)})
64     return
65     try:
66         out = analyze_web(x, meta, burst=burst)
67         self._json(200, out)
68     except Exception as exc: # surface DSP failures to the UI
69         self._json(500, {"error": f"분석 실패: {exc}"})
70
71     def do_GET(self) -> None: # noqa: N802
72         rel = urlparse(self.path).path.lstrip("/") or "index.html"
73         target = (_WEB / rel).resolve()
74         if not (target.is_file() and target.is_relative_to(_WEB.resolve())):
75             self._json(404, {"error": "not found"})
76         return
77         body = target.read_bytes()
78         self.send_response(200)
79         self.send_header("Content-Type", _MIME.get(target.suffix, "application/octet-stream"))
80         self.send_header("Content-Length", str(len(body)))
81         self.end_headers()
82         self.wfile.write(body)
83
84
85     def run(host: str = "127.0.0.1", port: int = 8000) -> None:
86         srv = ThreadingHTTPServer((host, port), Handler)
87         print(f"signus UI -> http://{host}:{srv.server_address[1]}")
88         srv.serve_forever()

```

```

1  <!doctype html>
2  <html lang="ko">
3  <head>
4  <meta charset="utf-8">
5  <meta name="viewport" content="width=device-width, initial-scale=1">
6  <title>Signus - 신호 복조 분석기</title>
7  <link rel="icon" href="data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 1
8    6 16'%3E%3Crect width='16' height='16' rx='5' fill='%233182F6'/%3E%3C/svg%3E">
9  <link rel="stylesheet" href="/style.css">
10 <script defer src="/app.js"></script>
11 </head>
12 <body>
13 <header class="topbar">
14   <div class="brand">
15     <span class="logo" aria-hidden="true"></span>
16     <div>
17       <h1>Signus</h1>
18       <p class="tagline">블라인드 PSK/QAM/FSK 신호 복조 분석기</p>
19     </div>
20   </div>
21 </header>
22 <main class="wrap">
23   <section id="dropCard" class="card drop" tabindex="0" role="button"
24     ..... aria-label="신호 파일 선택 또는 드래그">
25     <input id="fileInput" type="file" multiple hidden>
26     <div class="drop-icon" aria-hidden="true"></div>
27     <p class="drop-title">신호 파일을 여기에 놓으세요</p>
28     <p class="drop-sub">여러 개를 놓으면 일괄 분석 · <code>.json</code> 정답 · <code>.sigmf-meta</co
29     de> 지원</p>
30   </section>
31   <section id="metaForm" class="card form hidden">
32     <p class="form-title">파일 이름에서 정보를 찾지 못했어요. 직접 입력해주세요.</p>
33     <div class="form-row">
34       <label>샘플레이트 (Hz)</label>
35       <input id="mFs" type="number" min="1" step="any" placeholder="예: 1000000"></label>
36       <label>포맷</label>
37       <select id="mFmt">
38         ..... <option value="">선택</option><option value="iq">IQ</option><option value="real">Real</opt
39         ..... ion>
40       </select></label>
41       <label>데이터 타입</label>
42       <select id="mDtype">
43         ..... <option value="i16">i16 (16t)</option><option value="i8">i8 (8t)</option>
44         ..... <option value="u8">u8 (8o)</option><option value="u16">u16 (16o)</option>
45         ..... <option value="f32">f32</option><option value="f64">f64</option>
46       </select></label>
47     </div>
48     <button id="metaGo" class="btn">분석 시작</button>
49   </section>
50   <section id="loading" class="card loading hidden" aria-live="polite">
51     <div class="spinner" aria-hidden="true"></div>
52     <p id="loadMsg">신호를 분석하고 있어요...</p>
53   </section>
54   <section id="errorCard" class="card error hidden" role="alert">
55     <div class="err-mark" aria-hidden="true">!</div>
56     <div>
57

```

```

58     <p class="err-title">분석에 실패했어요</p>
59     <p id="errMsg" class="err-msg"></p>
60 </div>
61 </section>
62
63 <!-- Batch results table -->
64 <section id="batchCard" class="card hidden">
65     <div class="card-head">
66         <h2 class="card-title">일괄 분석 결과</h2>
67         <button id="dlCsv" class="btn ghost small">CSV 내보내기</button>
68     </div>
69     <div class="table-wrap">
70         <table class="batch">
71             <thead><tr><th>파일</th><th>변조</th><th>중심주파수</th><th>심볼레이트</th><th>Lock</th></tr>
72             <tbody id="batchBody"></tbody>
73         </table>
74     </div>
75     <p class="hint">행을 클릭하면 상세 화면으로 이동합니다.</p>
76 </section>
77
78 <section id="results" class="results hidden">
79     <button id="backBatch" class="btn ghost small hidden">← 목록으로</button>
80
81     <div class="card summary">
82         <div class="summary-head">
83             <span id="statusPill" class="pill"></span>
84             <p id="fileName" class="file-name"></p>
85         </div>
86         <div class="summary-body">
87             <div class="gauge">
88                 <svg viewBox="0 0 120 120" class="donut" aria-hidden="true">
89                     <circle class="donut-track" cx="60" cy="60" r="52"></circle>
90                     <circle id="donutArc" class="donut-arc" cx="60" cy="60" r="52"></circle>
91                 </svg>
92                 <div class="gauge-center"><span id="lockNum" class="gauge-num">0</span>
93                 <span class="gauge-cap tip" title="별자리가 얼마나 조밀·정렬됐는지 (0~100). 60 이상이면 복조 신뢰">LOCK</span></div>
94             </div>
95             <div class="metrics">
96                 <div class="metric"><span class="metric-label tip" title="변조 오차비 — 높을수록 깨끗 (dB)">MER</span>
97                 <span id="merVal" class="metric-val"></span></div>
98                 <div class="metric"><span class="metric-label tip" title="오차 벡터 크기 — 낮을수록 좋음 (%)">EVM</span>
99                 <span id="evmVal" class="metric-val"></span></div>
100                <div class="metric"><span class="metric-label tip" title="추정 심볼 신호 대 잡음비 (dB)">S
101                NR</span>
102                <span id="snrVal" class="metric-val"></span></div>
103            </div>
104            <div id="flagRow" class="chips"></div>
105            <div id="burstRow" class="chips"></div>
106        </div>
107
108        <div id="burstMap" class="card hidden">
109            <h2 class="card-title">버스트 지도</h2>
110            <div class="wf-wrap">
111                <canvas id="burstFall" class="fall"></canvas>
112                <div id="burstBoxes" class="boxes"></div>

```

```

113     </div>
114     <p class="hint">전체 녹음의 스펙트로그램 (가로=시간, 위=+주파수) · 위 번호와 아래 띠가 검출
115     ↳       된 버스트. 구간을 클릭하면 그 구간을 분석합니다.</p>
116     </div>
117     <div class="card">
118     <h2 class="card-title">파워 스펙트럼</h2>
119     <canvas id="specCanvas" class="spec"></canvas>
120     </div>
121
122     <div class="card const-card">
123     <div class="card-head">
124     <h2 class="card-title">성상도</h2>
125     <div class="play-row">
126     <button id="playBtn" class="btn ghost small" aria-label="재생/일시정지">⏮ 일시정지</button>
127     >
128     <select id="speedSel" class="sel small" aria-label="재생 속도">
129     <option value="1">1x</option><option value="4" selected>4x</option>
130     <option value="16">16x</option>
131     </select>
132     </div>
133     </div>
134     <div class="const">
135     <canvas id="constCanvas" class="const"></canvas>
136     <input id="scrub" class="scrub" type="range" min="0" max="1000" value="0"
137     ↳       aria-label="재생 위치">
138     <p id="playInfo" class="hint"></p>
139     </div>
140
141     <div id="paramGrid" class="param-grid"></div>
142
143     <div class="card downloads">
144     <h2 class="card-title">다운로드</h2>
145     <div class="dl-row">
146     <button id="copyBits" class="btn ghost">비트 복사</button>
147     <button id="dlBits" class="btn ghost">비트 .txt</button>
148     <button id="dlSyms" class="btn ghost">심볼 .csv</button>
149     <button id="dlReport" class="btn ghost">리포트 .json</button>
150     </div>
151     </div>
152     </section>
153 </main>
</body>
</html>

```

```

1  :root {
2    --bg: #f4f6f8; --card: #fff; --ink: #191f28; --sub: #6b7684;
3    --blue: #3182F6; --blue-weak: #e8f1fe; --line: #eef1f4;
4    --green: #12b886; --amber: #f59f00; --red: #fa5252;
5    --field: #fff; --hover: #f7fafd; --border: #dfe4ea; /* theme-sensitive surfaces */
6    --ok-bg: #e6f7f1; --warn-bg: #fff4e0; --bad-bg: #ffecec;
7    --radius: 20px; --shadow: 0 6px 24px rgba(30, 42, 66, .07);
8  }
9  /* dark mode: follows the OS. The signal canvases are already dark, so they blend in. */
10 @media (prefers-color-scheme: dark) {
11   :root {
12     --bg: #0e131b; --card: #161d28; --ink: #e6ebf2; --sub: #94a1b2;
13     --blue-weak: #17273c; --line: #232c39; --field: #1b2330; --hover: #1d2632;
14     --border: #2b3543; --ok-bg: #102c24; --warn-bg: #302512; --bad-bg: #321b1e;
15     --shadow: 0 6px 24px rgba(0, 0, 0, .38);
16   }
17 }
18
19 * { box-sizing: border-box; }
20
21 body {
22   margin: 0; background: var(--bg); color: var(--ink);
23   font-family: -apple-system, 'Apple SD Gothic Neo', 'Segoe UI', 'Malgun Gothic', sans-serif;
24   line-height: 1.5; -webkit-font-smoothing: antialiased;
25 }
26
27 .wrap { max-width: 580px; margin: 0 auto; padding: 8px 20px 64px; }
28
29 .topbar { max-width: 580px; margin: 0 auto; padding: 28px 20px 12px; }
30 .brand { display: flex; align-items: center; gap: 12px; }
31 .logo {
32   width: 40px; height: 40px; border-radius: 12px; flex: none;
33   background: linear-gradient(135deg, var(--blue), #6aa8ff);
34 }
35 h1 { margin: 0; font-size: 22px; font-weight: 800; letter-spacing: -.4px; }
36 .tagline { margin: 2px 0 0; color: var(--sub); font-size: 13px; }
37
38 .card {
39   background: var(--card); border-radius: var(--radius); padding: 20px;
40   box-shadow: var(--shadow); margin-top: 16px; animation: rise .35s ease both;
41 }
42 .card-title { margin: 0 0 12px; font-size: 15px; font-weight: 700; }
43 .hidden { display: none !important; }
44
45 @keyframes rise { from { opacity: 0; transform: translateY(10px); } to { opacity: 1; transform: none
46   ; } }
47
48 /* Dropzone */
49 .drop {
50   text-align: center; cursor: pointer; border: 2px dashed var(--border);
51   transition: border-color .18s, background .18s, transform .18s;
52 }
53 .drop:hover, .drop:focus-visible { border-color: var(--blue); background: var(--hover); outline: none; }
54 .drop.drag { border-color: var(--blue); background: var(--blue-weak); transform: scale(1.01); }
55 .drop-icon {
56   width: 52px; height: 52px; margin: 6px auto 14px; border-radius: 16px;
57   background: var(--blue-weak); position: relative;
58 }
59 .drop-icon::after {

```

```

59   content: ''; position: absolute; inset: 16px 15px; border: 3px solid var(--blue);
60   border-radius: 3px; border-top: none; border-right: none;
61   transform: rotate(45deg) translate(2px, -2px);
62 }
63 .drop-title { margin: 0; font-size: 16px; font-weight: 700; }
64 .drop-sub { margin: 6px 0 0; color: var(--sub); font-size: 13px; }
65 code { background: var(--line); padding: 1px 6px; border-radius: 6px; font-size: 12px; }
66
67 /* Meta form */
68 .form-title { margin: 0 0 12px; font-size: 14px; font-weight: 600; }
69 .form-row { display: grid; grid-template-columns: 1fr 1fr 1fr; gap: 12px; }
70 .form-row label { display: flex; flex-direction: column; gap: 6px; font-size: 12px; color: var(--sub)
  4   ); }
71 input, select {
72   font: inherit; padding: 10px 12px; border: 1px solid var(--border); border-radius: 12px;
73   background: var(--field); color: var(--ink);
74 }
75 input:focus, select:focus { outline: 2px solid var(--blue); border-color: transparent; }
76
77 .btn {
78   margin-top: 16px; width: 100%; padding: 13px; border: none; border-radius: 14px;
79   background: var(--blue); color: #fff; font-size: 15px; font-weight: 700; cursor: pointer;
80   transition: filter .15s, transform .1s;
81 }
82 .btn:hover { filter: brightness(1.05); }
83 .btn:active { transform: scale(.98); }
84 .btn.ghost { background: var(--blue-weak); color: var(--blue); margin-top: 0; }
85
86 /* Loading */
87 .loading { display: flex; align-items: center; gap: 14px; color: var(--sub); }
88 .spinner {
89   width: 26px; height: 26px; border: 3px solid var(--line);
90   border-top-color: var(--blue); border-radius: 50%; animation: spin .8s linear infinite;
91 }
92 @keyframes spin { to { transform: rotate(360deg); } }
93
94 /* Error */
95 .error { display: flex; gap: 14px; align-items: center; border-left: 4px solid var(--red); }
96 .err-mark {
97   width: 34px; height: 34px; flex: none; border-radius: 50%; background: var(--bad-bg);
98   color: var(--red); font-weight: 800; display: grid; place-items: center;
99 }
100 .err-title { margin: 0; font-weight: 700; }
101 .err-msg { margin: 2px 0 0; color: var(--sub); font-size: 14px; }
102
103 /* Summary + gauge */
104 .summary-head { display: flex; align-items: center; gap: 12px; margin-bottom: 16px; }
105 .pill { padding: 6px 14px; border-radius: 999px; font-size: 13px; font-weight: 700; }
106 .pill.ok { background: var(--ok-bg); color: var(--green); }
107 .pill.warn { background: var(--warn-bg); color: var(--amber); }
108 .pill.bad { background: var(--bad-bg); color: var(--red); }
109 .file-name { margin: 0; font-size: 13px; color: var(--sub); overflow: hidden; text-overflow: ellipsi
  4   s; white-space: nowrap; }
110
111 .summary-body { display: flex; align-items: center; gap: 24px; }
112 .gauge { position: relative; width: 120px; height: 120px; flex: none; }
113 .donut { width: 120px; height: 120px; }
114 .donut-track { fill: none; stroke: var(--line); stroke-width: 12; }
115 .donut-arc {
116   fill: none; stroke: var(--blue); stroke-width: 12; stroke-linecap: round;

```

```

117   transform: rotate(-90deg); transform-origin: center;
118   transition: stroke-dashoffset .9s cubic-bezier(.22, .61, .36, 1);
119 }
120 .gauge-center { position: absolute; inset: 0; display: grid; place-items: center; }
121 .gauge-num { font-size: 34px; font-weight: 800; line-height: 1; font-variant-numeric: tabular-nums
122   ; }
123 .gauge-cap { font-size: 11px; color: var(--sub); letter-spacing: 1px; }
124
125 .metrics { display: flex; flex-direction: column; gap: 12px; }
126 .metric { display: flex; flex-direction: column; gap: 2px; }
127 .metric-label { font-size: 12px; color: var(--sub); }
128 .tip { cursor: help; text-decoration: underline dotted; text-underline-offset: 3px;
129   text-decoration-color: color-mix(in srgb, var(--sub) 45%, transparent); }
130 .metric-val { font-size: 20px; font-weight: 700; font-variant-numeric: tabular-nums; }
131
132 /* Card head with actions: spacing lives on the CONTAINER so the title and the
133   action stay vertically centered on the same line */
134 .card-head { display: flex; align-items: center; justify-content: space-between;
135   gap: 12px; margin-bottom: 12px; }
136 .card-head .card-title { margin-bottom: 0; }
137 .btn.small { width: auto; margin-top: 0; padding: 8px 14px; font-size: 13px; border-radius: 10px; }
138 .sel.small {
139   font: inherit; font-size: 13px; font-weight: 600; padding: 7px 10px; border-radius: 10px;
140   border: 1px solid var(--border); background: var(--field); color: var(--ink);
141 }
142 .play-row { display: flex; gap: 8px; align-items: center; }
143 .hint { margin: 8px 0 0; font-size: 12px; color: var(--sub); }
144
145 /* Spectrum + spectrogram strip */
146 .spec { width: 100%; height: 120px; display: block; border-radius: 12px 12px 0 0; background: #0d132
147   0; }
148 .fall { width: 100%; height: 150px; display: block; border-radius: 0 0 12px 12px; background: #000
149   ; image-rendering: pixelated; }
150
151 /* Constellation + playback */
152 .const { width: 100%; aspect-ratio: 1; border-radius: 14px; display: block; background: #0d1320; }
153 .scrub { width: 100%; margin-top: 12px; accent-color: var(--blue); }
154
155 /* Batch table */
156 .table-wrap { overflow-x: auto; }
157 table.batch { width: 100%; border-collapse: collapse; font-size: 13.5px; }
158 table.batch th {
159   text-align: left; font-size: 12px; color: var(--sub); font-weight: 600;
160   padding: 8px 10px; border-bottom: 1px solid var(--line);
161 }
162 table.batch td { padding: 11px 10px; border-bottom: 1px solid var(--line);
163   font-variant-numeric: tabular-nums; }
164 table.batch .pill { padding: 3px 10px; font-size: 12px; }
165 table.batch tbody tr { cursor: pointer; transition: background .12s; }
166 table.batch tbody tr:hover { background: var(--hover); }
167 table.batch .fn { font-weight: 600; max-width: 210px; overflow: hidden;
168   text-overflow: ellipsis; white-space: nowrap; }
169
170 /* Param grid */
171 .param-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 16px; margin-top: 16px; }
172 .param { padding: 16px; margin: 0; animation: rise .4s ease both; }
173 .param-label { font-size: 12px; color: var(--sub); }
174 .param-val { font-size: 22px; font-weight: 800; margin-top: 4px; letter-spacing: -.3px;
175   font-variant-numeric: tabular-nums; }
176 .param-note { font-size: 11px; color: var(--sub); margin-top: 2px; }

```

```

174 .chips { display: flex; flex-wrap: wrap; gap: 6px; margin-top: 10px; }
175 .chips:empty { display: none; } /* no flags -> no ghost margin */
176 .chip { font-size: 11px; font-weight: 700; padding: 4px 9px; border-radius: 999px; }
177 .chip.hit { background: var(--ok-bg); color: var(--green); }
178 .chip.miss { background: var(--bad-bg); color: var(--red); }
179 .chip.warn { background: var(--warn-bg); color: var(--amber); }
180
181 /* Burst selector (chips that act) */
182 .chip-btn {
183   font: inherit; font-size: 11px; font-weight: 700; padding: 4px 10px;
184   border-radius: 999px; border: 1px solid var(--border); background: var(--field);
185   color: var(--sub); cursor: pointer; transition: color .12s, border-color .12s;
186 }
187 .chip-btn:hover { border-color: var(--blue); color: var(--blue); }
188 .chip-btn.on { background: var(--blue-weak); color: var(--blue); border-color: transparent; }
189
190 /* Downloads */
191 .dl-row { display: flex; gap: 10px; flex-wrap: wrap; }
192 .dl-row .btn { flex: 1; min-width: 120px; }
193
194 /* Burst map: whole-record spectrogram strip + clickable burst boxes */
195 .wf-wrap { position: relative; }
196 .wf-wrap .fall { height: 230px; border-radius: 12px; }
197 .boxes { position: absolute; inset: 0; pointer-events: none; }
198 .box {
199   position: absolute; box-sizing: border-box; border: 1px solid rgba(255, 255, 255, .55);
200   border-radius: 0; background: rgba(255, 255, 255, .06); cursor: pointer;
201   pointer-events: auto; min-width: 7px; min-height: 12px;
202   transition: background .12s, box-shadow .12s;
203 }
204 .box:hover, .box:focus-visible {
205   background: rgba(255, 255, 255, .18); box-shadow: 0 0 0 1px #fff; outline: none;
206 }
207 .box-num { /* sa 의 번호 레인: 버스트 위쪽에 항상 보이는 번호 */
208   position: absolute; top: 2px; left: 50%; transform: translateX(-50%);
209   color: #ebebeb; font: bold 11px ui-monospace, SFMono-Regular, monospace;
210   text-shadow: 0 0 3px #000, 0 0 3px #000; pointer-events: none;
211 }
212 .box-lane { /* sa 의 검출 띠: 버스트 구간 아래쪽 흰 띠 */
213   position: absolute; left: 0; right: 0; bottom: 2px; height: 4px;
214   background: #ebebeb; pointer-events: none;
215 }
216 .box-lbl { /* hidden by default (a spectrogram full of labels is unreadable); shown on hover */
217   position: absolute; top: -17px; left: -2px; white-space: nowrap; font-size: 10px;
218   font-weight: 700; color: #fff; background: rgba(13, 19, 32, .85); padding: 1px 6px;
219   border-radius: 6px; pointer-events: none; display: none; z-index: 2;
220 }
221 .box:hover .box-lbl, .box:focus-visible .box-lbl, .box.sel .box-lbl { display: block; }
222 .box.sel { border-color: #fff; box-shadow: inset 0 0 0 1px #fff; }
223 .pill.gray { background: var(--line); color: var(--sub); }
224 table.batch .fc { font-weight: 600; font-variant-numeric: tabular-nums; }
225
226 /* Non-digital emitter inline info */
227
228 @media (max-width: 420px) {
229   .form-row { grid-template-columns: 1fr; }
230   .param-grid { grid-template-columns: 1fr; }
231   .summary-body { gap: 18px; }
232 }
233

```

```
234 | @media (prefers-reduced-motion: reduce) {  
235 |   * { animation: none !important; transition: none !important; }  
236 | }
```

```

1  "use strict";
2  const $ = (id) => document.getElementById(id);
3  const API = "/api/analyze";
4  const FS_RE = /(?:^[_.])fs(?:\d+(?:\.\d+)?(?:e[+-]?\d+)?(?:[_.]|$))/i;
5  const RF_RE = /(?:^[_.])rf(?:\d+(?:\.\d+)?(?:e[+-]?\d+)?(?:[_.]|$))/i;
6  const REDUCED = matchMedia("(prefers-reduced-motion: reduce)").matches;
7  const state = { file: null, sidecar: null, meta: null, resp: null, batch: [],
8    .....      burst: null, overview: null };
9
10 /* --- filename parsing (mirror of sigio.parse_name; read-necessities only) --- */
11 const DTYPES = ["i8", "u8", "i16", "u16", "f32", "f64"];
12 // baudline-style aliases: <bits>t = two's complement (signed), <bits>o = offset (unsigned)
13 const DTYPE_ALIAS = { "8t": "i8", "8o": "u8", "16t": "i16", "16o": "u16", "32f": "f32", "64f": "f64",
14   .....      " " };
15 const FMTS = { cplx: "iq", real: "real", iq: "iq" }; // iq = 옛 밀줄 형식의 이름
16 /* <이름>.cplx|real.<샘플레이트>.<16t>.pcm - 샘플레이트는 접두사가 없으므로 포맷 바로
17   다음 자리로 찾는다. 옛 밀줄 형식(<이름>_fs_..._iq_i16.iq)도 그대로 읽는다. sigio.py 와 규칙이
18   같아야 한다: 여기서 칼리면 웹은 되고 CLI 는 안 되는(또는 그 반대) 조용한 불일치가 된다. */
19 function parseName(name) {
20   const base = name.replace(/^.*/, "").toLowerCase();
21   const mt = FS_RE.exec(base), rt = RF_RE.exec(base), toks = base.split(/[_.]/);
22   // 진짜 포맷 토막 = "숫자 + 아는 제원 토막"을 데리고 다니는 마지막 후보. 라벨의 real/cplx/
23   // u8/be 오염과 점 켜 샘플레이트(2.4e6 -> 2 Hz)를 같은 관문으로 막는다 (sigio.py 와 동일 규칙).
24   const spec = (t) => DTYPES.includes(t) || t in DTYPE_ALIAS || t === "be" || t === "bitrev"
25     .....      || t === "pcm" || t.startsWith("rf");
26   const hit = (k) => /^d+$/ .test(toks[k + 1] || "") && spec(toks[k + 2] || "");
27   const cand = toks.map((t, k) => (t in FMTS ? k : -1)).filter((k) => k >= 0);
28   const hits = cand.filter(hit);
29   const i = hits.length ? hits[hits.length - 1] : (cand.length ? cand[0] : -1);
30   const tail = i >= 0 ? toks.slice(i + 1) : toks; // 제원은 포맷 토막 뒤에만 산다
31   const dt = tail.find((t) => DTYPES.includes(t) || t in DTYPE_ALIAS);
32   return {
33     fs: hits.includes(i) ? parseFloat(toks[i + 1]) : (mt ? parseFloat(mt[1]) : null),
34     fmt: i >= 0 ? FMTS[toks[i]] : null,
35     dtype: dt ? (DTYPE_ALIAS[dt] || dt) : "i16",
36     endian: tail.includes("be") ? "be" : "le",
37     bitrev: tail.includes("bitrev"),
38     rf: rt ? parseFloat(rt[1]) : null,
39   };
40 }
41 /* SigMF: core:datatype -> our fmt/dtype/endian */
42 const SIGMF = { cf32: ["iq", "f32"], ci16: ["iq", "i16"], ci8: ["iq", "i8"],
43   .....      cu8: ["iq", "u8"], rf32: ["real", "f32"], ri16: ["real", "i16"] };
44 function metaFromSigmf(doc) {
45   const g = doc && doc.global;
46   if (!g || !g["core:datatype"]) return null;
47   const dt = g["core:datatype"], base = dt.replace(/_[lb]e$/, "");
48   if (!(base in SIGMF)) return null;
49   const [fmt, dtype] = SIGMF[base];
50   const cap = (doc.captures || []).length ? doc.captures[0] : {};
51   return { fs: Number(g["core:sample_rate"]), fmt, dtype,
52     .....      endian: dt.endsWith("_be") ? "be" : "le", bitrev: false,
53     .....      rf: cap["core:frequency"] != null ? Number(cap["core:frequency"]) : null };
54 }
55 /* --- number formatting --- */
56 function sig(x) {
57   const a = Math.abs(x);
58   return parseFloat(a >= 100 ? x.toFixed(0) : a >= 10 ? x.toFixed(1) : x.toFixed(2)).toString();
59 }

```

```

60 function fmtHz(v) {
61   if (v == null) return "-";
62   const a = Math.abs(v);
63   if (a >= 1e6) return sig(v / 1e6) + " MHz";
64   if (a >= 1e3) return sig(v / 1e3) + " kHz";
65   return sig(v) + " Hz";
66 }
67 function fmtRf(v) { // absolute RF: keep kHz resolution even in the 100s-of-MHz range
68   return (v / 1e6).toFixed(3) + " MHz";
69 }
70
71 /* --- ideal constellation points (mirror of constellations.ideal_points) --- */
72 function idealPoints(mod) {
73   if (mod === "bpsk") return [{ i: 1, q: 0 }, { i: -1, q: 0 }];
74   if (mod === "qpsk" || mod === "8psk") {
75     const m = mod === "qpsk" ? 4 : 8, off = mod === "qpsk" ? Math.PI / 4 : 0, p = [];
76     for (let k = 0; k < m; k++) p.push({ i: Math.cos(off + 2 * Math.PI * k / m), q: Math.sin(of
77       f + 2 * Math.PI * k / m) });
78     return p;
79   }
80   const n = mod === "16qam" ? 4 : mod === "32qam" ? 6 : 8, lv = [], p = [];
81   for (let k = 0; k < n; k++) lv.push(k * 2 - (n - 1));
82   for (const qi of lv) for (const ii of lv) {
83     if (mod === "32qam" && Math.abs(ii * qi) === 25) continue; // cross: corners removed
84     p.push({ i: ii, q: qi });
85   }
86   let e = 0;
87   for (const pt of p) e += pt.i * pt.i + pt.q * pt.q;
88   const norm = Math.sqrt(e / p.length);
89   return p.map((pt) => ({ i: pt.i / norm, q: pt.q / norm }));
90 }
91
92 /* --- file intake: one data file (+sidecar) or many (batch) --- */
93 async function acceptFiles(list) {
94   const files = [...list];
95   const metas = files.filter((f) => /\. (json|sigmf-meta)$/i.test(f.name));
96   const data = files.filter((f) => !metas.includes(f));
97   if (!data.length) return showError("데이터 파일을 찾을 수 없어요.");
98   if (data.length > 1) return runBatch(data, metas);
99
100   state.file = data[0];
101   state.resp = null;
102   state.batch = [];
103   state.burst = null;
104   state.overview = null;
105   // 사이드카는 이름이 같은 캡처의 것만 쓴다 -- 확장자만 보고 집으면 다른 신호의 fs/fmt 로
106   // 조용히 읽는다 (일괄 모드는 이미 stem 대조를 한다: 두 모드가 다르면 그게 또 함정이다)
107   const mine = (m, ext) => {
108     const stem = m.name.toLowerCase().slice(0, -ext.length);
109     const dn = state.file.name.toLowerCase();
110     return dn.startsWith(stem) && (dn.length === stem.length || dn[stem.length] === ".");
111   };
112   const sm = metas.find((m) => m.name.toLowerCase().endsWith(".sigmf-meta") && mine(m, ".sigmf-meta"
113     ));
114   const truth = metas.find((m) => m.name.toLowerCase().endsWith(".json") && mine(m, ".json"));
115   state.sidecar = truth ? await readJson(truth) : null; // client-side only, never sent
116   state.meta = (sm && metaFromSigmf(await readJson(sm))) || parseName(state.file.name);
117   if (state.meta && state.meta.fs && state.meta.fmt) runFirst();
118   else showMetaForm();
119 }

```

```

118 const readJson = (f) => f.text().then(JSON.parse).catch(() => null);
119
120 function showMetaForm() {
121   hideAll();
122   state.meta = state.meta || {};
123   if (state.meta.fs) $("mFs").value = state.meta.fs;
124   $("mFmt").value = state.meta.fmt || "";
125   $("mDtype").value = state.meta.dtype || "i16";
126   show($("metaForm"));
127 }
128 $("metaGo").onclick = () => {
129   const fs = parseFloat($("mFs").value), fmt = $("mFmt").value;
130   if (!(fs > 0) || !fmt) return showError("샘플레이트와 포맷을 입력해주세요.");
131   state.meta = { fs, fmt, dtype: $("mDtype").value, endian: "le", bitrev: false };
132   runFirst();
133 };
134
135 /* --- server call (contract-frozen) --- */
136 function query(file, m, burst) {
137   const q = new URLSearchParams({ name: file.name });
138   if (m.fs) q.set("fs", m.fs);
139   if (m.fmt) q.set("fmt", m.fmt);
140   if (m.dtype) q.set("dtype", m.dtype);
141   if (m.endian) q.set("endian", m.endian);
142   if (m.bitrev) q.set("bitrev", "1");
143   if (m.rf != null) q.set("rf", m.rf);
144   if (burst != null) q.set("burst", burst);
145   return q;
146 }
147 async function postTo(api, file, m, burst) {
148   const res = await fetch(api + "?" + query(file, m, burst), { method: "POST", body: file });
149   const data = await res.json();
150   if (!res.ok || data.error) throw new Error(data.error || `서버 오류 (${res.status})`);
151   return data;
152 }
153 async function runFirst() { // entry: first analyze of a fresh capture
154   hideAll();
155   $("loadMsg").textContent = "신호를 분석하고 있어요...";
156   show($("loading"));
157   try {
158     const resp = await postTo(API, state.file, state.meta);
159     state.overview = resp.overview || null;
160     state.resp = resp;
161     render(resp);
162     if (state.overview) renderBurstMap(state.overview, resp);
163   } catch (e) {
164     showError(e.message);
165   }
166 }
167 async function analyze() { // burst re-selection within a single signal
168   hideAll();
169   $("loadMsg").textContent = "신호를 분석하고 있어요...";
170   show($("loading"));
171   try {
172     state.resp = await postTo(API, state.file, state.meta, state.burst);
173     render(state.resp);
174     if (state.overview) renderBurstMap(state.overview, state.resp);
175     $("backBatch").classList.toggle("hidden", !state.batch.length);
176   } catch (e) {
177     showError(e.message);

```

```

178 }
179 }
180
181 /* --- batch mode --- */
182 async function runBatch(data, metas) {
183   hideAll();
184   show$("loading");
185   const truths = new Map(metas.filter((m) => /\.json$/i.test(m.name))
186     .map((m) => [m.name.replace(/\.json$/i, ""), m]));
187   const sigmfs = new Map(metas.filter((m) => /\sigmf-meta$/i.test(m.name))
188     .map((m) => [m.name.replace(/\sigmf-meta$/i, ""), m]));
189   const rows = [];
190   for (let i = 0; i < data.length; i++) {
191     const f = data[i];
192     $("loadMsg").textContent = `일괄 분석 중... (${i + 1}/${data.length}) ${f.name}`;
193     // same meta precedence as single-file mode: a .sigmf-meta sidecar (matched by stem)
194     // beats filename tokens
195     const sm = sigmfs.get(f.name.replace(/\.([^\.]$)/, ""));
196     const meta = (sm && metaFromSigmf(await readJson(sm))) || parseName(f.name);
197     let resp = null, err = null;
198     try {
199       if (!meta.fs || !meta.fmt) throw new Error("파일명에 fs/포맷 정보가 없어요");
200       resp = await postTo(API, f, meta);
201     } catch (e) { err = e.message; }
202     const tf = truths.get(f.name);
203     // meta is stored per row: a later burst-chip re-analyze posts with THIS file's meta --
204     // it used to read state.meta, which a fresh batch never set (crash) and a previous
205     // single-file run left stale (silent wrong-fs decode)
206     rows.push({ file: f, meta, resp, err, sidecar: tf ? await readJson(tf) : null });
207   }
208   state.batch = rows;
209   hideAll();
210   renderBatch(rows);
211 }
212
213 function renderBatch(rows) {
214   $("batchBody").innerHTML = rows.map((r, i) => {
215     if (r.err) return `<tr data-i="${i}"><td class="fn">${r.file.name}</td>` +
216       `<td colspan="4" style="color:var(--red)">${r.err}</td></tr>`;
217     const d = r.resp.detected, lock = Math.round(r.resp.quality.lock);
218     const cls = lock >= 60 ? "ok" : lock >= 40 ? "warn" : "bad";
219     return `<tr data-i="${i}"><td class="fn">${r.file.name}</td>` +
220       `<td>${d.mod.toUpperCase()}</td><td>${fmtHz(d.fc)}</td><td>${fmtHz(d.baud)}</td>` +
221       `<td><span class="pill ${cls}">${lock}</span></td></tr>`;
222   }).join("");
223   $("batchBody").querySelectorAll("tr").forEach((tr) => {
224     tr.onclick = () => {
225       const r = state.batch[+tr.dataset.i];
226       if (!r.resp) return;
227       state.file = r.file; state.meta = r.meta; state.resp = r.resp; state.sidecar = r.sidecar;
228       state.burst = null; state.overview = null;
229       hideAll();
230       render(r.resp);
231       const b = $("backBatch");
232       b.textContent = "← 목록으로";
233       b.onclick = () => { hideAll(); renderBatch(state.batch); };
234       b.classList.remove("hidden");
235     };
236   });
237   show$("batchCard");

```

```

238 }
239 $("backBatch").onclick = () => { hideAll(); renderBatch(state.batch); };
240 $("dlCsv").onclick = () => {
241     const head = "file,mod,fc_hz,baud_hz,rolloff,lock,mer_db,snr_est_db\n";
242     const body = state.batch.filter((r) => r.resp).map((r) => {
243         const d = r.resp.detected, q = r.resp.quality;
244         return [r.file.name, d.mod, d.fc, d.baud, d.rolloff ?? "", q.lock, q.mer_db,
245             ..... r.resp.snr_est_db ?? ""].join(",");
246     }).join("\n");
247     saveBlob("signus_batch.csv", head + body, "text/csv");
248 };
249
250 /* --- burst map: whole-record spectrogram strip with clickable time-burst boxes --- */
251 function renderBurstMap(ov, result) {
252     show($("#burstMap"));
253     const n = ov.n || 1, bs = result.bursts || [];
254     paintStrip($("#burstFall"), ov.strip);
255     const host = $("#burstBoxes");
256     host.innerHTML = "";
257     bs.forEach((b, i) => {
258         const left = Math.max(0, b.start / n * 100);
259         const dur = (b.end - b.start) / ov.fs;
260         const lbl = `버스트 ${i + 1} · ${dur >= 1 ? dur.toFixed(2) + " s" : (dur * 1e3).toFixed(0) + " m
261             ..... s"}`;
262         const box = document.createElement("button"); // full height (one signal), spans t0..t1 in tim
263             ..... e
264         box.className = "box" + (i === result.burst_idx ? " sel" : "");
265         box.style.top = "0%"; box.style.height = "100%"; box.style.left = left + "%";
266         box.style.width = Math.min(100 - left, Math.max(0.8, (b.end - b.start) / n * 100)) + "%";
267         box.title = lbl;
268         box.innerHTML = `<span class="box-num">${i + 1}</span><span class="box-lane"></span>` +
269             ..... `<span class="box-lbl">${lbl}</span>`;
270         box.onclick = () => { state.burst = i; analyze(); }; // re-analyse that burst; map persists
271         host.appendChild(box);
272     });
273 }
274
275 /* --- rendering --- */
276 function statusOf(lock) {
277     if (lock >= 60) return ["복조 성공", "ok"];
278     if (lock >= 40) return ["부분 복조", "warn"];
279     return ["복조 실패", "bad"];
280 }
281
282 function render(d) {
283     hideAll();
284     show($("#results"));
285     $("#burstMap").classList.add("hidden"); // re-shown by renderBurstMap only in multi-burst singl
286         ..... e mode
287     const lock = d.quality.lock;
288     let [txt, cls] = statusOf(lock);
289     if (d.detected.chirp) { txt = "처프 특성 보고"; cls = "warn"; }
290     const pill = $("#statusPill");
291     pill.textContent = txt;
292     pill.className = "pill " + cls;
293     $("#fileName").textContent =
294         `${state.file.name} · ${fmtHz(d.fs)} · ${d.fmt === "real" ? "Real PCM" : "IQ"}`;
295     $("#merVal").textContent = d.quality.mer_db == null ? "-" : d.quality.mer_db.toFixed(1) + " dB";
296     $("#evmVal").textContent = d.quality.evm == null ? "-" : (d.quality.evm * 100).toFixed(1) + " %";
297     $("#snrVal").textContent = snrText(d);
298 }

```

```

295 const flags = [];
296 if (d.family === "fsk") flags.push('<span class="chip warn">FSK 계열 · 주파수 판별</span>');
297 if (d.family === "chirp") flags.push('<span class="chip warn">처프/CSS · 특성 판독</span>');
298 if (d.eq && d.eq.applied) flags.push('<span class="chip hit">등화기 적용 · ${
299   d.eq.mode === "fse" ? "T/2 분수간격" : "심볼간격"></span>');
300 if (d.detected.alias_resolved) flags.push('<span class="chip warn">반송파 앨리어스 보정</span>');
301 if (d.detected.baud_fallback) flags.push('<span class="chip warn">심볼레이트 대역폭 폴백</span>');
302 if (d.detected.carrier_ambiguous) flags.push('<span class="chip warn">반송파 모호 · 앨리어싱 가능<
  ↳ /span>');
303 $("flagRow").innerHTML = flags.join("");
304
305 renderBursts(d);
306 animateGauge(lock, cls);
307 renderParams(d);
308 drawSpectrum(d);
309 startPlay(d);
310 }
311 function snrText(d) {
312   const s = d.snr_est_db;
313   if (s == null || d.quality.lock < 30) return "-";
314   return (s > 28 ? "≥28" : s.toFixed(1)) + " dB";
315 }
316
317 function renderBursts(d) {
318   const row = $("burstRow"), bs = d.bursts || [];
319   // when the burst MAP is shown (multi-burst capture) the map replaces the chips
320   if (state.overview || bs.length < 2) { row.innerHTML = ""; return; }
321   const dur = (b) => {
322     const sec = (b.end - b.start) / d.fs;
323     return sec >= 1 ? sec.toFixed(2) + " s" : (sec * 1e3).toFixed(1) + " ms";
324   };
325   row.innerHTML = bs.map((b, i) =>
326     `<button class="chip-btn${i === d.burst_idx ? " on" : ""}" data-i="${i}">` +
327     `버스트 ${i + 1} · ${dur(b)}</button>`).join("");
328   row.querySelectorAll("button").forEach((el) => {
329     el.onclick = () => { state.burst = +el.dataset.i; analyze(); };
330   });
331 }
332
333 function chip(hit, truthTxt) {
334   return `<span class="chip ${hit ? "hit" : "miss"}">정답 ${truthTxt} · ${hit ? "일치" : "불일치"}</
  ↳ span>`;
335 }
336 function renderParams(d) {
337   const det = d.detected, t = state.sidecar && state.sidecar.truth;
338   const near = (a, b, tol) => Math.abs(a - b) <= tol;
339   if (det.chirp) { // 처프/CSS: 성상도 제원 대신 특성 (복조 없음, 판독만)
340     const c = det.chirp;
341     const crows = [
342       ["중심주파수", det.rf_hz != null ? fmtRf(det.rf_hz) : fmtHz(det.fc), null,
343         det.rf_hz != null ? `기저대역 ${fmtHz(det.fc)}` : ""],
344       ["처프", `${c.up ? "▲ 상승" : "▼ 하강"} · ${(c.mu / 1e9).toFixed(3)} MHz/ms`, null,
345         `점유대역폭 ${fmtHz(c.bw)}`],
346       ["변조방식", c.sf ? `LoRa 추정 SF${c.sf}` : "처프(FMCW/CSS)", null,
347         c.sf ? `심볼레이트 ${fmtHz(c.rs)} · 심볼시간 ${(c.tsym * 1e3).toFixed(2)} ms` : ""],
348     ];
349     $("paramGrid").innerHTML = crows.map(paramCard).join("");
350     return;
351   }
352   const rows = [

```

```

353     ["중심주파수", det.rf_hz != null ? fmtRf(det.rf_hz) : fmtHz(det.fc),
354     t && t.fc != null && chip(near(det.fc, t.fc, Math.max(300, 0.02 * t.baud)), fmtHz(t.fc)),
355     det.rf_hz != null ? `기저대역 ${fmtHz(det.fc)}` : ""],
356     ["심볼레이트", fmtHz(det.baud), t && t.baud != null && chip(near(det.baud, t.baud, 0.03 * t.baud
    ↳   ), fmtHz(t.baud)),
357     det.baud_conf ? `스펙트럴 라인 강도 ×${Math.round(det.baud_conf)}` : ""],
358     ["변조방식", det.mod.toUpperCase(), t && t.mod != null && chip(det.mod === t.mod, t.mod.toUpperCase()
    ↳   ase()),
359     det.symmetry ? `대칭 ${det.symmetry}` : "주파수 레벨"],
360 ];
361 if (d.family === "fsk") {
362     rows.push(["변조지수 h", det.h.toFixed(2),
363     t && t.h != null && chip(near(det.h, t.h, 0.15), t.h.toFixed(2)), ""]);
364 } else {
365     rows.push(["롤오프", det.rolloff.toFixed(2),
366     t && t.rolloff != null && chip(near(det.rolloff, t.rolloff, 0.11), t.rolloff.toFixed(2)), ""]);
367     ;
368 }
369 $("paramGrid").innerHTML = rows.map(paramCard).join("");
370 }
371 function paramCard([label, val, hit, note]) {
372     return `
373     <div class="card param">
374     <div class="param-label">${label}</div>
375     <div class="param-val">${val}</div>
376     ${note ? `<div class="param-note">${note}</div>` : ""}
377     ${hit ? `<div class="chips">${hit}</div>` : ""}
378     </div>`;
379 }
380
381 function animateGauge(lock, cls) {
382     const arc = $("donutArc"), num = $("lockNum"), C = 2 * Math.PI * 52;
383     arc.style.stroke = { ok: "#12b886", warn: "#f59f00", bad: "#fa5252" }[cls];
384     arc.style.strokeDasharray = C;
385     const target = C * (1 - Math.max(0, Math.min(100, lock)) / 100);
386     if (REDUCED) { arc.style.strokeDashoffset = target; num.textContent = Math.round(lock); return; }
387     arc.style.strokeDashoffset = C;
388     requestAnimationFrame(() => { arc.style.strokeDashoffset = target; });
389     const t0 = performance.now();
390     const step = (t) => {
391         const k = Math.min(1, (t - t0) / 900);
392         num.textContent = Math.round(lock * (1 - Math.pow(1 - k, 3)));
393         if (k < 1) requestAnimationFrame(step);
394     };
395     requestAnimationFrame(step);
396 }
397
398 /* --- canvas helpers --- */
399 function fit(cv, hCss) {
400     const dpr = window.devicePixelRatio || 1, w = cv.clientWidth || 320;
401     const h = hCss || w;
402     cv.width = w * dpr; cv.height = h * dpr;
403     const g = cv.getContext("2d");
404     g.setTransform(dpr, 0, 0, dpr, 0, 0);
405     return [g, w, h];
406 }
407 function pct(a, p) {
408     const s = [...a].sort((x, y) => x - y);
409     return s[Math.min(s.length - 1, Math.max(0, Math.round(p * (s.length - 1))))];

```

```

410 }
411
412 /* --- spectrum --- */
413 function drawSpectrum(d) {
414     // sa 프로브의 power spectrum 판과 같은 조판: 흑배경, 백색 곡선, 회색(0.35) 점선 격자,
415     // 바닥 p5-2 .. 최대+2 dB. 검출된 중심주파수(실선)와 ±baud/2(점선)만 기능선으로 얹는다.
416     const [g, w, h] = fit$("specCanvas", 120);
417     g.fillStyle = "#000"; g.fillRect(0, 0, w, h);
418     if (!d.spectrum) return;
419     const f = d.spectrum.f, db = d.spectrum.db;
420     const lo = pct(db, 0.05) - 2, hi = pct(db, 1) + 2;
421     const fmin = f[0], fmax = f[f.length - 1];
422     const X = (v) => (v - fmin) / (fmax - fmin) * w;
423     const Y = (v) => h - (Math.max(lo, Math.min(hi, v)) - lo) / (hi - lo) * (h - 8) - 4;
424     g.strokeStyle = "rgba(255,255,255,.35)"; g.setLineDash([1, 5]);
425     for (let k = 1; k <= 4; k++) { // fs/2 의 k/5 지점 세로 격자 (sa 와 동일)
426         const gx = fmin + (fmax - fmin) * k / 5;
427         g.beginPath(); g.moveTo(X(gx), 0); g.lineTo(X(gx), h); g.stroke();
428     }
429     const det = d.detected, fca = Math.abs(det.fc) / 1e3, half = det.baud / 2e3;
430     g.setLineDash([4, 4]);
431     for (const gf of [fca - half, fca + half]) { // 단측(0..fs/2) 측 -- fc 는 접힌 절대값
432         g.beginPath(); g.moveTo(X(gf), 0); g.lineTo(X(gf), h); g.stroke();
433     }
434     g.setLineDash([]);
435     g.strokeStyle = "rgba(255,255,255,.6)";
436     g.beginPath(); g.moveTo(X(fca), 0); g.lineTo(X(fca), h); g.stroke();
437     g.strokeStyle = "#4da3ff"; g.lineWidth = 1.4; g.beginPath(); // 파란선 (바늘이 주인공)
438     f.forEach((v, k) => (k ? g.lineTo(X(v), Y(db[k])) : g.moveTo(X(v), Y(db[k]))));
439     g.stroke();
440     g.fillStyle = "rgba(255,255,255,.6)"; g.font = "10px sans-serif";
441     g.fillText(sig(fmin) + " kHz", 4, h - 4);
442     const tmax = sig(fmax) + " kHz";
443     g.fillText(tmax, w - g.measureText(tmax).width - 4, h - 4);
444 }
445
446 /* --- spectrogram strip (sa 프로브의 PNG 와 같은 조판, 버스트 지도 전용) --- */
447 function paintStrip(canvas, st) {
448     // 서버가 sa 와 같은 규칙(흑백 0..235, hamming 256/128, p25+10..35dB, 열 최대 폴링)으로
449     // 만든 격자를 해상도 그대로 찍는다. CSS 의 image-rendering: pixelated 가 sa PNG 를
450     // 확대해 보는 것과 같은 방식으로 늘린다 -- 보간 뭉개짐이 없다.
451     const g = canvas.getContext("2d");
452     canvas.width = st && st.cols ? st.cols : 1;
453     canvas.height = st && st.rows ? st.rows : 1;
454     if (!st || !st.cols) { g.fillStyle = "#000"; g.fillRect(0, 0, 1, 1); return; }
455     const img = g.createImageData(st.cols, st.rows);
456     for (let k = 0; k < st.g.length; k++) {
457         const o = k * 4;
458         img.data[o] = img.data[o + 1] = img.data[o + 2] = st.g[k];
459         img.data[o + 3] = 255;
460     }
461     g.putImageData(img, 0, 0);
462 }
463
464 /* --- constellation playback (the headline) --- */
465 const play = { raf: 0, i: 0, on: false, data: null };
466 let CG = null; // cached constellation geometry
467
468 function stopPlay() { cancelAnimationFrame(play.raf); play.raf = 0; play.on = false; }
469 function setPlayBtn(on) { $("playBtn").textContent = on ? "⏸ 일시정지" : "▶ 재생"; }

```

```

470
471 function startPlay(d) {
472   stopPlay();
473   play.data = d;
474   play.i = 0;
475   const n = d.constellation.i.length;
476   $("scrub").max = String(Math.max(1, n));
477   $("playInfo").textContent = `${n.toLocaleString()} 심볼 · 시간 순서로 재생 (잔광 효과)`;
478   drawConstBase(d);
479   if (REDUCED) { // static full view, no motion
480     drawPoints(d, 0, n);
481     $("scrub").value = String(n);
482     setPlayBtn(false);
483     return;
484   }
485   play.on = true;
486   setPlayBtn(true);
487   play.raf = requestAnimationFrame(loop);
488 }
489 $("playBtn").onclick = () => {
490   if (!play.data) return;
491   if (play.on) { stopPlay(); setPlayBtn(false); }
492   else { play.on = true; setPlayBtn(true); play.raf = requestAnimationFrame(loop); }
493 };
494 $("scrub").oninput = () => {
495   if (!play.data) return;
496   stopPlay(); setPlayBtn(false);
497   play.i = +$("scrub").value;
498   drawConstBase(play.data);
499   drawPoints(play.data, 0, play.i); // redraw history up to the scrub point
500 };
501
502 function loop() {
503   const d = play.data, n = d.constellation.i.length;
504   const step = Math.max(1, Math.round(n / 600) * (+$("speedSel").value));
505   const to = Math.min(n, play.i + step);
506   fade(); // phosphor persistence
507   drawPoints(d, play.i, to);
508   play.i = to >= n ? 0 : to;
509   if (play.i === 0) drawConstBase(d); // loop: clear the trail
510   $("scrub").value = String(play.i);
511   if (play.on) play.raf = requestAnimationFrame(loop);
512 }
513
514 function drawConstBase(d) {
515   const [g, w] = fit($("constCanvas"));
516   const fsk = d.family === "fsk";
517   const I = d.constellation.i, Q = d.constellation.q;
518   const ideal = fsk ? [] : idealPoints(d.detected.mod);
519   let lim = 0;
520   for (let k = 0; k < I.length; k++) lim = Math.max(lim, Math.abs(I[k]), Math.abs(Q[k]));
521   for (const p of ideal) lim = Math.max(lim, Math.abs(p.i), Math.abs(p.q));
522   lim = (lim || 1) * 1.12;
523   const R = w / 2 * 0.9;
524   CG = { g, w, lim, R, map: (re, im) => [w / 2 + re / lim * R, w / 2 - im / lim * R] };
525
526   g.fillStyle = "#0d1320"; g.fillRect(0, 0, w, w);
527   g.strokeStyle = "rgba(255,255,255,.06)"; g.lineWidth = 1;
528   for (let f = -1; f <= 1; f += 0.5) {
529     const [gx] = CG.map(f, 0), [, gy] = CG.map(0, f);

```

```

530     g.beginPath(); g.moveTo(gx, 0); g.lineTo(gx, w); g.stroke();
531     g.beginPath(); g.moveTo(0, gy); g.lineTo(w, gy); g.stroke();
532 }
533 if (fsk) { // frequency levels as dashed bands
534     g.strokeStyle = "rgba(255,255,255,.5)"; g.setLineDash([6, 5]);
535     for (const lv of [...new Set(I.map((v) => Math.round(v))).filter((v) => Math.abs(v) <= 8)]) {
536         const [, yy] = CG.map(0, lv);
537         g.beginPath(); g.moveTo(0, yy); g.lineTo(w, yy); g.stroke();
538     }
539     g.setLineDash([]);
540 } else {
541     g.strokeStyle = "rgba(255,255,255,.75)"; g.lineWidth = 1.4;
542     for (const p of ideal) {
543         const [x, y] = CG.map(p.i, p.q);
544         g.beginPath(); g.arc(x, y, 6, 0, 7); g.stroke();
545     }
546 }
547 }
548 function fade() {
549     if (!CG) return;
550     CG.g.fillStyle = "rgba(13,19,32,.14)"; // translucent wash = phosphor decay
551     CG.g.fillRect(0, 0, CG.w, CG.w);
552 }
553 function drawPoints(d, from, to) {
554     if (!CG || to <= from) return;
555     const g = CG.g, I = d.constellation.i, Q = d.constellation.q;
556     const fsk = d.family === "fsk";
557     const a = Math.max(0.08, Math.min(0.55, 150 / (to - from)));
558     g.globalCompositeOperation = "lighter";
559     g.fillStyle = `rgba(90,170,255,${a})`;
560     for (let k = from; k < to; k++) {
561         // FSK has no I/Q plane: spread levels vertically, jitter horizontally for density
562         const [x, y] = fsk ? CG.map((k % 89) / 89 - 0.5, I[k]) : CG.map(I[k], Q[k]);
563         g.beginPath(); g.arc(x, y, 1.8, 0, 7); g.fill();
564     }
565     g.globalCompositeOperation = "source-over";
566 }
567
568 /* --- downloads --- */
569 function saveBlob(name, text, type) {
570     const url = URL.createObjectURL(new Blob([text], { type }));
571     const a = document.createElement("a");
572     a.href = url; a.download = name; a.click();
573     URL.revokeObjectURL(url);
574 }
575 const stem = () => state.file.name.replace(/\.([^.]*)$/, "");
576 $("copyBits").onclick = async (e) => { // decoded bitstream -> clipboard, with brief feedback
577     const btn = e.currentTarget, was = btn.textContent;
578     try { await navigator.clipboard.writeText(state.resp.bits); btn.textContent = "복사됨 ✓"; }
579     catch { btn.textContent = "복사 실패"; }
580     setTimeout(() => { btn.textContent = was; }, 1400);
581 };
582 $("dlBits").onclick = () => saveBlob(stem() + ".bits.txt", state.resp.bits, "text/plain");
583 $("dlSyms").onclick = () => {
584     const c = state.resp.constellation;
585     saveBlob(stem() + ".symbols.csv", "i,q\n" + c.i.map((v, k) => `${v},${c.q[k]}`).join("\n"), "text/
586         csv");
587 };
588 $("dlReport").onclick = () =>
589     saveBlob(stem() + ".report.json", JSON.stringify(state.resp, null, 2), "application/json");

```

```
589
590 /* --- view helpers --- */
591 function show(el) { el.classList.remove("hidden"); }
592 function hideAll() {
593   stopPlay();
594   ["metaForm", "loading", "errorCard", "results", "batchCard"]
595     .forEach((id) => $(id).classList.add("hidden"));
596   $("backBatch").classList.add("hidden");
597 }
598 function showError(msg) { hideAll(); $("errMsg").textContent = msg; show($("errorCard")); }
599
600 /* --- dropzone wiring --- */
601 const drop = $("dropCard"), input = $("fileInput");
602 // reset value first: re-picking the SAME file fires no change event otherwise
603 const pick = () => { input.value = ""; input.click(); };
604 drop.onclick = pick;
605 drop.onkeydown = (e) => { if (e.key === "Enter" || e.key === " ") { e.preventDefault(); pick(); } };
606 input.onchange = () => { if (input.files.length) acceptFiles(input.files); };
607 drop.addEventListener("dragover", (e) => { e.preventDefault(); drop.classList.add("drag"); });
608 drop.addEventListener("dragleave", () => drop.classList.remove("drag"));
609 drop.addEventListener("drop", (e) => {
610   e.preventDefault(); drop.classList.remove("drag");
611   if (e.dataTransfer.files.length) acceptFiles(e.dataTransfer.files);
612 });
```

```

1  """sigc.py - 실신호 특징 4줄 + find_bursts 단계 관찰 프로브 (장비 필사용, signus/ 옆에 저장).
2      python3 sigc.py kat                # 자기검증 - 표지의 기대 4줄과 비교
3      python3 sigc.py <캡처> [1-6]       # 요약 4줄(받아쳐 회신) / 단계 관찰(그림 또는 ASCII)
4  """
5  import sys
6
7  import numpy as np
8  from scipy.ndimage import uniform_filter1d
9  from scipy.signal import get_window, stft
10
11 from signus import dsp, sigio
12 from signus.cli import check_code
13
14
15 def otsu(v, bins=128):
16     h, ed = np.histogram(v, bins=bins)
17     c = (ed[:-1] + ed[1:]) / 2
18     w = np.cumsum(h).astype(float)
19     m = np.cumsum(h * c)
20     mb = m / np.where(w > 0, w, 1)
21     mf = (m[-1] - m) / np.where(w[-1] - w > 0, w[-1] - w, 1)
22     return float(c[int(np.argmax(w * (w[-1] - w) * (mb - mf) ** 2))])
23
24
25 def runs_of(mask):
26     d = np.diff(mask.astype(np.int8))
27     s = list(np.where(d == 1)[0] + 1)
28     e = list(np.where(d == -1)[0] + 1)
29     if mask[0]:
30         s.insert(0, 0)
31     if mask[-1]:
32         e.append(mask.size)
33     return list(zip(s, e, strict=True))
34
35
36 if len(sys.argv) < 2:
37     raise SystemExit("사용법: python3 sigc.py kat | python3 sigc.py <캡처파일> [단계 1-6]")
38 arg, step = sys.argv[1], int(sys.argv[2]) if len(sys.argv) > 2 else 0
39 if arg == "kat":
40     fs, t = 1e6, np.arange(66000.0)      # 66000: 마지막 버스트가 파일 끝까지 이어진다
41     rng = np.random.default_rng(7)       # 시드 고정 스트림 - numpy 가 재현을 보장한다
42     x0 = 0.01 * (rng.standard_normal(66000) + 1j * rng.standard_normal(66000))
43     x0 += 0.2 * np.exp(2j * np.pi * 0.29 * t)
44     x0 += 0.5 * np.exp(-2j * np.pi * 0.11 * t) * ((t // 6000).astype(int) % 2 == 0)
45     x0 += 0.0028 * np.exp(2j * np.pi * 0.41 * t)    # 문턱 경계에 걸친 성분들: 필사 오타가
46     x0[31000:35000] += 0.0088 * np.exp(2j * np.pi * 0.35 * t[:4000])    # 상수 하나만
47     x0[42400:47600] += 0.0075 * np.exp(2j * np.pi * -0.37 * t[:5200])    # 스쳐도 아래
48     x0[200:260] = 0.95                # 요약 4줄 어딘가가 바뀌게 만든다
49     x0[21000] += 8.0
50     x0[15000] += 2.0
51 else:
52     x0, meta = sigio.read(arg)
53     fs = meta.fs
54 cx = np.iscomplexobj(x0)
55 raw = np.abs(np.asarray([x0.real, x0.imag])).max(0) if cx else np.abs(x0)
56 cp = float((raw > 0.98 * np.percentile(raw, 99.9)).mean())    # 만점(±1)이 아니라 캡처 자신의
57 # 99.9퍼센타일 기준: float 캡처는 만점 정규화가 안 돼 ±1 기준이면 통짜 오경보다(실측 cp972)
58 x = dsp.analytic(x0)
59 n, pw = x.size, np.abs(x) ** 2
60 rms = float(np.sqrt(pw.mean()) + 1e-30)

```

```

61 dc = abs(complex(x.mean())) / rms # dc 는 블록 전에 재고, 그 뒤는 pipeline 과 똑같이
62 x = x - x.mean() # DC 블록한 신호로 켜다 - 안 그러면 dc 큰 캡처에서
63 pw = np.abs(x) ** 2 # 프로브와 analyze 가 정반대 진단을 낸다
64 # 러닝합 잔차 때문에 정확히 0 인 구간(스켈치/ 뮤트)에서 음수가 나와 log10 이 NaN 이 되고,
65 # otsu 의 histogram 이 장비에서 생 트레이스백으로 죽었다 - 0 으로 눌러 막는다.
66 lp = np.log10(np.maximum(uniform_filter1d(pw, max(64, n // 1000)), 0) + 1e-20)
67 hot = lp >= (et := otsu(lp))
68 ev = float(lp[hot].mean() - lp[~hot].mean()) if 0 < hot.sum() < n else 0.0
69 ed = float(hot.mean())
70 sp = 10 * np.log10(pw.max() / (pw.mean() + 1e-30) + 1e-30)
71 nper = int(min(128, max(64, 1 << int(np.log2(max(n // 2, 64)))))
72 hop = nper // 2
73 _, _, z = stft(x, fs=fs, window=get_window("blackmanharris", nper), nperseg=nper,
74             ..., ..., noverlap=nper - hop, return_onesided=False, boundary=None, padded=False)
75 P = np.abs(z) ** 2
76 nb, nc = P.shape
77 fl = np.percentile(P, 10, axis=1)
78 r = P / (fl[:, None] + 1e-30)
79 k = max(3, nb // 16)
80 sc = uniform_filter1d(np.log10(np.sort(r, axis=0)[-k:].mean(axis=0) + 1e-30), 3)
81 thr = otsu(sc, bins=64)
82 quiet = sc[sc < thr]
83 base = float(np.median(quiet)) if quiet.size else 9.99
84 cn = float(np.log10((np.log(nb / k) + 1) / 0.105))
85 dlo, dhi = 0.25, 0.45 # find_bursts 의 두 문턱 여유. 상수는 한 곳에만 쓰고
86 lov, hiv = base + dlo, base + dhi # b 줄에 이 숫자를 그대로 되찍는다 - 오타면 그게 드러난다
87 above, hi_m = sc >= lov, sc >= hiv
88 rr = [(s, e) for s, e in runs_of(above) if hi_m[s:e].any()]
89 spans = [(max(0, s * hop) + (hop if e - s >= 6 else 0), # 6열 이상은 양끝 한 hop
90          min(n, (e - 1) * hop + nper) - (hop if e - s >= 6 else 0)) # 씹 자른다 - dsp 의
91          for s, e in rr] # 스파이크 게이트와 같은
92 segs = [pw[s:e] for s, e in spans] # 구간에서 재려고
93 sk = sum(float(sg.max() / (sg.mean() + 1e-30)) > 60.0 for sg in segs)
94 gp_ = np.array([rr[i + 1][0] - rr[i][1] for i in range(len(rr) - 1)])
95 sb = round(10 * float(np.median([sc[s:e].max() for s, e in rr]) - base)) if rr else 0
96 g = float(np.median(fl if cx else fl[:nb // 2]) + 1e-30) # real 은 해석신호라 음수 반쪽이
97 # 비어 있다: 전 빈 중앙값을 쓰면 바닥이 0 으로 내려가 밴드 절반이 '점유'로 읽힌다(실측)
98 Ps, fls = np.fft.fftshift(P, 0), np.fft.fftshift(fl)
99 du = (Ps > 40 * g).mean(1)
100 occ = du > 0.1
101 kb, kcont = int(occ.sum()), int((fls > 5 * g).sum())
102 grp = runs_of(occ) if occ.any() else []
103 wd = max((e - s for s, e in grp), default=0)
104 p95 = np.percentile(Ps, 95, axis=1) / g
105 pk = int(np.argmax(p95))
106 pgrp = next(((s, e) for s, e in grp if s <= pk < e), (pk, pk + 1))
107 m2 = np.where(occ, p95, 0.0)
108 m2[pgrp[0]:pgrp[1]] = 0.0
109 q2 = int(np.argmax(m2))
110 qo, qd, qs = (q2 - nb // 2, round(100 * float(du[q2])),
111             ..., ..., round(10 * np.log10(m2[q2] + 1e-30))) if m2[q2] > 0 else (999, 0, 0)
112 la = (f"sigc a n{n} f{fs:.0f} ev{round(100 * ev)} ed{round(100 * ed)}"
113      f" sp{round(float(sp))} dc{round(100 * dc)} cp{round(1000 * cp)}"
114      f" cx{int(cx)}")
115 lb = (f"sigc b c{nc} g{nb} b{round(100 * base)} cn{round(100 * cn)}"
116      f" t{round(100 * thr)} m{round(100 * float(sc.max()))}"
117      f" dlo{round(100 * dlo)} dhi{round(100 * dhi)}") # 뽀렘으로 되계산하면 안 된다 --
118 # (base+0.45)-base 가 44.99999999999999 라, int 로 잘못 옮기면 44 로 찍혔다 (2026-08-14 실측)
119 lc = (f"sigc c r{len(rr)} dn{round(float(np.median([e - s for s, e in rr]))} if rr else 0}"
120      f" gp{round(float(np.median(gp_))} if gp_.size else 0} sb{sb}")

```

```

121     f" av{round(100 * float(above.mean()))} ah{round(100 * float(hi_m.mean()))}"
122     f" sk{sk}")
123 ld = (f"sigc d kb{kb} kc{kcont} w{wd} p{pk - nb // 2} pd{round(100 * float(du[pk]))}"
124     f" ps{round(10 * np.log10(p95[pk] + 1e-30))} q{qo} qd{qd} qs{qs}")
125 if step == 0:                                # 요약 4줄 - 이것만 받아쳐서 회신한다
126     for s in (la, lb, lc, ld):
127         print(f"{s} #{check_code(s)}")
128     sys.exit()
129
130 # — 여기부터는 그림 단계(1~6) 전용이다. 요약 4줄만 쓸 거면 여기서 멈춰도 된다 —
131
132 try:
133     import matplotlib.pyplot as plt
134     plt = None if plt.get_backend().lower() == "agg" else plt    # 무화면(ssh)이면 ASCII 로
135 except ImportError:
136     plt = None
137
138
139 def pool(a, m):
140     a = np.asarray(a, float)
141     k = max(1, a.shape[-1] // m)
142     return a[..., :(a.shape[-1] // k) * k].reshape(*a.shape[:-1], -1, k).max(-1)
143
144
145 def curve(y, marks):
146     if plt:
147         plt.plot(y)
148         for la, v in marks:
149             plt.axhline(v, ls="--", label=f"{la} {v:.2f}")
150         plt.legend()
151         return plt.show()
152     # 얼마나 최소·최대를 같이 그린다: max 만 그리면 버스트 사이의 골이 지워져 계단이 통짜
153     # 블록으로 보이고, 관측자가 "평평하다(=베토)" 라고 정반대로 회신하게 된다.
154     kk = max(1, y.size // 100)
155     y2 = y[:y.size // kk * kk].reshape(-1, kk)
156     ylo, yhi = y2.min(1), y2.max(1)
157     bot = float(min(ylo.min(), *(v for _, v in marks)))
158     st = (float(max(yhi.max(), *(v for _, v in marks))) - bot) / 18 or 1.0
159     rl, rh = np.round((ylo - bot) / st).astype(int), np.round((yhi - bot) / st).astype(int)
160     rm = {}
161     for la, v in marks:
162         rm.setdefault(round((v - bot) / st), []).append(f"{la} {v:.2f}")
163     for r in range(18, -1, -1):
164         bg = "-" if r in rm else " "                # 문턱은 가로 전체 선으로 (겹치면 라벨 합침)
165         print("".join("#" if rl[i] >= r else ("+" if rh[i] >= r else bg)
166             for i in range(rl.size)) + "|" + " / ".join(rm.get(r, [])))
167     print("#=이 구간 내내 그 위 · +=봉우리만 그 위 · -=문턱선")
168
169
170 def image(img):
171     if plt:
172         plt.imshow(img, aspect="auto", origin="lower", interpolation="nearest")
173         return plt.show()
174     sm = pool(pool(img, 100).T, 32).T
175     hi = sm.max() + 1e-30
176     lo = max(float(np.percentile(sm, 5)), hi - 4)    # 표시폭 4 decades 고정 - real 캡처는 빈
177     # 음수 반쪽이 1e-16 이라 5퍼센타일을 쓰면 눈금이 14 decades 로 늘어나 통짜 벽이 된다
178     for row in np.clip((sm - lo) / (hi - lo) * 9.99, 0, 9).astype(int)[::-1]:
179         print("".join(" .:~+*#%@"[i] for i in row))
180

```

```

181
182 if step == 1:
183     print(f"n {n} fs {fs:.0f} 길이 {n / fs:.3f}s 형식 {'iq' if np.iscomplexobj(x0) else 'real'}")
184     print(f"rms {rms:.4f} dc {100 * dc:.1f}% 클리핑 {1000 * cp:.1f}% 피크비 {float(sp):.0f}dB")
185 elif step == 2:
186     curve(lp, [("otsu", et)])
187     print(f"포락선 분리도 {ev:.3f} decades - 0.12 미만이면 find_bursts 는 통짜 [(0,n)] (베토)")
188     print(f"포락선 듀티 {100 * ed:.0f}% (버스트가 계단으로 보여야 정상)")
189 elif step == 3:
190     image(np.log10(Ps + 1e-20))
191     print("위터폴 절대전력 (아래=-fs/2, 위=+fs/2) - 가로로 끊기지 않는 띠 = 연속 방사체")
192 elif step == 4:
193     image(np.log10(np.fft.fftshift(r, 0) + 1e-30))
194     print("빈별 바닥 대비 비율 - find_bursts 가 보는 그림. 세기가 일정한 연속 방사체는 여기서"
195           " 사라지지만, 세기가 흔들리면 가짜 버스트로 남는다 (d 줄 pd 로 확인)")
196 elif step == 5:
197     curve(sc, [("base", base), ("hi", hiv), ("c_noise", cn)])
198     print(f"열 점수 - hi 위 봉우리가 버스트 후보. 런 {len(rr)}개, base {base:.2f}, cn {cn:.2f}")
199 elif step == 6:
200     for (s, e), (a0, a1), sg in zip(rr, spans, segs, strict=True):
201         print(f"열 {s}-{e} 샘플 {a0}-{a1} 높이 {10 * (float(sc[s:e].max()) - base):.0f}dB"
202               f" 피크/평균 {float(sg.max() / (sg.mean() + 1e-30)):.0f} (60 초과=스파이크 탈락)")
203     print(f"위는 병합/가드 전 원시 후보 {len(rr)}개 - 아래가 실제로 분석에 쓰이는 답이다:")
204     fb = dsp.find_bursts(x, fs)
205     print(f"find_bursts → {fb[:6]}{'...' if len(fb) > 6 else ''}"
206           + (" ※ 통짜 = 버스트 미검출" if fb == [(0, n)] else f" 버스트 {len(fb)}개"))

```

```

1  """sa.py - 종합 관찰 프로브 (장비 필사용, signus/ 옆에 저장). 한 번에:
2  디버그 출력(포락선·버스트 표) + <캡처>.sa.png (스펙트로그램 + 검출 띠 + 버스트 번호 +
3  상위 버스트별 [PSD |  $x^2$  |  $x^4$  |  $x^8$  | AM검파] 판독 행 - 바늘이 처음 서는 판이 변조).
4  python3 sa.py kat # 자기검증: 표지 기대 줄과 비교, sa-kat.png 생성
5  python3 sa.py <캡처파일> [K] # 판독할 버스트 수 K (기본 4, 에너지 상위부터)
6   $x^m$  판은 접힌 축( $0 \sim fs/2$ , 음수 주파수를 겹쳐 최대값) - 바늘 위치가  $m \cdot fc \pmod{fs}$  다.
7  """
8  import struct
9  import sys
10 import zlib
11
12 import numpy as np
13 from scipy.signal import stft, welch
14
15 from signus import dsp, sigio
16 from signus.chirp import is_chirp, sweeps_band
17 from signus.cli import check_code
18 from signus.fsk import fsk_gate
19
20 GLYPH = {"0": (14, 17, 19, 21, 25, 17, 14), "1": (4, 12, 4, 4, 4, 4, 14),
21          "2": (14, 17, 1, 2, 4, 8, 31), "3": (31, 2, 4, 2, 1, 17, 14),
22          "4": (2, 6, 10, 18, 31, 2, 2), "5": (31, 16, 30, 1, 1, 17, 14),
23          "6": (6, 8, 16, 30, 17, 17, 14), "7": (31, 1, 2, 4, 8, 8, 8),
24          "8": (14, 17, 17, 14, 17, 17, 14), "9": (14, 17, 17, 15, 1, 2, 12)}
25
26
27 def draw_text(img, row, col, text, s=2):
28     for ch in text:
29         for r, bits in enumerate(GLYPH[ch]):
30             for c in range(5):
31                 if bits >> (4 - c) & 1:
32                     img[row + r * s:row + r * s + s, col + c * s:col + c * s + s] = 1.0
33             col += 6 * s
34
35
36 def png_gray(img, path):
37     img = np.clip(img * 255, 0, 255).astype(np.uint8)
38     h, w = img.shape
39
40     def chunk(tag, data):
41         c = tag + data
42         return struct.pack(">I", len(data)) + c + struct.pack(">I", zlib.crc32(c))
43
44     raw = b"".join(b"\x00" + img[r].tobytes() for r in range(h))
45     with open(path, "wb") as fh:
46         fh.write(b"\x89PNG\r\n\x1a\n"
47                 + chunk(b"IHDR", struct.pack(">IBBBBB", w, h, 8, 0, 0, 0, 0))
48                 + chunk(b>IDAT", zlib.compress(raw, 6)) + chunk(b"IEND", b""))
49
50
51 def panel(fvals, dbvals, wp=500, hp=120, grid=()):
52     """스펙트럼 곡선 한 판 - 바늘 보존을 위해 표시 칸마다 최대값 풀림."""
53     cols = np.clip((fvals / fvals.max() * (wp - 1)).astype(int), 0, wp - 1)
54     cur = np.full(wp, -1e9)
55     np.maximum.at(cur, cols, dbvals)
56     bad = cur < -1e8
57     cur[bad] = np.interp(np.flatnonzero(bad), np.flatnonzero(~bad), cur[~bad])
58     g_lo, g_hi = np.percentile(cur, 5) - 2, cur.max() + 2
59     img = np.zeros((hp, wp))
60     for gf in grid:

```

```

61         img[:,6, int(gf / fvals.max() * (wp - 1))] = 0.35
62     rows = ((hp - 1) * (1 - np.clip((cur - g_lo) / (g_hi - g_lo), 0, 1))).astype(int)
63     for cx in range(wp):
64         r0, r1 = (rows[cx], rows[cx - 1]) if cx else (rows[0], rows[0])
65         img[min(r0, r1):max(r0, r1) + 1, cx] = 1.0
66     return img
67
68
69 def peak(fv, dbv, fmin=20.0):
70     """판 하나의 최대 바늘: (주파수 Hz, 중앙값 대비 돌출 dB) -- 숫자로 받아칠 수 있게."""
71     ok = fv >= fmin
72     i = int(np.argmax(dbv[ok]))
73     return round(float(fv[ok][i]), round(float(dbv[ok][i] - np.median(dbv[ok]))))
74
75
76 def verdict(z, fs, d2, d4, d8, dam):
77     """바늘 돌출로 버스트의 변조 부류를 판정 -- 문턱은 합성 배터리로 캘리브레이션
78     (bpsk d2 54 / qpsk d4 38 / 16qam d4 30 / 8psk d8 29 / fsk am 17, 광대역 잡음꼴 무바늘)."""
79     if is_chirp(z, fs) and sweeps_band(z, fs):
80         return "chirp" # 처프/LoRa 류 -- 성상도 복조 대상 아님
81     if fsk_gate(z, fs):
82         return "fsk"
83     if d2 >= 35 and d2 >= d4 + 5:
84         return "bpsk"
85     if d4 >= 28:
86         return "qpsk" # qpsk 또는 사각 QAM (4차에서 무너짐)
87     if d8 >= 24:
88         return "8psk"
89     return "lin" if dam >= 25 else "flat" # lin=선형(차수불명) / flat=바늘 없음(잡음꼴)
90
91
92 def burst_row(z, fs, label):
93     """한 버스트의 판독 행: [PSD | x^2 | x^4 | x^8 | AM]. x^m 은 음수 주파수를 접는다."""
94     grid = tuple(fs / 2 * k / 5 for k in (1, 2, 3, 4))
95     fr, praw = welch(z.real, fs=fs, nperseg=min(2048, z.size))
96     ps = [panel(fr, 10 * np.log10(praw + 1e-18), grid=grid)]
97     nfft, nums = 1 << 15, []
98     f = np.fft.fftfreq(nfft, 1 / fs)
99     half = nfft // 2
100    for m in (2, 4, 8):
101        mag = np.abs(np.fft.fft((z ** m) * np.hanning(z.size), nfft))
102        fold = np.maximum(mag[:half], np.r_[mag[0], mag[1:half][::-1] * 0
103            + mag[half + 1][::-1]])
104        ps.append(panel(f[:half], 20 * np.log10(fold + 1e-12), grid=grid))
105        pf, pd = peak(f[:half], 20 * np.log10(fold + 1e-12))
106        nums.append(f"p{m} f{pf} d{pd}")
107    u = np.abs(z) ** 2
108    u = u - u.mean()
109    su = np.abs(np.fft.rfft(u * np.hanning(u.size), nfft))
110    ps.append(panel(np.fft.rfftfreq(nfft, 1 / fs), 20 * np.log10(su + 1e-12), grid=grid))
111    af, ad = peak(np.fft.rfftfreq(nfft, 1 / fs), 20 * np.log10(su + 1e-12))
112    nums.append(f"am f{af} d{ad}")
113    lab = verdict(z, fs, int(nums[0].split("d")[1]), int(nums[1].split("d")[1]),
114        int(nums[2].split("d")[1]), ad)
115    nums.insert(0, lab)
116    div = np.full((120, 4), 0.5)
117    row = np.hstack(sum([p, div] for p in ps[:-1]), []) + [ps[-1]]
118    tag = np.zeros((120, 26))
119    draw_text(tag, 4, 4, label)
120    return np.hstack([tag, row]), " ".join(nums)

```

```

121
122
123 if len(sys.argv) < 2:
124     raise SystemExit("사용법: python3 sa.py kat | python3 sa.py <캡처파일> [버스트수 K]")
125 arg = sys.argv[1]
126 kmax = int(sys.argv[2]) if len(sys.argv) > 2 else 4
127 if arg == "kat":
128     fs, n = 10000.0, 80000
129     rng = np.random.default_rng(21) # 시드 고정 -- numpy 가 재현을 보장한다
130     x0 = 0.18 * rng.standard_normal(n)
131     t = np.arange(15000) / fs
132     for s0, fc, ph in ((12000, 550.0, 2), (50000, 550.0, 4)): # 앞=BPSK, 뒤=QPSK
133         up = np.zeros(15000, complex)
134         up[:,34] = np.exp(2j * np.pi * rng.integers(0, ph, 442) / ph)
135         sym = np.convolve(up, np.hanning(68), "same") # 간이 펄스 성형
136         sym /= np.sqrt(np.mean(np.abs(sym) ** 2))
137         x0[s0:s0 + 15000] += (sym * np.exp(2j * np.pi * fc * t)).real * np.sqrt(2)
138     name = "sa-kat"
139 else:
140     x0, meta = sigio.read(arg)
141     fs = meta.fs
142     name = arg + ".sa"
143 xa = dsp.analytic(x0)
144 xa = xa - xa.mean()
145 n = xa.size
146 fb = dsp.find_bursts(xa, fs)
147 full = fb == [(0, n)]
148 pw = np.abs(xa) ** 2
149 lp = np.log10(np.maximum(np.convolve(pw, np.ones(64) / 64, "same"), 0) + 1e-20)
150 et = np.percentile(lp, 50)
151 ev = float(lp[lp >= et].mean() - lp[lp < et].mean())
152 print(f"\n {n} fs {fs:.0f} 길이 {n / fs:.3f}s 포락선 분리(중앙 기준) {ev:.2f} decades")
153 print(f"find_bursts → {'통짜 [(0,n)] = 미검출' if full else str(len(fb)) + '개'}")
154 order = sorted(range(len(fb)), key=lambda i: -float(pw[fb[i][0]:fb[i][1]].sum()))
155 pick = sorted(order[:kmax]) if not full else []
156 for i, (s0, e0) in enumerate(fb if not full else []):
157     star = " ★판독" if i in pick else ""
158     print(f" {i + 1:2} 샘플 [{s0:>8}, {e0:>8}] {1000 * (e0 - s0) / fs:8.1f} ms"
159           f" 전력 {10 * np.log10(pw[s0:e0].mean() / (pw.mean() + 1e-30) + 1e-30):+5.1f} dB{star}")
160
161 x_disp = x0.real if np.iscomplexobj(x0) else x0
162 _, _, z = stft(x_disp, fs=fs, window="hamming", nperseg=256, noverlap=128,
163               return_onesided=True, boundary=None, padded=False)
164 db = 10 * np.log10(np.abs(z) ** 2 + 1e-12)
165 lo = float(np.percentile(db, 25))
166 rg = float(max(10.0, min(35.0, np.percentile(db, 99.5) - lo)))
167 spec = np.clip((db - lo) / rg, 0, 1)[::-1]
168 kp = max(1, spec.shape[1] // 4000)
169 spec = spec[:, :spec.shape[1] // kp * kp].reshape(spec.shape[0], -1, kp).max(2)
170 pw_row = 26 + 5 * 500 + 4 * 4 # 판독 행 폭 -- 스트립을 여기에 맞춰 늘린다
171 fx = max(1, pw_row // spec.shape[1])
172 spec = np.repeat(spec, fx, axis=1)
173 ncol = spec.shape[1]
174 mark = np.zeros((18, ncol))
175 lane = np.zeros(ncol)
176 for i, (s0, e0) in enumerate(fb if not full else []):
177     c0 = (s0 // 128) // kp * fx
178     c1 = min(ncol - 1, (e0 // 128) // kp * fx)
179     lane[c0:c1 + 1] = 1.0
180     draw_text(mark, 2, min(c0 + 2, ncol - 14 * len(str(i + 1))), str(i + 1))

```

```

181 rows = [mark, spec, np.full((2, ncol), 0.35), np.tile(lane, (10, 1)),
182         np.full((6, ncol), 0.0)]
183 width = max(ncol, pw_row)
184 rows = [np.pad(r, ((0, 0), (0, width - r.shape[1]))) for r in rows]
185 for i in pick:
186     r, num = burst_row(xa[fb[i][0]:fb[i][1]], fs, str(i + 1))
187     ln = f"sa b{i + 1} {num}" # 바늘 주파수·돌출 -- 받아칠 판독 숫자줄
188     print(f"{ln} #{check_code(ln)}")
189     rows += [np.pad(r, ((0, 0), (0, width - r.shape[1]))), np.full((6, width), 0.0)]
190 png_gray(np.vstack(rows), name + ".png")
191 line = (f"sa {'kat' if arg == 'kat' else 'cap'} n{n} f{fs:.0f} s{n / fs:.1f}"
192        f" fb{0 if full else len(fb)} ev{round(100 * ev)}")
193 print(f"{line} #{check_code(line)}")
194 print(f"{name}.png 저장 - 위: 스펙트로그램+검출 띠+버스트 번호, 아래: ★버스트별"
195        " [PSD|x2|x4|x8|AM] (x2 바늘=BPSK, x4=QPSK, x8=8PSK, AM 봉우리=심볼레이트)")

```